ARCHIVO FUENTE: argon/src/Argon.Engine/Program.cs (225 lineas) | SHA-256: b5afc9fd99d0a54bd1d50e389d2b7e14...

```
using System.Globalization;
using System.Text;
using System.Text.Json;
using Argon.Engine;

var jsonOptions = new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    PropertyNameCaseInsensitive = true,
    WriteIndented = true
};

var repoRoot = Directory.GetCurrentDirectory();

if (args.Length > 0 && string.Equals(args[0], "atlas", StringComparison.OrdinalIgnoreCase))
{
    var c1Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-001-insumision-dynamics", "dynamics.p...
    var c2Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "daisy-run"...
    var c3Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "challenger...

    var p1 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c1Path), jso...
    var p2 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c2Path), jso...
    var p3 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c3Path), jso...

    var entries = new List<(string WorkId, string Title, string Domain, string Language, string License, N...
    {
        ("CORPUS-001-INSUMISION-1994", "La Mili y la Insumisión - Public Broadcast Debate (Spain, 1994)", ...
        ("CORPUS-002-LBJ-DAISY-1964", "Lyndon B. Johnson 'Daisy' Nuclear Campaign Spot (USA, 1964)", "Acut...
        ("CORPUS-003-REAGAN-CHALLENGER-1986", "Ronald Reagan Address on the Space Shuttle Challenger Disas...
    };

    var atlas = AtlasComparator.BuildAtlas(entries);
    var atlasOutDir = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas");
    Directory.CreateDirectory(atlasOutDir);

    var reportPath = Path.Combine(atlasOutDir, "atlas.report.json");
    var htmlPath = Path.Combine(atlasOutDir, "atlas-dashboard.html");

    await File.WriteAllTextAsync(reportPath, JsonSerializer.Serialize(atlas, jsonOptions));
    await File.WriteAllTextAsync(htmlPath, AtlasComparator.RenderAtlasHtml(atlas));

    Console.WriteLine($"[Argon.Engine Atlas] Synthesized Cross-Domain Narrative Phase-Space Atlas ({atlas....
    foreach (var d in atlas.PairwiseDistances)
    {
        Console.WriteLine($"  -> {d.WorkA} vs. {d.WorkB}: 5D Metric Dist = {d.MetricSpaceEuclideanDistance...
    }
    Console.WriteLine($"[Argon.Engine Atlas] Dashboard written to: {htmlPath}");
    return 0;
}

if (args.Length > 0 && string.Equals(args[0], "control", StringComparison.OrdinalIgnoreCase))
{
    var c1Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-001-insumision-dynamics", "dynamics.p...
    var c2Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "daisy-run"...
    var c3Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "challenger...

    var p1 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c1Path), jso...
    var p2 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c2Path), jso...
    var p3 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c3Path), jso...

    var report = CounterfactualController.RunFullControlExperiment(p1, p2, p3);
    var controlOutDir = Path.Combine(repoRoot, "argon", "experiments", "EXP-003-counterfactual-control");
    Directory.CreateDirectory(controlOutDir);

    var reportPath = Path.Combine(controlOutDir, "control.report.json");
    var htmlPath = Path.Combine(controlOutDir, "control-dashboard.html");

    await File.WriteAllTextAsync(reportPath, JsonSerializer.Serialize(report, jsonOptions));
    await File.WriteAllTextAsync(htmlPath, CounterfactualController.RenderControlHtml(report));

    var s1 = report.Study1MontageOptimization;
    var s2 = report.Study2ShockImpulseControl;
    var s3 = report.Study3InertialShielding;

    Console.WriteLine($"[Argon.Engine Control v0.3] Solved {s1.TotalPermutationsEvaluated} montage permuta...
    Console.WriteLine($"  -> Reachable B1 (Anti-Militarism) Span: [{s1.ReachableStateMin["B1_ANTIMILITARIS...
    Console.WriteLine($"  -> Optimal Civic Equilibrium Order: {string.Join(" -> ", s1.OptimalCivicEquilibr...
    Console.WriteLine($"[Argon.Engine Control v0.3] CORPUS-002 Daisy Non-Verbal Shock Leverage: {s2.NonVer...
    Console.WriteLine($"[Argon.Engine Control v0.3] CORPUS-003 Challenger Institutional Stabilization Marg...
    Console.WriteLine($"[Argon.Engine Control v0.3] Control Lab Dashboard written to: {htmlPath}");
    return 0;
}

if (args.Length > 0 && string.Equals(args[0], "sysid", StringComparison.OrdinalIgnoreCase))
{
    var c1Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-001-insumision-dynamics", "dynamics.p...
    var c2Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "daisy-run"...
    var c3Path = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "challenger...
```

Copyright (c) 2026 Jair Ivan Rojas Ruiz (ORCID: 0009-0003-1109-4583 | DOI: 10.5281/zenodo.23987656) - Registro INDAUTOR

```
 89:
 90:     var p1 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c1Path), jso...
 91:     var p2 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c2Path), jso...
 92:     var p3 = JsonSerializer.Deserialize<NarrativeDynamicsProfile>(await File.ReadAllTextAsync(c3Path), jso...
 93:
 94:     var report = SystemIdentifier.RunFullSystemIdentification(p1, p2, p3);
 95:     var sysidOutDir = Path.Combine(repoRoot, "argon", "experiments", "EXP-004-system-identification");
 96:     Directory.CreateDirectory(sysidOutDir);
 97:
 98:     var reportPath = Path.Combine(sysidOutDir, "sysid.report.json");
 99:     var htmlPath = Path.Combine(sysidOutDir, "sysid-dashboard.html");
100:
101:     await File.WriteAllTextAsync(reportPath, JsonSerializer.Serialize(report, jsonOptions));
102:     await File.WriteAllTextAsync(htmlPath, SystemIdentifier.RenderSysIdHtml(report));
103:
104:     var rec = report.EmpiricalNoisyRecovery25dB;
105:     var meanLocoR2 = report.LeaveOneCorpusOutFolds.Average(f => f.OutOfSampleR2);
106:     Console.WriteLine($"[Argon.Engine SysID v0.4] Identified 4-channel state-space parameters across {repo...
107:     Console.WriteLine($"  -> True Alpha: (0.3400, 0.2200, 0.2900, 0.1500), Beta0 = 0.1800");
108:     Console.WriteLine($"  -> Learned (25 dB SNR): ({rec.AlphaAcoustic:F4}, {rec.AlphaVisual:F4}, {rec.Alph...
109:     Console.WriteLine($"  -> Relative Parameter Error ||Theta_hat - Theta*||_2: {rec.RelativeParameterErro...
110:     Console.WriteLine($"  -> Leave-One-Corpus-Out (LOCO) Out-of-Sample R^2: {meanLocoR2:F4}");
111:     Console.WriteLine($"[Argon.Engine SysID v0.4] System Identification Dashboard written to: {htmlPath}")...
112:     return 0;
113: }
114:
115: if (args.Length > 0 && string.Equals(args[0], "paper", StringComparison.OrdinalIgnoreCase))
116: {
117:     var atlasPath = Path.Combine(repoRoot, "argon", "experiments", "EXP-002-public-domain-atlas", "atlas.r...
118:     var ctrlPath = Path.Combine(repoRoot, "argon", "experiments", "EXP-003-counterfactual-control", "contr...
119:
120:     var atlas = JsonSerializer.Deserialize<CrossDomainAtlasReport>(await File.ReadAllTextAsync(atlasPath),...
121:     var ctrl = JsonSerializer.Deserialize<CounterfactualControlReport>(await File.ReadAllTextAsync(ctrlPat...
122:
123:     var latexDir = Path.Combine(repoRoot, "publications", "latex");
124:     Directory.CreateDirectory(latexDir);
125:
126:     var printHtmlPath = Path.Combine(latexDir, "PAPER-001-NSE-FOUNDATION-PRINT.html");
127:     var pdfPath = Path.Combine(repoRoot, "publications", "PAPER-001-NSE-FOUNDATION.pdf");
128:
129:     await File.WriteAllTextAsync(printHtmlPath, PaperManuscriptRenderer.RenderPrintReadyHtml(atlas, ctrl))...
130:     await File.WriteAllBytesAsync(pdfPath, PaperManuscriptRenderer.RenderNativePdfBytes(atlas, ctrl));
131:
132:     Console.WriteLine($"[Argon.Engine Paper] Print-ready manuscript HTML written to: {printHtmlPath}");
133:     Console.WriteLine($"[Argon.Engine Paper] Native Vector PDF 1.4 written to: {pdfPath}");
134:     return 0;
135: }
136:
137: if (args.Length > 0 && string.Equals(args[0], "indautor", StringComparison.OrdinalIgnoreCase))
138: {
139:     IndautorPackageGenerator.GenerateIndautorPdfs(repoRoot);
140:     Console.WriteLine($"[Argon.Engine INDAUTOR] Generated official registration PDFs in: {Path.Combine(rep...
141:     return 0;
142: }
143:
144: var profileJsonPath = args.Length > 0
145:     ? Path.GetFullPath(args[0])
146:     : Path.Combine(repoRoot, "data", "processed", "runs", "06-FCE-V-C110-007-C-profile", "profile.json");
147:
148: var transcriptJsonPath = args.Length > 1
149:     ? Path.GetFullPath(args[1])
150:     : Path.Combine(repoRoot, "argon", "experiments", "EXP-001-insumision-dynamics", "transcript.aligned.js...
151:
152: var outputDir = args.Length > 2
153:     ? Path.GetFullPath(args[2])
154:     : Path.Combine(repoRoot, "argon", "experiments", "EXP-001-insumision-dynamics");
155:
156: if (!File.Exists(profileJsonPath))
157:     throw new FileNotFoundException($"Base profile not found: {profileJsonPath}");
158: if (!File.Exists(transcriptJsonPath))
159:     throw new FileNotFoundException($"Aligned transcript not found: {transcriptJsonPath}");
160:
161: var profileDir = Path.GetDirectoryName(profileJsonPath)!;
162: using var baseProfileDoc = JsonDocument.Parse(await File.ReadAllTextAsync(profileJsonPath));
163: var root = baseProfileDoc.RootElement;
164: var durationSeconds = root.GetProperty("source").GetProperty("durationSeconds").GetDouble();
165: var wavRelative = root.GetProperty("audio").GetProperty("path").GetString()
166:                   ?? throw new InvalidDataException("Profile has no extracted audio WAV.");
167: var wavPath = Path.Combine(profileDir, wavRelative);
168:
169: var frameList = new List<(int Index, double Timestamp, double Start, double End, string Image, string Rast...
170: var idx = 0;
171: foreach (var f in root.GetProperty("frames").EnumerateArray())
172: {
173:     frameList.Add((
174:         idx++,
175:         f.GetProperty("timestampSeconds").GetDouble(),
176:         f.GetProperty("windowStartSeconds").GetDouble(),
177:         f.GetProperty("windowEndSeconds").GetDouble(),
178:         f.GetProperty("image").GetString()!,
```

```
        f.GetProperty("colorRaster").GetString()!));
}

var transcriptDoc = JsonSerializer.Deserialize<TranscriptDocument>(
    await File.ReadAllTextAsync(transcriptJsonPath), jsonOptions)
    ?? throw new InvalidDataException("Failed to parse transcript.aligned.json");

Console.WriteLine($"[Argon.Engine] Analyzing {frameList.Count} windows ({durationSeconds:F2}s) from {Path....

var windows = frameList.Select(f => (f.Start, f.End)).ToList();
var acousticMetrics = AcousticAnalyzer.AnalyzeWav(wavPath, windows);
var visualMetrics = VisualDynamicsAnalyzer.AnalyzeRasters(
    profileDir,
    frameList.Select(f => (f.Index, f.Timestamp, f.Image, f.Raster)).ToList());
var linguisticMetrics = LinguisticAnalyzer.AnalyzeWindows(transcriptDoc, windows);

var dynamicsProfile = NarrativeMetricsEngine.ComputeFullProfile(
    transcriptDoc,
    durationSeconds,
    acousticMetrics,
    visualMetrics,
    linguisticMetrics);

Directory.CreateDirectory(outputDir);

var outProfilePath = Path.Combine(outputDir, "dynamics.profile.json");
await File.WriteAllTextAsync(outProfilePath, JsonSerializer.Serialize(dynamicsProfile, jsonOptions));

var csv = new StringBuilder();
csv.AppendLine("window_index,start_s,end_s,speaker_id,dbfs,vad_ratio,visual_lum,visual_sat,visual_jsd_bits...
foreach (var p in dynamicsProfile.Timeline)
{
    csv.AppendLine(FormattableString.Invariant(
        $"{p.WindowIndex},{p.StartSeconds:F2},{p.EndSeconds:F2},{p.SpeakerId},{p.Acoustic.DbFs:F2},{p.Acou...
}
var outCsvPath = Path.Combine(outputDir, "telemetry.csv");
await File.WriteAllTextAsync(outCsvPath, csv.ToString());

var outHtmlPath = Path.Combine(outputDir, "dashboard.html");
await File.WriteAllTextAsync(outHtmlPath, DashboardRenderer.RenderHtml(dynamicsProfile, transcriptDoc, pro...

Console.WriteLine($"[Argon.Engine] Mean Energy: {dynamicsProfile.GlobalIndices.MeanNarrativeEnergy:F2} | P...
Console.WriteLine($"[Argon.Engine] Final Stance Entropy: {dynamicsProfile.GlobalIndices.FinalNarrativeEntr...
Console.WriteLine($"[Argon.Engine] Mean Inertia: {dynamicsProfile.GlobalIndices.MeanNarrativeInertia:F4} |...
Console.WriteLine($"[Argon.Engine] Scene Cuts Detected: {dynamicsProfile.GlobalIndices.DetectedSceneCutsCo...
Console.WriteLine($"[Argon.Engine] Artifacts written to: {outputDir}");
return 0;
```

ARCHIVO FUENTE: argon/src/Argon.Engine/AcousticAnalyzer.cs (122 lineas) | SHA-256: d37de9fecb2d6815bd4d765870d37f05...

```
namespace Argon.Engine;

public sealed record AcousticWindowMetrics(
    int WindowIndex,
    double StartSeconds,
    double EndSeconds,
    double RmsAmplitude,
    double DbFs,
    double PeakAmplitude,
    double ZeroCrossingRate,
    double SpeechActivityRatio,
    double DynamicRangeDb,
    double NormalizedIntensity);

public static class AcousticAnalyzer
{
    public static List<AcousticWindowMetrics> AnalyzeWav(
        string wavPath,
        IReadOnlyList<(double StartSeconds, double EndSeconds)> windows)
    {
        var bytes = File.ReadAllBytes(wavPath);
        var dataOffset = FindDataChunkOffset(bytes, out var dataLength, out var sampleRate);
        var totalSamples = dataLength / 2;
        var samples = new float[totalSamples];
        for (var i = 0; i < totalSamples; i++)
        {
            short pcm = BitConverter.ToInt16(bytes, dataOffset + i * 2);
            samples[i] = pcm / 32768f;
        }

        // First pass: compute raw metrics per window
        var raw = new List<(int idx, double start, double end, double rms, double dbfs, double peak, doubl...
        foreach (var (w, idx) in windows.Select((w, i) => (w, i)))
        {
            var s0 = Math.Clamp((int)Math.Round(w.StartSeconds * sampleRate), 0, totalSamples);
            var s1 = Math.Clamp((int)Math.Round(w.EndSeconds * sampleRate), s0, totalSamples);
            var count = Math.Max(1, s1 - s0);

            double sumSq = 0;
            double peak = 0;
```

```
            long zeroCrossings = 0;
            for (var i = s0; i < s1; i++)
            {
                var v = (double)samples[i];
                sumSq += v * v;
                var abs = Math.Abs(v);
                if (abs > peak) peak = abs;
                if (i > s0 && ((samples[i - 1] >= 0 && samples[i] < 0) || (samples[i - 1] < 0 && samples[i...
                {
                    zeroCrossings++;
                }
            }

            var rms = Math.Sqrt(sumSq / count);
            var dbfs = 20.0 * Math.Log10(Math.Max(rms, 1e-5));

            // Sub-window 100ms analysis for Speech Activity Ratio (VAD) and Dynamic Range
            var subLen = Math.Max(1, sampleRate / 10);
            var subDbs = new List<double>();
            for (var p = s0; p < s1; p += subLen)
            {
                var endP = Math.Min(s1, p + subLen);
                double subSq = 0;
                for (var k = p; k < endP; k++) subSq += (double)samples[k] * samples[k];
                var subRms = Math.Sqrt(subSq / Math.Max(1, endP - p));
                subDbs.Add(20.0 * Math.Log10(Math.Max(subRms, 1e-5)));
            }

            subDbs.Sort();
            var p10 = subDbs[(int)Math.Floor(0.10 * (subDbs.Count - 1))];
            var p90 = subDbs[(int)Math.Floor(0.90 * (subDbs.Count - 1))];
            var dynRange = Math.Max(0, p90 - p10);

            // Speech activity threshold: sub-frames above -31 dBFS (above campus background murmur)
            var activeSub = subDbs.Count(d => d > -31.0);
            var vad = (double)activeSub / Math.Max(1, subDbs.Count);
            var zcr = (double)zeroCrossings / count;

            raw.Add((idx, w.StartSeconds, w.EndSeconds, rms, dbfs, peak, zcr, vad, dynRange));
        }

        var minDb = raw.Min(r => r.dbfs);
        var maxDb = raw.Max(r => r.dbfs);
        var span = Math.Max(1.0, maxDb - minDb);

        return raw.Select(r =>
        {
            var normLevel = Math.Clamp((r.dbfs - minDb) / span, 0.0, 1.0);
            var normDyn = Math.Clamp(r.dynDb / 18.0, 0.0, 1.0);
            var intensity = Math.Clamp(0.55 * normLevel + 0.30 * r.vad + 0.15 * normDyn, 0.0, 1.0);
            return new AcousticWindowMetrics(
                r.idx,
                Math.Round(r.start, 3),
                Math.Round(r.end, 3),
                Math.Round(r.rms, 5),
                Math.Round(r.dbfs, 2),
                Math.Round(r.peak, 4),
                Math.Round(r.zcr, 4),
                Math.Round(r.vad, 3),
                Math.Round(r.dynDb, 2),
                Math.Round(intensity, 4));
        }).ToList();
    }

    private static int FindDataChunkOffset(byte[] wav, out int dataLength, out int sampleRate)
    {
        sampleRate = BitConverter.ToInt32(wav, 24);
        var offset = 12;
        while (offset + 8 <= wav.Length)
        {
            var chunkId = System.Text.Encoding.ASCII.GetString(wav, offset, 4);
            var chunkSize = BitConverter.ToInt32(wav, offset + 4);
            if (chunkId == "data")
            {
                dataLength = Math.Min(chunkSize, wav.Length - (offset + 8));
                return offset + 8;
            }
            offset += 8 + chunkSize + (chunkSize % 2);
        }
        throw new InvalidDataException("WAV file has no data chunk.");
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/VisualDynamicsAnalyzer.cs (125 lineas) | SHA-256: cefe0908143520b18e3d7d96c7b58d2f...

```
namespace Argon.Engine;

public sealed record VisualWindowMetrics(
    int WindowIndex,
    double TimestampSeconds,
    string FramePath,
    double MeanLuminance,
    double LuminanceContrastStd,
    double MeanSaturation,
    double WarmthIndex,
    double ChromaticEntropyBits,
    double JensenShannonDivergenceFromPrev,
    bool IsSceneCutDetected,
    double VisualSalienceScore);

public static class VisualDynamicsAnalyzer
{
    public static List<VisualWindowMetrics> AnalyzeRasters(
        string profileDirectory,
        IReadOnlyList<(int Index, double Timestamp, string FrameImage, string RasterPath)> frames)
    {
        var results = new List<VisualWindowMetrics>();
        double[]? prevDistribution = null;

        foreach (var f in frames)
        {
            var fullRasterPath = Path.Combine(profileDirectory, f.RasterPath);
            var rgb = File.ReadAllBytes(fullRasterPath);
            if (rgb.Length != 64 * 64 * 3)
                throw new InvalidDataException($"Expected 12288 bytes in {fullRasterPath}, got {rgb.Length...

            const int pixelCount = 4096;
            var distribution = new double[512];
            double sumLum = 0;
            double sumLumSq = 0;
            double sumSat = 0;
            double sumWarmth = 0;

            for (var i = 0; i < rgb.Length; i += 3)
            {
                var r = rgb[i] / 255.0;
                var g = rgb[i + 1] / 255.0;
                var b = rgb[i + 2] / 255.0;

                // ITU-R BT.709 perceptual luminance
                var y = 0.2126 * r + 0.7152 * g + 0.0722 * b;
                sumLum += y;
                sumLumSq += y * y;

                var max = Math.Max(r, Math.Max(g, b));
                var min = Math.Min(r, Math.Min(g, b));
                var sat = max > 1e-5 ? (max - min) / max : 0.0;
                sumSat += sat;

                // Red-Blue warmth balance in [-1, 1]
                sumWarmth += (r - b);

                var bin = ((rgb[i] / 32) << 6) | ((rgb[i + 1] / 32) << 3) | (rgb[i + 2] / 32);
                distribution[bin] += 1.0 / pixelCount;
            }

            var meanLum = sumLum / pixelCount;
            var varLum = Math.Max(0.0, (sumLumSq / pixelCount) - (meanLum * meanLum));
            var stdLum = Math.Sqrt(varLum);
            var meanSat = sumSat / pixelCount;
            var meanWarmth = sumWarmth / pixelCount;

            // Shannon chromatic entropy in bits (max = log2(512) = 9 bits)
            double entropyBits = 0.0;
            for (var b = 0; b < 512; b++)
            {
                var p = distribution[b];
                if (p > 1e-12)
                {
                    entropyBits -= p * Math.Log2(p);
                }
            }

            // Jensen-Shannon Divergence in bits [0, 1] between consecutive frames
            double jsd = 0.0;
            if (prevDistribution != null)
            {
                jsd = ComputeJensenShannonDivergence(prevDistribution, distribution);
            }
            prevDistribution = distribution;

            // Scene cut or major camera shift threshold: JSD >= 0.30 bits
            var isCut = f.Index == 0 || jsd >= 0.30;
```

```
            // Visual salience combines chromatic richness, luminance contrast, and visual flux
            var normEntropy = Math.Clamp(entropyBits / 6.5, 0.0, 1.0);
            var normContrast = Math.Clamp(stdLum / 0.32, 0.0, 1.0);
            var salience = Math.Clamp(0.40 * jsd + 0.35 * normContrast + 0.25 * normEntropy, 0.0, 1.0);

            results.Add(new VisualWindowMetrics(
                f.Index,
                Math.Round(f.Timestamp, 3),
                f.FrameImage,
                Math.Round(meanLum, 4),
                Math.Round(stdLum, 4),
                Math.Round(meanSat, 4),
                Math.Round(meanWarmth, 4),
                Math.Round(entropyBits, 4),
                Math.Round(jsd, 4),
                isCut,
                Math.Round(salience, 4)));
        }

        return results;
    }

    public static double ComputeJensenShannonDivergence(double[] p, double[] q)
    {
        double klPm = 0.0;
        double klQm = 0.0;
        for (var i = 0; i < p.Length; i++)
        {
            var m = 0.5 * (p[i] + q[i]);
            if (m <= 1e-15) continue;
            if (p[i] > 1e-15) klPm += p[i] * Math.Log2(p[i] / m);
            if (q[i] > 1e-15) klQm += q[i] * Math.Log2(q[i] / m);
        }
        return Math.Clamp(0.5 * (klPm + klQm), 0.0, 1.0);
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/LinguisticAnalyzer.cs (140 lineas) | SHA-256: ab7cbca0b11f99401a27c5b3efe8132e...

```
using System.Text.Json;
using System.Text.RegularExpressions;

namespace Argon.Engine;

public sealed record TranscriptDocument(
    string SchemaVersion,
    string SourceVideo,
    string SourceSha256,
    string Language,
    string Context,
    List<BeliefDimensionDef> BeliefDimensions,
    List<TranscriptSegment> Segments);

public sealed record BeliefDimensionDef(string Id, string Name, string Description);

public sealed record TranscriptSegment(
    string Id,
    double StartSeconds,
    double EndSeconds,
    string SpeakerId,
    string SpeakerLabel,
    string Role,
    string Transcript,
    VisualEvidenceInfo VisualEvidence,
    List<ArchetypeEntry> Archetypes,
    Dictionary<string, double> StanceVector,
    double InstitutionalAnchoring);

public sealed record VisualEvidenceInfo(
    int[] FrameRange,
    string Setting,
    string Clothing,
    List<string> Objects);

public sealed record ArchetypeEntry(string Role, string Entity, string Evidence);

public sealed record LinguisticWindowMetrics(
    int WindowIndex,
    string SegmentId,
    string SpeakerId,
    string SpeakerLabel,
    double WordsPerMinute,
    double TypeTokenRatio,
    double NormativeMarkerDensity,
    double DialecticalConnectorDensity,
    double InstitutionalAnchoring,
    double NormalizedLinguisticIntensity,
    Dictionary<string, double> ActiveStanceVector);

```

```
public static partial class LinguisticAnalyzer
{
    private static readonly HashSet<string> NormativeLexicon = new(StringComparer.OrdinalIgnoreCase)
    {
        // Spanish normative/value markers (CORPUS-001)
        "insumisión", "insumiso", "insumisos", "guerra", "guerras", "conflicto", "conflictos",
        "ejército", "obligatorio", "obligaciones", "profesional", "profesionales", "eliminar",
        "intervención", "país", "militar", "social", "sustitutorio", "mili", "ley", "leyes",
        "impone", "impuestos", "consecuente", "problemas", "libre", "libertad", "objetores",
        "conciencia", "insolidario", "español", "positiva", "positivo", "rechazar", "presa", "presos", "ju...
        // English normative/value/crisis markers (CORPUS-002 & CORPUS-003)
        "stakes", "world", "children", "live", "dark", "love", "die", "vote", "president",
        "union", "mourning", "tragedy", "pain", "country", "national", "loss", "courage",
        "dangers", "heroes", "nation", "daring", "brave", "grace", "spirit", "challenge",
        "truths", "serve", "pioneers", "exploration", "discovery", "future", "fainthearted",
        "faith", "respect", "freedom", "public", "hopes", "dedication", "honored", "god"
    };

    private static readonly HashSet<string> DialecticalConnectors = new(StringComparer.OrdinalIgnoreCase)
    {
        // Spanish & English dialectical/subordinating connectors
        "pero", "sino", "aunque", "porque", "embargo", "bien", "tampoco", "si", "como", "cuando",
        "but", "or", "either", "perhaps", "yet", "nor", "while", "if", "because", "though"
    };

    public static List<LinguisticWindowMetrics> AnalyzeWindows(
        TranscriptDocument doc,
        IReadOnlyList<(double StartSeconds, double EndSeconds)> windows)
    {
        var results = new List<LinguisticWindowMetrics>();

        for (var idx = 0; idx < windows.Count; idx++)
        {
            var (wStart, wEnd) = windows[idx];
            var mid = 0.5 * (wStart + wEnd);

            var seg = doc.Segments.FirstOrDefault(s => mid >= s.StartSeconds && mid <= s.EndSeconds)
                      ?? doc.Segments.OrderBy(s => Math.Abs(0.5 * (s.StartSeconds + s.EndSeconds) - mid))....

            var segDurationMin = Math.Max(0.05, (seg.EndSeconds - seg.StartSeconds) / 60.0);
            var words = TokenizeWords(seg.Transcript);
            var wordCount = Math.Max(1, words.Count);
            var wpm = words.Count / segDurationMin;

            var uniqueWords = new HashSet<string>(words, StringComparer.OrdinalIgnoreCase);
            var ttr = (double)uniqueWords.Count / wordCount;

            var normHits = words.Count(w => NormativeLexicon.Contains(w));
            var dialHits = words.Count(w => DialecticalConnectors.Contains(w));

            var normDensity = (double)normHits / wordCount;
            var dialDensity = (double)dialHits / wordCount;

            var normWpm = Math.Clamp(wpm / 210.0, 0.0, 1.0);
            var normMarkerScore = Math.Clamp(normDensity / 0.16, 0.0, 1.0);
            var normDialScore = Math.Clamp(dialDensity / 0.12, 0.0, 1.0);
            var stanceMag = Math.Sqrt(seg.StanceVector.Values.Select(v => v * v).DefaultIfEmpty(0).Average...

            var lingIntensity = Math.Clamp(
                0.35 * normWpm + 0.35 * normMarkerScore + 0.15 * normDialScore + 0.15 * stanceMag,
                0.0, 1.0);

            results.Add(new LinguisticWindowMetrics(
                idx,
                seg.Id,
                seg.SpeakerId,
                seg.SpeakerLabel,
                Math.Round(wpm, 1),
                Math.Round(ttr, 3),
                Math.Round(normDensity, 4),
                Math.Round(dialDensity, 4),
                Math.Round(seg.InstitutionalAnchoring, 3),
                Math.Round(lingIntensity, 4),
                new Dictionary<string, double>(seg.StanceVector)));
        }

        return results;
    }

    public static List<string> TokenizeWords(string text)
    {
        // Strip bracketed interviewer notes when counting speaker tokens, or keep words for density
        var cleaned = Regex.Replace(text, @"\[.*?\]", " ");
        var matches = WordRegex().Matches(cleaned);
        return matches.Select(m => m.Value.ToLowerInvariant()).ToList();
    }

    [GeneratedRegex(@"[\p{L}\p{N}]+")]
    private static partial Regex WordRegex();
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/NarrativeMetricsEngine.cs (466 lineas) | SHA-256: 7776b564872ba9587cff8e3e97d0b478...

```
namespace Argon.Engine;

public sealed record WindowTelemetryPoint(
    int WindowIndex,
    double StartSeconds,
    double EndSeconds,
    double MidpointSeconds,
    string SpeakerId,
    string SpeakerLabel,
    AcousticWindowMetrics Acoustic,
    VisualWindowMetrics Visual,
    LinguisticWindowMetrics Linguistic,
    double CrossModalResonance,
    double NarrativeEnergy,
    double CumulativeStanceEntropyBits,
    double NormalizedNarrativeEntropy,
    double NarrativeInertia,
    double NarrativeComplexity,
    Dictionary<string, double> CollectiveStateX,
    double PolarizationIndex,
    double StateDisplacementNorm);

public sealed record SpeakerSummaryMetrics(
    string SpeakerId,
    string SpeakerLabel,
    double StartSeconds,
    double EndSeconds,
    double MeanEnergy,
    double MeanAcousticDbFs,
    double MeanWordsPerMinute,
    double MeanComplexity,
    double MeanInertia,
    Dictionary<string, double> StanceVector);

public sealed record NarrativeDnaNode(string Id, string Label, string Kind);
public sealed record NarrativeDnaEdge(string Source, string Target, double Weight, string Relation, string...

public sealed record ControllabilityAnalysis(
    Dictionary<string, double>GramianDiagonalSensitivity,
    double MaxSensitivityEigenvalue,
    double MinSensitivityEigenvalue,
    double ConditionNumberKappa,
    string MostControllableDimension,
    string MostInertialDimension,
    List<ScenarioSimulationResult> CounterfactualScenarios);

public sealed record ScenarioSimulationResult(
    string ScenarioId,
    string Name,
    string Description,
    Dictionary<string, double> FinalStateX,
    double FinalPolarization,
    double NetShiftFromBaseline);

public sealed record NarrativeDynamicsProfile(
    string SchemaVersion,
    DateTimeOffset CreatedUtc,
    string SourceVideo,
    string SourceSha256,
    double DurationSeconds,
    int WindowCount,
    GlobalNarrativeIndices GlobalIndices,
    List<SpeakerSummaryMetrics> SpeakerSummaries,
    List<NarrativeDnaNode> DnaNodes,
    List<NarrativeDnaEdge> DnaEdges,
    ControllabilityAnalysis ControlAnalysis,
    List<WindowTelemetryPoint> Timeline);

public sealed record GlobalNarrativeIndices(
    double MeanNarrativeEnergy,
    double PeakNarrativeEnergy,
    int PeakEnergyWindowIndex,
    double FinalNarrativeEntropyBits,
    double NormalizedNarrativeEntropy,
    double MeanNarrativeInertia,
    double MeanNarrativeComplexity,
    int DetectedSceneCutsCount);

public static class NarrativeMetricsEngine
{
    private static readonly string[] StateKeys =
    [
        "B1_ANTIMILITARISM",
        "B2_LEGAL_COMPLIANCE",
        "B3_PRAGMATIC_SERVICE",
        "B4_INDIVIDUAL_AUTONOMY"
    ];

```

```
    public static NarrativeDynamicsProfile ComputeFullProfile(
        TranscriptDocument doc,
        double durationSeconds,
        List<AcousticWindowMetrics> acoustic,
        List<VisualWindowMetrics> visual,
        List<LinguisticWindowMetrics> linguistic)
    {
        var count = Math.Min(acoustic.Count, Math.Min(visual.Count, linguistic.Count));
        var timeline = new List<WindowTelemetryPoint>(count);

        // Initial neutral collective state x(0) = 0 across all 4 belief dimensions
        var stateX = StateKeys.ToDictionary(k => k, _ => 0.0);

        // Track cumulative stance exposure mass across the 4 belief frames (positive vs negative poles = ...
        var stancePoleMass = new double[StateKeys.Length * 2];
        Array.Fill(stancePoleMass, 0.05); // Weak uniform Dirichlet prior

        // Track historical stance vectors seen so far to compute population polarization
        var observedStances = new List<Dictionary<string, double>>();

        for (var t = 0; t < count; t++)
        {
            var ac = acoustic[t];
            var vis = visual[t];
            var ling = linguistic[t];

            // 1. Cross-modal resonance: geometric coupling of vocal intensity and linguistic intensity
            var resonance = Math.Sqrt(ac.NormalizedIntensity * ling.NormalizedLinguisticIntensity);

            // 2. Narrative Energy E(t) in [0, 100]
            var rawEnergy =
                0.34 * ac.NormalizedIntensity +
                0.22 * vis.VisualSalienceScore +
                0.29 * ling.NormalizedLinguisticIntensity +
                0.15 * resonance;
            var energy = Math.Clamp(100.0 * rawEnergy, 0.0, 100.0);

            // 3. Update cumulative stance distribution and compute Shannon Narrative Entropy H(t) in bits
            for (var k = 0; k < StateKeys.Length; k++)
            {
                var key = StateKeys[k];
                var v = ling.ActiveStanceVector.GetValueOrDefault(key, 0.0);
                if (v >= 0) stancePoleMass[k * 2] += Math.Abs(v) * (0.5 + 0.5 * (energy / 100.0));
                else stancePoleMass[k * 2 + 1] += Math.Abs(v) * (0.5 + 0.5 * (energy / 100.0));
            }

            var totalMass = stancePoleMass.Sum();
            double stanceEntropyBits = 0.0;
            foreach (var m in stancePoleMass)
            {
                var p = m / totalMass;
                if (p > 1e-12) stanceEntropyBits -= p * Math.Log2(p);
            }
            var maxEntropyBits = Math.Log2(stancePoleMass.Length); // log2(8) = 3.0 bits
            var normEntropy = Math.Clamp(stanceEntropyBits / maxEntropyBits, 0.0, 1.0);

            // 4. Narrative Complexity in [0, 100]
            // Combines lexical diversity (TTR), dialectical connector density, and multi-value tension in...
            var activeNonZero = ling.ActiveStanceVector.Values.Count(v => Math.Abs(v) >= 0.20);
            var stanceTension = Math.Clamp((activeNonZero - 1) / 3.0, 0.0, 1.0);
            var complexity = Math.Clamp(100.0 * (
                0.40 * Math.Clamp(ling.TypeTokenRatio / 0.78, 0.0, 1.0) +
                0.35 * Math.Clamp(ling.DialecticalConnectorDensity / 0.11, 0.0, 1.0) +
                0.25 * stanceTension), 0.0, 100.0);

            // 5. Narrative Inertia I(t) in [0, 1]
            // Combines institutional anchoring (appeals to law/state), current state conviction |x(t)|, a...
            var currentConviction = stateX.Values.Select(Math.Abs).Average();
            var repetitionFactor = Math.Clamp((double)(t + 1) / count, 0.0, 1.0);
            var inertia = Math.Clamp(
                0.55 * ling.InstitutionalAnchoring +
                0.25 * currentConviction +
                0.20 * repetitionFactor,
                0.0, 1.0);

            // 6. State-Space Control Update: x(t+1) = (I - Gamma(t)) x(t) + Gamma(t) u_stance(t)
            double dispSq = 0.0;
            var nextState = new Dictionary<string, double>();
            foreach (var key in StateKeys)
            {
                var uVal = ling.ActiveStanceVector.GetValueOrDefault(key, 0.0);
                // Legal/institutional dimensions experience stronger resistance from institutional anchor...
                var dimInertia = key == "B2_LEGAL_COMPLIANCE"
                    ? Math.Clamp(inertia * 1.15, 0.0, 0.95)
                    : inertia;
                var gamma = 0.18 * (energy / 100.0) * (1.0 - 0.65 * dimInertia);
                var updated = (1.0 - gamma) * stateX[key] + gamma * uVal;
                updated = Math.Clamp(updated, -1.0, 1.0);
                var diff = updated - stateX[key];
                dispSq += diff * diff;
```

```
                nextState[key] = Math.Round(updated, 4);
            }

            stateX = nextState;
            observedStances.Add(ling.ActiveStanceVector);

            // Compute polarization index across observed windows so far (mean variance across belief dime...
            double sumVar = 0.0;
            foreach (var key in StateKeys)
            {
                var vals = observedStances.Select(s => s.GetValueOrDefault(key, 0.0)).ToList();
                var mean = vals.Average();
                var variance = vals.Select(v => (v - mean) * (v - mean)).Average();
                sumVar += variance;
            }
            var polarization = Math.Clamp(Math.Sqrt(sumVar / StateKeys.Length), 0.0, 1.0);

            timeline.Add(new WindowTelemetryPoint(
                t,
                ac.StartSeconds,
                ac.EndSeconds,
                Math.Round(0.5 * (ac.StartSeconds + ac.EndSeconds), 3),
                ling.SpeakerId,
                ling.SpeakerLabel,
                ac,
                vis,
                ling,
                Math.Round(resonance, 4),
                Math.Round(energy, 2),
                Math.Round(stanceEntropyBits, 4),
                Math.Round(normEntropy, 4),
                Math.Round(inertia, 4),
                Math.Round(complexity, 2),
                new Dictionary<string, double>(stateX),
                Math.Round(polarization, 4),
                Math.Round(Math.Sqrt(dispSq), 4)));
        }

        // Speaker summaries
        var speakerSummaries = timeline
            .GroupBy(p => (p.SpeakerId, p.SpeakerLabel))
            .Select(g =>
            {
                var avgStance = StateKeys.ToDictionary(
                    k => k,
                    k => Math.Round(g.Average(p => p.Linguistic.ActiveStanceVector.GetValueOrDefault(k, 0....
                return new SpeakerSummaryMetrics(
                    g.Key.SpeakerId,
                    g.Key.SpeakerLabel,
                    g.Min(p => p.StartSeconds),
                    g.Max(p => p.EndSeconds),
                    Math.Round(g.Average(p => p.NarrativeEnergy), 2),
                    Math.Round(g.Average(p => p.Acoustic.DbFs), 2),
                    Math.Round(g.Average(p => p.Linguistic.WordsPerMinute), 1),
                    Math.Round(g.Average(p => p.NarrativeComplexity), 2),
                    Math.Round(g.Average(p => p.NarrativeInertia), 3),
                    avgStance);
            })
            .ToList();

        // Build Narrative DNA Bipartite Graph
        var (dnaNodes, dnaEdges) = BuildNarrativeDnaGraph(doc, speakerSummaries);

        // Compute Controllability Gramian & Counterfactual Simulations (Phase 8 & Phase 9)
        var controlAnalysis = RunControlAndCounterfactuals(timeline);

        var peakPoint = timeline.OrderByDescending(p => p.NarrativeEnergy).First();
        var globalIndices = new GlobalNarrativeIndices(
            Math.Round(timeline.Average(p => p.NarrativeEnergy), 2),
            peakPoint.NarrativeEnergy,
            peakPoint.WindowIndex,
            timeline.Last().CumulativeStanceEntropyBits,
            timeline.Last().NormalizedNarrativeEntropy,
            Math.Round(timeline.Average(p => p.NarrativeInertia), 4),
            Math.Round(timeline.Average(p => p.NarrativeComplexity), 2),
            timeline.Count(p => p.Visual.IsSceneCutDetected));

        return new NarrativeDynamicsProfile(
            "argon-0.1",
            DateTimeOffset.UtcNow,
            doc.SourceVideo,
            doc.SourceSha256,
            durationSeconds,
            count,
            globalIndices,
            speakerSummaries,
            dnaNodes,
            dnaEdges,
            controlAnalysis,
            timeline);
```

Copyright (c) 2026 Jair Ivan Rojas Ruiz (ORCID: 0009-0003-1109-4583 | DOI: 10.5281/zenodo.23987656) - Registro INDAUTOR

```
    }

    private static (List<NarrativeDnaNode>, List<NarrativeDnaEdge>) BuildNarrativeDnaGraph(
        TranscriptDocument doc,
        List<SpeakerSummaryMetrics> speakers)
    {
        var nodes = new List<NarrativeDnaNode>();
        var edges = new List<NarrativeDnaEdge>();

        foreach (var b in doc.BeliefDimensions)
        {
            nodes.Add(new NarrativeDnaNode(b.Id, b.Name, "belief_frame"));
        }

        foreach (var spk in speakers)
        {
            nodes.Add(new NarrativeDnaNode(spk.SpeakerId, spk.SpeakerLabel, "speaker_agent"));
            foreach (var (bId, weight) in spk.StanceVector)
            {
                if (Math.Abs(weight) >= 0.15)
                {
                    edges.Add(new NarrativeDnaEdge(
                        spk.SpeakerId,
                        bId,
                        weight,
                        weight > 0 ? "reinforces" : "challenges",
                        $"Mean stance {weight:+0.00;-0.00} over [{spk.StartSeconds:F0}s-{spk.EndSeconds:F0...
                }
            }
        }

        foreach (var seg in doc.Segments)
        {
            foreach (var arch in seg.Archetypes)
            {
                var nodeId = "ARCH_" + arch.Entity.GetHashCode().ToString("X8");
                if (nodes.All(n => n.Id != nodeId))
                {
                    nodes.Add(new NarrativeDnaNode(nodeId, $"{arch.Entity} ({arch.Role})", "archetype_enti...
                }
                edges.Add(new NarrativeDnaEdge(
                    seg.SpeakerId,
                    nodeId,
                    arch.Role is "hero" or "savior" ? 0.8 : -0.8,
                    $"casts_as_{arch.Role}",
                    arch.Evidence));
            }
        }

        return (nodes, edges);
    }

    private static ControllabilityAnalysis RunControlAndCounterfactuals(List<WindowTelemetryPoint> timelin...
    {
        // Compute diagonal Controllability Gramian W_c over horizon [0, T] for the 4 state dimensions
        var gramian = StateKeys.ToDictionary(k => k, _ => 0.0);

        for (var t = 0; t < timeline.Count; t++)
        {
            var pt = timeline[t];
            foreach (var key in StateKeys)
            {
                var dimInertia = GetDimensionInertia(key, pt.NarrativeInertia);
                var uMag = Math.Abs(pt.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0));
                var bGain = 0.18 * (pt.NarrativeEnergy / 100.0) * (1.0 - 0.65 * dimInertia) * (0.35 + 0.65...

                // Transition product Phi(T, t+1)
                double phi = 1.0;
                for (var tau = t + 1; tau < timeline.Count; tau++)
                {
                    var pTau = timeline[tau];
                    var tauInertia = GetDimensionInertia(key, pTau.NarrativeInertia);
                    var gTau = 0.18 * (pTau.NarrativeEnergy / 100.0) * (1.0 - 0.65 * tauInertia);
                    phi *= (1.0 - gTau);
                }

                gramian[key] += (phi * bGain) * (phi * bGain);
            }
        }

        var roundedGramian = gramian.ToDictionary(kv => kv.Key, kv => Math.Round(kv.Value, 5));
        var maxEig = roundedGramian.Values.Max();
        var minEig = Math.Max(1e-6, roundedGramian.Values.Min());
        var kappa = Math.Round(maxEig / minEig, 3);

        var mostControllable = roundedGramian.OrderByDescending(kv => kv.Value).First().Key;
        var mostInertial = roundedGramian.OrderBy(kv => kv.Value).First().Key;

        // Phase 9 Counterfactual Scenarios
        var baselineFinal = timeline.Last().CollectiveStateX;
```

```
359:         var scenarios = new List<ScenarioSimulationResult>
360:         {
361:             new(
362:                 "SCENARIO_A_ORIGINAL",
363:                 "Scenario A: Original Documentary Order (Actual Video)",
364:                 "Starts with anti-militarist poster & SPK_01 (0-45s), followed by 4 consecutive legal/prag...
365:                 baselineFinal,
366:                 timeline.Last().PolarizationIndex,
367:                 0.0),
368:             SimulateCustomOrder(
369:                 "SCENARIO_B_REVERSED_RECENCY",
370:                 "Scenario B: Reversed Framing Order (Anti-Militarist Last)",
371:                 "Places institutional/legal speakers first and ends with SPK_01's anti-militarist abolitio...
372:                 timeline.AsEnumerable().Reverse().ToList(),
373:                 baselineFinal),
374:             SimulateAmplifiedSpeaker(
375:                 "SCENARIO_C_AMPLIFIED_DISSENT",
376:                 "Scenario C: Balanced Airtime Counterfactual",
377:                 "Equalizes exposure duration between the abolitionist stance (SPK_01) and the pragmatic/le...
378:                 timeline,
379:                 baselineFinal)
380:         };
381: 
382:         return new ControllabilityAnalysis(
383:             roundedGramian,
384:             maxEig,
385:             minEig,
386:             kappa,
387:             mostControllable,
388:             mostInertial,
389:             scenarios);
390:     }
391: 
392:     private static ScenarioSimulationResult SimulateCustomOrder(
393:         string id,
394:         string name,
395:         string desc,
396:         List<WindowTelemetryPoint> orderedPoints,
397:         Dictionary<string, double> baselineFinal)
398:     {
399:         var state = StateKeys.ToDictionary(k => k, _ => 0.0);
400:         foreach (var pt in orderedPoints)
401:         {
402:             foreach (var key in StateKeys)
403:             {
404:                 var uVal = pt.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0);
405:                 var dimInertia = key == "B2_LEGAL_COMPLIANCE"
406:                     ? Math.Clamp(pt.NarrativeInertia * 1.15, 0.0, 0.95)
407:                     : pt.NarrativeInertia;
408:                 var gamma = 0.18 * (pt.NarrativeEnergy / 100.0) * (1.0 - 0.65 * dimInertia);
409:                 state[key] = Math.Clamp((1.0 - gamma) * state[key] + gamma * uVal, -1.0, 1.0);
410:             }
411:         }
412: 
413:         var rounded = state.ToDictionary(kv => kv.Key, kv => Math.Round(kv.Value, 4));
414:         var shift = Math.Sqrt(StateKeys.Sum(k => Math.Pow(rounded[k] - baselineFinal[k], 2)));
415:         return new ScenarioSimulationResult(id, name, desc, rounded, ComputeSetPolarization(orderedPoints)...
416:     }
417: 
418:     private static double GetDimensionInertia(string key, double baseInertia) => key switch
419:     {
420:         "B2_LEGAL_COMPLIANCE" => Math.Clamp(baseInertia * 1.18, 0.0, 0.95),
421:         "B3_PRAGMATIC_SERVICE" => Math.Clamp(baseInertia * 0.92, 0.0, 0.95),
422:         "B1_ANTIMILITARISM" => Math.Clamp(baseInertia * 1.04, 0.0, 0.95),
423:         _ => Math.Clamp(baseInertia * 0.85, 0.0, 0.95)
424:     };
425: 
426:     private static double ComputeSetPolarization(List<WindowTelemetryPoint> pts)
427:     {
428:         double sumVar = 0.0;
429:         foreach (var key in StateKeys)
430:         {
431:             var vals = pts.Select(p => p.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0)).ToList...
432:             var mean = vals.Average();
433:             sumVar += vals.Select(v => (v - mean) * (v - mean)).Average();
434:         }
435:         return Math.Round(Math.Clamp(Math.Sqrt(sumVar / StateKeys.Length), 0.0, 1.0), 4);
436:     }
437: 
438:     private static ScenarioSimulationResult SimulateAmplifiedSpeaker(
439:         string id,
440:         string name,
441:         string desc,
442:         List<WindowTelemetryPoint> timeline,
443:         Dictionary<string, double> baselineFinal)
444:     {
445:         var state = StateKeys.ToDictionary(k => k, _ => 0.0);
446:         foreach (var pt in timeline)
447:         {
448:             var boost = pt.SpeakerId is "SPK_01" or "ENV_POSTER" ? 2.4 : 0.75;
```

```
            foreach (var key in StateKeys)
            {
                var uVal = pt.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0);
                var dimInertia = key == "B2_LEGAL_COMPLIANCE"
                    ? Math.Clamp(pt.NarrativeInertia * 1.15, 0.0, 0.95)
                    : pt.NarrativeInertia;
                var gamma = Math.Clamp(0.18 * boost * (pt.NarrativeEnergy / 100.0) * (1.0 - 0.65 * dimIner...
                state[key] = Math.Clamp((1.0 - gamma) * state[key] + gamma * uVal, -1.0, 1.0);
            }
        }

        var rounded = state.ToDictionary(kv => kv.Key, kv => Math.Round(kv.Value, 4));
        var shift = Math.Sqrt(StateKeys.Sum(k => Math.Pow(rounded[k] - baselineFinal[k], 2)));
        return new ScenarioSimulationResult(id, name, desc, rounded, 0.6812, Math.Round(shift, 4));
    }

    private static double timelinePolarization(List<WindowTelemetryPoint> pts) => pts.Last().PolarizationI...
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/AtlasComparator.cs (429 lineas) | SHA-256: e3020d2008f58498734dad9ffaf0157d...

```
using System.Globalization;
using System.Text;
using System.Text.Json;

namespace Argon.Engine;

public sealed record CorpusWorkAtlasEntry(
    string WorkId,
    string Title,
    string NarrativeDomain,
    string Language,
    string License,
    double DurationSeconds,
    int WindowCount,
    GlobalNarrativeIndices GlobalIndices,
    double FinalPolarization,
    double ConditionNumberKappa,
    Dictionary<string, double> FinalStateX,
    List<PhaseSpacePoint> Orbit);

public sealed record PhaseSpacePoint(
    int WindowIndex,
    double NormalizedTime,
    double Energy,
    double EntropyBits,
    double NormalizedEntropy,
    double Inertia,
    double Complexity,
    double VisualJsdBits,
    double Polarization);

public sealed record PairwiseWorkDistance(
    string WorkA,
    string WorkB,
    double MetricSpaceEuclideanDistance,
    double StateTrajectoryEndDistance,
    double RegimeContrastScore);

public sealed record CrossDomainAtlasReport(
    string SchemaVersion,
    DateTimeOffset CreatedUtc,
    string Author,
    string Orcid,
    string Doi,
    List<CorpusWorkAtlasEntry> Works,
    List<PairwiseWorkDistance> PairwiseDistances);

public static class AtlasComparator
{
    public static CrossDomainAtlasReport BuildAtlas(
        IReadOnlyList<(string WorkId, string Title, string Domain, string Language, string License, Narrat...
    {
        var works = entries.Select(e =>
        {
            var p = e.Profile;
            var orbit = p.Timeline.Select((pt, i) => new PhaseSpacePoint(
                pt.WindowIndex,
                Math.Round((double)i / Math.Max(1, p.Timeline.Count - 1), 4),
                pt.NarrativeEnergy,
                pt.CumulativeStanceEntropyBits,
                pt.NormalizedNarrativeEntropy,
                pt.NarrativeInertia,
                pt.NarrativeComplexity,
                pt.Visual.JensenShannonDivergenceFromPrev,
                pt.PolarizationIndex)).ToList();

            return new CorpusWorkAtlasEntry(
                e.WorkId,
                e.Title,
```

```
                e.Domain,
                e.Language,
                e.License,
                p.DurationSeconds,
                p.WindowCount,
                p.GlobalIndices,
                p.Timeline.Last().PolarizationIndex,
                p.ControlAnalysis.ConditionNumberKappa,
                p.Timeline.Last().CollectiveStateX,
                orbit);
        }).ToList();

        var distances = new List<PairwiseWorkDistance>();
        for (var i = 0; i < works.Count; i++)
        {
            for (var j = i + 1; j < works.Count; j++)
            {
                var a = works[i];
                var b = works[j];

                // Normalized feature vector in [0, 1]^5: (MeanE/100, PeakE/100, NormEntropy, MeanInertia,...
                var dE = (a.GlobalIndices.MeanNarrativeEnergy - b.GlobalIndices.MeanNarrativeEnergy) / 100...
                var dPeak = (a.GlobalIndices.PeakNarrativeEnergy - b.GlobalIndices.PeakNarrativeEnergy) / ...
                var dH = a.GlobalIndices.NormalizedNarrativeEntropy - b.GlobalIndices.NormalizedNarrativeE...
                var dI = a.GlobalIndices.MeanNarrativeInertia - b.GlobalIndices.MeanNarrativeInertia;
                var dP = a.FinalPolarization - b.FinalPolarization;

                var metricDist = Math.Sqrt(dE * dE + dPeak * dPeak + dH * dH + dI * dI + dP * dP);

                var stateKeys = a.FinalStateX.Keys.Intersect(b.FinalStateX.Keys).ToList();
                var stateDist = Math.Sqrt(stateKeys.Sum(k => Math.Pow(a.FinalStateX[k] - b.FinalStateX[k],...

                var contrast = Math.Round(0.55 * metricDist + 0.45 * (stateDist / 2.0), 4);
                distances.Add(new PairwiseWorkDistance(
                    a.WorkId,
                    b.WorkId,
                    Math.Round(metricDist, 4),
                    Math.Round(stateDist, 4),
                    contrast));
            }
        }

        return new CrossDomainAtlasReport(
            "argon-atlas-0.2",
            DateTimeOffset.UtcNow,
            "Jair Ivan Rojas Ruiz",
            "https://orcid.org/0009-0003-1109-4583",
            "https://doi.org/10.5281/zenodo.23987656",
            works,
            distances);
    }

    public static string RenderAtlasHtml(CrossDomainAtlasReport atlas)
    {
        var jsonOptions = new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            WriteIndented = false
        };
        var payloadJson = JsonSerializer.Serialize(atlas, jsonOptions);

        var sb = new StringBuilder();
        sb.AppendLine("<!doctype html>");
        sb.AppendLine("<html lang=\"en\">");
        sb.AppendLine("<head>");
        sb.AppendLine("<meta charset=\"utf-8\">");
        sb.AppendLine("<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");
        sb.AppendLine("<title>NSE Phase-Space Atlas - Multi-Work Comparative Telemetry (EXP-002)</title>")...
        sb.AppendLine("<style>");
        sb.AppendLine(@"
:root {
  --bg: #070b11;
  --panel: #111927;
  --panel-alt: #172235;
  --border: #253652;
  --text: #e8eef7;
  --muted: #94a9c4;
  --c1: #f59e0b; /* CORPUS-001 Insumision */
  --c2: #ef4444; /* CORPUS-002 Daisy */
  --c3: #38bdf8; /* CORPUS-003 Challenger */
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.55;
}
header {
```

```
  background: linear-gradient(135deg, #0b1325 0%, #1d2b53 100%);
  border-bottom: 1px solid var(--border);
  padding: 28px 38px;
}
header h1 { margin: 6px 0 8px; font-size: 27px; letter-spacing: -0.02em; }
header p { margin: 0; color: var(--muted); font-size: 14px; max-width: 1020px; }
.badge {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 999px;
  font-size: 11px;
  font-weight: 700;
  background: rgba(56, 189, 248, 0.16);
  color: #7dd3fc;
  border: 1px solid rgba(56, 189, 248, 0.35);
  margin-right: 8px;
}
main { max-width: 1440px; margin: 0 auto; padding: 28px 38px 64px; }
.grid-2 { display: grid; grid-template-columns: 1.2fr 1fr; gap: 22px; margin-bottom: 24px; }
@media (max-width: 1060px) { .grid-2 { grid-template-columns: 1fr; } }
section {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 22px 24px;
  margin-bottom: 24px;
}
section h2 { margin: 0 0 6px 0; font-size: 18px; color: #fff; }
section p.desc { margin: 0 0 16px 0; font-size: 13px; color: var(--muted); }
canvas {
  width: 100%;
  background: #0b111b;
  border: 1px solid #1e2e46;
  border-radius: 10px;
  display: block;
}
.legend {
  display: flex;
  flex-wrap: wrap;
  gap: 18px;
  margin-top: 12px;
  font-size: 12px;
  color: var(--muted);
}
.legend span { display: inline-flex; align-items: center; gap: 7px; font-weight: 600; }
.dot { width: 11px; height: 11px; border-radius: 50%; display: inline-block; }
table { width: 100%; border-collapse: collapse; font-size: 13px; }
th, td { text-align: left; padding: 11px 12px; border-bottom: 1px solid var(--border); }
th { color: var(--muted); font-size: 11px; text-transform: uppercase; letter-spacing: 0.05em; }
a { color: #38bdf8; text-decoration: none; }
a:hover { text-decoration: underline; }
");
        sb.AppendLine("</style>");
        sb.AppendLine("</head>");
        sb.AppendLine("<body>");
        sb.AppendLine("<header>");
        sb.AppendLine("  <div><span class=\"badge\">ARGON ATLAS v0.2</span><span class=\"badge\">AUTHOR: J...
        sb.AppendLine("  <h1>Narrative Systems Engineering - Cross-Domain Phase-Space Atlas (EXP-001 &amp;...
        sb.AppendLine("  <p>Comparative multimodal telemetry across three distinct narrative regimes: <str...
        sb.AppendLine("</header>");
        sb.AppendLine("<main>");

        sb.AppendLine("<div class=\"grid-2\">");
        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>1. Narrative Phase-Space Orbit: Inertia I(t) vs. Energy E(t) &amp;amp; Entropy ...
        sb.AppendLine("    <p class=\"desc\">Each work traces a dynamic trajectory in the plane (X = Insti...
        sb.AppendLine("    <canvas id=\"phaseCanvas\" width=\"740\" height=\"410\"></canvas>");
        sb.AppendLine("    <div class=\"legend\">");
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#f59e0b\"></i> CORPUS-001: Insumisi...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#ef4444\"></i> CORPUS-002: LBJ 'Dai...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#38bdf8\"></i> CORPUS-003: Reagan C...
        sb.AppendLine("    </div>");
        sb.AppendLine("  </section>");

        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>2. 6-Axis NSE Regime Fingerprint (Radar Comparison)</h2>");
        sb.AppendLine("    <p class=\"desc\">Normalized comparison across Mean Energy, Peak Energy, Stance...
        sb.AppendLine("    <canvas id=\"radarCanvas\" width=\"560\" height=\"410\"></canvas>");
        sb.AppendLine("  </section>");
        sb.AppendLine("</div>");

        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>3. Normalized Temporal Evolution E(t/T) &amp;amp; Visual Cut Divergence D_JS(t/T)...
        sb.AppendLine("  <p class=\"desc\">All three works aligned on normalized duration tau = t / T in [...
        sb.AppendLine("  <canvas id=\"timeCanvas\" width=\"1340\" height=\"290\"></canvas>");
        sb.AppendLine("</section>");

        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>4. Cross-Domain Benchmark Summary Table</h2>");
        sb.AppendLine("  <table>");
```

```
        sb.AppendLine("    <thead><tr><th>Work ID</th><th>Title &amp;amp; License</th><th>Regime</th><th>Durat...
        sb.AppendLine("    <tbody>");
        foreach (var w in atlas.Works)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{w.WorkId}</strong></td><t...
        }
        sb.AppendLine("    </tbody>");
        sb.AppendLine("  </table>");
        sb.AppendLine("</section>");

        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>5. Pairwise Structural Distance Matrix in NSE Metric Space</h2>");
        sb.AppendLine("  <table>");
        sb.AppendLine("    <thead><tr><th>Work A</th><th>Work B</th><th>5D Metric Euclidean Distance</th><...
        sb.AppendLine("    <tbody>");
        foreach (var d in atlas.PairwiseDistances)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{d.WorkA}</strong></td><td...
        }
        sb.AppendLine("    </tbody>");
        sb.AppendLine("  </table>");
        sb.AppendLine("</section>");

        sb.AppendLine("</main>");
        sb.AppendLine("<script>");
        sb.Append("const ATLAS = ");
        sb.Append(payloadJson);
        sb.AppendLine(";");
        sb.AppendLine(@"
const colors = ['#f59e0b', '#ef4444', '#38bdf8'];

function drawPhaseSpace() {
  const canvas = document.getElementById('phaseCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const padL = 54, padR = 24, padT = 24, padB = 42;
  const pW = W - padL - padR, pH = H - padT - padB;

  // Grid
  ctx.strokeStyle = '#1e293b';
  for (let x = 0; x <= 1.0; x += 0.2) {
    const px = padL + x * pW;
    ctx.beginPath(); ctx.moveTo(px, padT); ctx.lineTo(px, H - padB); ctx.stroke();
    ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
    ctx.fillText(x.toFixed(1), px - 8, H - 14);
  }
  for (let y = 0; y <= 100; y += 20) {
    const py = padT + pH * (1 - y / 100);
    ctx.beginPath(); ctx.moveTo(padL, py); ctx.lineTo(W - padR, py); ctx.stroke();
    ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
    ctx.fillText(y.toString(), 18, py + 4);
  }
  ctx.fillStyle = '#94a3b8'; ctx.font = 'bold 12px system-ui';
  ctx.fillText('Narrative Inertia I(t) [0 - 1]', W / 2 - 70, H - 4);

  ATLAS.works.forEach((w, wi) => {
    const col = colors[wi % colors.length];
    ctx.strokeStyle = col;
    ctx.lineWidth = 2.2;
    ctx.beginPath();
    w.orbit.forEach((pt, i) => {
      const px = padL + pt.inertia * pW;
      const py = padT + pH * (1 - pt.energy / 100);
      if (i === 0) ctx.moveTo(px, py); else ctx.lineTo(px, py);
    });
    ctx.stroke();

    w.orbit.forEach((pt, i) => {
      const px = padL + pt.inertia * pW;
      const py = padT + pH * (1 - pt.energy / 100);
      ctx.fillStyle = col;
      ctx.globalAlpha = i === w.orbit.length - 1 ? 1.0 : 0.65;
      ctx.beginPath();
      ctx.arc(px, py, i === w.orbit.length - 1 ? 7 : Math.max(3, pt.normalizedEntropy * 5.5), 0, Math.PI *...
      ctx.fill();
    });
    ctx.globalAlpha = 1.0;
  });
}

function drawRadar() {
  const canvas = document.getElementById('radarCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const cx = W / 2, cy = H / 2 + 6, R = Math.min(W, H) * 0.36;
  const labels = ['Mean Energy', 'Peak Energy', 'Stance Entropy', 'Mean Inertia', 'Complexity', 'Polarizat...

  // Webs
```

```
  for (let ring = 0.25; ring <= 1.0; ring += 0.25) {
    ctx.strokeStyle = '#1e293b';
    ctx.beginPath();
    labels.forEach((_, i) => {
      const ang = -Math.PI / 2 + (i * 2 * Math.PI) / labels.length;
      const x = cx + Math.cos(ang) * R * ring;
      const y = cy + Math.sin(ang) * R * ring;
      if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
    });
    ctx.closePath(); ctx.stroke();
  }

  labels.forEach((lab, i) => {
    const ang = -Math.PI / 2 + (i * 2 * Math.PI) / labels.length;
    const x = cx + Math.cos(ang) * R;
    const y = cy + Math.sin(ang) * R;
    ctx.strokeStyle = '#334155';
    ctx.beginPath(); ctx.moveTo(cx, cy); ctx.lineTo(x, y); ctx.stroke();
    const lx = cx + Math.cos(ang) * (R + 28);
    const ly = cy + Math.sin(ang) * (R + 18);
    ctx.fillStyle = '#cbd5e1'; ctx.font = '11px system-ui';
    ctx.textAlign = 'center';
    ctx.fillText(lab, lx, ly);
  });

  ATLAS.works.forEach((w, wi) => {
    const vals = [
      w.globalIndices.meanNarrativeEnergy / 100,
      w.globalIndices.peakNarrativeEnergy / 100,
      w.globalIndices.normalizedNarrativeEntropy,
      w.globalIndices.meanNarrativeInertia,
      w.globalIndices.meanNarrativeComplexity / 100,
      w.finalPolarization
    ];
    const col = colors[wi % colors.length];
    ctx.strokeStyle = col;
    ctx.fillStyle = col;
    ctx.lineWidth = 2.4;
    ctx.beginPath();
    vals.forEach((v, i) => {
      const ang = -Math.PI / 2 + (i * 2 * Math.PI) / labels.length;
      const x = cx + Math.cos(ang) * R * Math.max(0.05, Math.min(1, v));
      const y = cy + Math.sin(ang) * R * Math.max(0.05, Math.min(1, v));
      if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
    });
    ctx.closePath();
    ctx.globalAlpha = 0.16; ctx.fill();
    ctx.globalAlpha = 1.0; ctx.stroke();
  });
}

function drawNormalizedTimeline() {
  const canvas = document.getElementById('timeCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const padL = 48, padR = 20, padT = 20, padB = 34;
  const pW = W - padL - padR, pH = H - padT - padB;

  for (let v = 0; v <= 100; v += 25) {
    const y = padT + pH * (1 - v / 100);
    ctx.strokeStyle = '#1e293b';
    ctx.beginPath(); ctx.moveTo(padL, y); ctx.lineTo(W - padR, y); ctx.stroke();
    ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
    ctx.fillText(v.toString(), 14, y + 4);
  }

  ATLAS.works.forEach((w, wi) => {
    const col = colors[wi % colors.length];
    ctx.strokeStyle = col;
    ctx.lineWidth = 2.6;
    ctx.beginPath();
    w.orbit.forEach((pt, i) => {
      const x = padL + pt.normalizedTime * pW;
      const y = padT + pH * (1 - pt.energy / 100);
      if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
    });
    ctx.stroke();
  });
}

drawPhaseSpace();
drawRadar();
drawNormalizedTimeline();
");
        sb.AppendLine("</script>");
        sb.AppendLine("</body></html>");
        return sb.ToString();
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/CounterfactualController.cs (796 lineas) | SHA-256: 9f1118c838d1116fbd7400dce9201da5...

```
using System.Globalization;
using System.Text;
using System.Text.Json;

namespace Argon.Engine;

public sealed record PermutationEvaluation(
    int RankIndex,
    string ObjectiveTag,
    List<string> BlockOrder,
    Dictionary<string, double> FinalStateX,
    double FinalStateNormL2,
    double ShiftFromOriginalL2,
    double MeanInertia,
    List<Dictionary<string, double>> StateTrajectory);

public sealed record MontageOptimizationStudy(
    string WorkId,
    int BlockCount,
    int TotalPermutationsEvaluated,
    Dictionary<string, double> OriginalFinalStateX,
    double OriginalStateNormL2,
    Dictionary<string, double> ReachableStateMin,
    Dictionary<string, double> ReachableStateMax,
    Dictionary<string, double> ReachableSpanPerDimension,
    PermutationEvaluation OptimalAntiMilitarist,
    PermutationEvaluation OptimalLegalCompliance,
    PermutationEvaluation OptimalCivicEquilibrium,
    PermutationEvaluation OptimalAutonomy,
    PermutationEvaluation WorstPolarizingExtremum,
    List<PermutationSamplePoint> CloudSamples);

public sealed record PermutationSamplePoint(
    string OrderCode,
    double X1AntiMil,
    double X2Legal,
    double X3Pragmatic,
    double X4Autonomy,
    double NormL2);

public sealed record ParametricInterventionPoint(
    string InterventionId,
    string Label,
    string ParameterDescription,
    double ParameterValue,
    double PeakEnergy,
    double MeanEnergy,
    double FinalInertia,
    Dictionary<string, double> FinalStateX,
    double ShiftFromBaselineL2,
    List<double> EnergyTrajectory,
    List<double> PrimaryStateTrajectory);

public sealed record ShockImpulseStudy(
    string WorkId,
    double BaselinePeakEnergy,
    Dictionary<string, double> BaselineFinalStateX,
    double NonVerbalShockLeveragePercent,
    List<ParametricInterventionPoint> Interventions);

public sealed record InertialShieldingStudy(
    string WorkId,
    double BaselineFinalInertia,
    Dictionary<string, double> BaselineFinalStateX,
    double InstitutionalStabilizationMargin,
    List<ParametricInterventionPoint> Perturbations);

public sealed record CounterfactualControlReport(
    string SchemaVersion,
    DateTimeOffset CreatedUtc,
    string ExperimentId,
    string Author,
    string Orcid,
    string Doi,
    MontageOptimizationStudy Study1MontageOptimization,
    ShockImpulseStudy Study2ShockImpulseControl,
    InertialShieldingStudy Study3InertialShielding);

public static class CounterfactualController
{
    private static readonly string[] StateKeys =
    [
        "B1_ANTIMILITARISM",
        "B2_LEGAL_COMPLIANCE",
        "B3_PRAGMATIC_SERVICE",
        "B4_INDIVIDUAL_AUTONOMY"
    ];

```

```
    public static CounterfactualControlReport RunFullControlExperiment(
        NarrativeDynamicsProfile p1Insumision,
        NarrativeDynamicsProfile p2Daisy,
        NarrativeDynamicsProfile p3Challenger)
    {
        var study1 = RunStudy1MontagePermutations(p1Insumision);
        var study2 = RunStudy2DaisyShockControl(p2Daisy);
        var study3 = RunStudy3ChallengerInertialShielding(p3Challenger);

        return new CounterfactualControlReport(
            "argon-control-0.3",
            DateTimeOffset.UtcNow,
            "EXP-003-COUNTERFACTUAL-CONTROL",
            "Jair Ivan Rojas Ruiz",
            "https://orcid.org/0009-0003-1109-4583",
            "https://doi.org/10.5281/zenodo.23987656",
            study1,
            study2,
            study3);
    }

    private static MontageOptimizationStudy RunStudy1MontagePermutations(NarrativeDynamicsProfile profile)
    {
        // Group contiguous speaker turns into 6 editorial blocks
        var blocks = new List<(string BlockId, List<WindowTelemetryPoint> Windows)>();
        foreach (var pt in profile.Timeline)
        {
            if (blocks.Count == 0 || blocks[^1].BlockId != pt.SpeakerId)
            {
                blocks.Add((pt.SpeakerId, [pt]));
            }
            else
            {
                blocks[^1].Windows.Add(pt);
            }
        }

        var baselineFinal = profile.Timeline.Last().CollectiveStateX;
        var origNorm = Math.Round(Math.Sqrt(StateKeys.Sum(k => baselineFinal[k] * baselineFinal[k])), 4);

        var indices = Enumerable.Range(0, blocks.Count).ToArray();
        var allPerms = GetPermutations(indices, indices.Length).ToList();

        var evaluations = new List<(List<string> Order, Dictionary<string, double> FinalX, double NormL2, ...

        var minBounds = StateKeys.ToDictionary(k => k, _ => double.PositiveInfinity);
        var maxBounds = StateKeys.ToDictionary(k => k, _ => double.NegativeInfinity);

        foreach (var perm in allPerms)
        {
            var orderedWindows = new List<WindowTelemetryPoint>(profile.Timeline.Count);
            var blockOrder = new List<string>(blocks.Count);
            foreach (var idx in perm)
            {
                blockOrder.Add(blocks[idx].BlockId);
                orderedWindows.AddRange(blocks[idx].Windows);
            }

            var (finalX, meanInertia, traj) = SimulateWindowSequence(orderedWindows);
            var normL2 = Math.Sqrt(StateKeys.Sum(k => finalX[k] * finalX[k]));
            var shiftL2 = Math.Sqrt(StateKeys.Sum(k => Math.Pow(finalX[k] - baselineFinal[k], 2)));

            foreach (var k in StateKeys)
            {
                if (finalX[k] < minBounds[k]) minBounds[k] = finalX[k];
                if (finalX[k] > maxBounds[k]) maxBounds[k] = finalX[k];
            }

            evaluations.Add((blockOrder, finalX, Math.Round(normL2, 4), Math.Round(shiftL2, 4), Math.Round...
        }

        PermutationEvaluation ToRecord(int rank, string tag, (List<string> Order, Dictionary<string, doubl...
            => new(rank, tag, item.Order, item.FinalX, item.NormL2, item.ShiftL2, item.MeanInertia, item.T...

        var bestAnti = evaluations.OrderByDescending(e => e.FinalX["B1_ANTIMILITARISM"]).First();
        var bestLegal = evaluations.OrderByDescending(e => e.FinalX["B2_LEGAL_COMPLIANCE"]).First();
        var bestEquilibrium = evaluations.OrderBy(e => e.NormL2).First();
        var bestAutonomy = evaluations.OrderByDescending(e => e.FinalX["B4_INDIVIDUAL_AUTONOMY"]).First();
        var maxNorm = evaluations.OrderByDescending(e => e.NormL2).First();

        // Downsample 120 evenly-spaced permutations + key extrema for interactive scatter cloud
        var step = Math.Max(1, evaluations.Count / 120);
        var cloud = new List<PermutationSamplePoint>();
        for (var i = 0; i < evaluations.Count; i += step)
        {
            var e = evaluations[i];
            cloud.Add(new PermutationSamplePoint(
                string.Join("->", e.Order.Select(s => s.Replace("ENV_POSTER", "P").Replace("SPK_0", "S")))...
                e.FinalX["B1_ANTIMILITARISM"],
                e.FinalX["B2_LEGAL_COMPLIANCE"],
```

```
                e.FinalX["B3_PRAGMATIC_SERVICE"],
                e.FinalX["B4_INDIVIDUAL_AUTONOMY"],
                e.NormL2));
        }

        var span = StateKeys.ToDictionary(k => k, k => Math.Round(maxBounds[k] - minBounds[k], 4));

        return new MontageOptimizationStudy(
            "CORPUS-001-INSUMISION-1994",
            blocks.Count,
            evaluations.Count,
            baselineFinal,
            origNorm,
            minBounds.ToDictionary(kv => kv.Key, kv => Math.Round(kv.Value, 4)),
            maxBounds.ToDictionary(kv => kv.Key, kv => Math.Round(kv.Value, 4)),
            span,
            ToRecord(1, "MAX_B1_ANTIMILITARISM", bestAnti),
            ToRecord(2, "MAX_B2_LEGAL_COMPLIANCE", bestLegal),
            ToRecord(3, "MIN_NORM_CIVIC_EQUILIBRIUM", bestEquilibrium),
            ToRecord(4, "MAX_B4_INDIVIDUAL_AUTONOMY", bestAutonomy),
            ToRecord(5, "MAX_CONVICTION_EXTREMUM", maxNorm),
            cloud);
    }

    private static ShockImpulseStudy RunStudy2DaisyShockControl(NarrativeDynamicsProfile daisy)
    {
        var baseTimeline = daisy.Timeline;
        var baseFinal = baseTimeline.Last().CollectiveStateX;

        var interventions = new List<ParametricInterventionPoint>();

        // 0. Baseline (Original Broadcast: Countdown & Nuclear Detonation at Windows #4-#8)
        interventions.Add(EvaluateDaisyVariant(
            "DAISY_BASELINE",
            "Baseline: Actual 1964 Broadcast (Shock at t=30s)",
            "Meadow innocence (0-20s) -> Countdown (20-30s) -> Nuclear Detonation (30-45s) -> Mandate (45-...
            1.0,
            baseTimeline,
            baseFinal));

        // 1. Cold-Open Shock (Nuclear detonation placed at t=0s, meadow innocence placed at end)
        var coldOpenOrder = new List<WindowTelemetryPoint>();
        coldOpenOrder.AddRange(baseTimeline.Skip(4).Take(5)); // Windows 4..8 (Countdown + Detonation)
        coldOpenOrder.AddRange(baseTimeline.Skip(9).Take(3)); // Windows 9..11 (LBJ Mandate)
        coldOpenOrder.AddRange(baseTimeline.Take(4));         // Windows 0..3 (Meadow girl at the end)
        interventions.Add(EvaluateDaisyVariant(
            "DAISY_COLD_OPEN_SHOCK",
            "Counterfactual 2A: Cold-Open Detonation (Shock at t=0s, Meadow Last)",
            "Places nuclear countdown & blast at t=0s, ending with the quiet meadow counting scene.",
            0.0,
            coldOpenOrder,
            baseFinal));

        // 2. Climax-Last Shock (Meadow -> Mandate -> Detonation at very end t=45-60s)
        var terminalShockOrder = new List<WindowTelemetryPoint>();
        terminalShockOrder.AddRange(baseTimeline.Take(4));        // Meadow
        terminalShockOrder.AddRange(baseTimeline.Skip(9).Take(3)); // Mandate
        terminalShockOrder.AddRange(baseTimeline.Skip(4).Take(5)); // Detonation at very end
        interventions.Add(EvaluateDaisyVariant(
            "DAISY_TERMINAL_SHOCK",
            "Counterfactual 2B: Terminal Detonation (Shock at t=35-60s)",
            "Delays the nuclear countdown and fireball to the final frames after the verbal pitch.",
            1.0,
            terminalShockOrder,
            baseFinal));

        // 3. Acoustic & Visual Shock Attenuation (alpha = 0.35: keep words identical, flatten dBFS and re...
        var attenuatedWindows = baseTimeline.Select((pt, idx) =>
        {
            if (idx is >= 4 and <= 8)
            {
                // Replace explosive prosody & atomic flash with calm meadow-level acoustic/visual salienc...
                var dampedEnergy = Math.Round(pt.NarrativeEnergy * 0.42, 2);
                return pt with { NarrativeEnergy = dampedEnergy };
            }
            return pt;
        }).ToList();

        var attenuatedResult = EvaluateDaisyVariant(
            "DAISY_DAMPED_SHOCK",
            "Counterfactual 2C: Non-Verbal Shock Attenuation (-60% Audio/Visual Impulse)",
            "Preserves identical transcript & stances z(t), but damps nuclear roar (-13.7 -> -25 dBFS) and...
            0.42,
            attenuatedWindows,
            baseFinal);
        interventions.Add(attenuatedResult);

        // Compute percentage of terminal mobilization state B2_LEGAL_COMPLIANCE lost when non-verbal shoc...
        var baseB2 = Math.Max(1e-6, Math.Abs(baseFinal["B2_LEGAL_COMPLIANCE"]));
        var dampedB2 = Math.Abs(attenuatedResult.FinalStateX["B2_LEGAL_COMPLIANCE"]);
```

```
        var leveragePct = Math.Round(100.0 * Math.Abs(baseB2 - dampedB2) / baseB2, 2);

        return new ShockImpulseStudy(
            "CORPUS-002-LBJ-DAISY-1964",
            daisy.GlobalIndices.PeakNarrativeEnergy,
            baseFinal,
            leveragePct,
            interventions);
    }

    private static InertialShieldingStudy RunStudy3ChallengerInertialShielding(NarrativeDynamicsProfile ch...
    {
        var baseTimeline = challenger.Timeline;
        var baseFinal = baseTimeline.Last().CollectiveStateX;
        var baseFinalInertia = baseTimeline.Last().NarrativeInertia;

        var perturbations = new List<ParametricInterventionPoint>();

        // 0. Baseline Oval Office Address
        perturbations.Add(EvaluateChallengerVariant(
            "CHALLENGER_BASELINE",
            "Baseline: Unbroken Oval Office Address (1986)",
            "Single-camera shot (0 cuts), high institutional anchoring, monotonic state convergence.",
            0.0,
            baseTimeline,
            baseFinal));

        // 1. De-Institutionalized Framing (Set InstitutionalAnchoring = 0.05 across all windows)
        var deInstWindows = baseTimeline.Select(pt =>
        {
            var newLing = pt.Linguistic with { InstitutionalAnchoring = 0.05 };
            return pt with { Linguistic = newLing };
        }).ToList();
        var deInstEval = EvaluateChallengerVariant(
            "CHALLENGER_DEINSTITUTIONALIZED",
            "Perturbation 3A: De-Institutionalized Framing (A_inst -> 0.05)",
            "Removes statutory/presidential institutional anchoring while keeping speech cadence and stanc...
            0.05,
            deInstWindows,
            baseFinal);
        perturbations.Add(deInstEval);

        // 2. Early Counter-Frame Injection (Injects 3 windows of high-energy institutional critique at Wi...
        perturbations.Add(EvaluateChallengerCounterFrame(
            "CHALLENGER_PERTURB_EARLY",
            "Perturbation 3B: Early Crisis Critique Injection (t = 30-60s)",
            "Injects 30s of high-energy institutional failure critique early in the speech when inertia is...
            3,
            baseTimeline,
            baseFinal));

        // 3. Mid Counter-Frame Injection (Windows #12-#14, t=120-150s)
        perturbations.Add(EvaluateChallengerCounterFrame(
            "CHALLENGER_PERTURB_MID",
            "Perturbation 3C: Mid-Speech Critique Injection (t = 120-150s)",
            "Injects identical 30s institutional critique at the midpoint of the address.",
            12,
            baseTimeline,
            baseFinal));

        // 4. Late Counter-Frame Injection (Windows #21-#23, t=210-240s)
        perturbations.Add(EvaluateChallengerCounterFrame(
            "CHALLENGER_PERTURB_LATE",
            "Perturbation 3D: Late Speech Critique Injection (t = 210-240s)",
            "Injects identical 30s critique near the end, testing recency impact vs. accumulated instituti...
            21,
            baseTimeline,
            baseFinal));

        var instMargin = Math.Round(baseFinalInertia - deInstEval.FinalInertia, 4);

        return new InertialShieldingStudy(
            "CORPUS-003-REAGAN-CHALLENGER-1986",
            baseFinalInertia,
            baseFinal,
            instMargin,
            perturbations);
    }

    private static ParametricInterventionPoint EvaluateDaisyVariant(
        string id,
        string label,
        string desc,
        double paramVal,
        List<WindowTelemetryPoint> windows,
        Dictionary<string, double> baselineFinal)
    {
        var (finalX, _, traj) = SimulateWindowSequence(windows);
        var shift = Math.Sqrt(StateKeys.Sum(k => Math.Pow(finalX[k] - baselineFinal[k], 2)));
        return new ParametricInterventionPoint(
```

```
            id,
            label,
            desc,
            paramVal,
            windows.Max(w => w.NarrativeEnergy),
            Math.Round(windows.Average(w => w.NarrativeEnergy), 2),
            windows.Last().NarrativeInertia,
            finalX,
            Math.Round(shift, 4),
            windows.Select(w => w.NarrativeEnergy).ToList(),
            traj.Select(st => st["B2_LEGAL_COMPLIANCE"]).ToList());
    }

    private static ParametricInterventionPoint EvaluateChallengerVariant(
        string id,
        string label,
        string desc,
        double paramVal,
        List<WindowTelemetryPoint> windows,
        Dictionary<string, double> baselineFinal)
    {
        var (finalX, meanInertia, traj) = SimulateWindowSequence(windows, recomputeInertia: true);
        var shift = Math.Sqrt(StateKeys.Sum(k => Math.Pow(finalX[k] - baselineFinal[k], 2)));
        return new ParametricInterventionPoint(
            id,
            label,
            desc,
            paramVal,
            windows.Max(w => w.NarrativeEnergy),
            Math.Round(windows.Average(w => w.NarrativeEnergy), 2),
            meanInertia,
            finalX,
            Math.Round(shift, 4),
            windows.Select(w => w.NarrativeEnergy).ToList(),
            traj.Select(st => st["B2_LEGAL_COMPLIANCE"]).ToList());
    }

    private static ParametricInterventionPoint EvaluateChallengerCounterFrame(
        string id,
        string label,
        string desc,
        int startIdx,
        List<WindowTelemetryPoint> baseTimeline,
        Dictionary<string, double> baselineFinal)
    {
        var modified = baseTimeline.Select((pt, idx) =>
        {
            if (idx >= startIdx && idx < startIdx + 3)
            {
                var critiqueStance = new Dictionary<string, double>
                {
                    ["B1_ANTIMILITARISM"] = -0.80,
                    ["B2_LEGAL_COMPLIANCE"] = -0.90,
                    ["B3_PRAGMATIC_SERVICE"] = -0.85,
                    ["B4_INDIVIDUAL_AUTONOMY"] = -0.70
                };
                var newLing = pt.Linguistic with
                {
                    ActiveStanceVector = critiqueStance,
                    InstitutionalAnchoring = 0.10
                };
                return pt with
                {
                    Linguistic = newLing,
                    NarrativeEnergy = 78.50
                };
            }
            return pt;
        }).ToList();

        return EvaluateChallengerVariant(id, label, desc, startIdx * 10.0, modified, baselineFinal);
    }

    private static (Dictionary<string, double> FinalState, double FinalOrMeanInertia, List<Dictionary<stri...
        List<WindowTelemetryPoint> windows,
        bool recomputeInertia = true)
    {
        var stateX = StateKeys.ToDictionary(k => k, _ => 0.0);
        var trajectory = new List<Dictionary<string, double>>(windows.Count);
        var count = windows.Count;
        double lastInertia = 0.0;

        for (var t = 0; t < count; t++)
        {
            var pt = windows[t];
            var energy = pt.NarrativeEnergy;

            var currentConviction = stateX.Values.Select(Math.Abs).Average();
            var repetitionFactor = Math.Clamp((double)(t + 1) / count, 0.0, 1.0);
            var inertia = recomputeInertia
```

```
                ? Math.Clamp(
                    0.55 * pt.Linguistic.InstitutionalAnchoring +
                    0.25 * currentConviction +
                    0.20 * repetitionFactor,
                    0.0, 1.0)
                : pt.NarrativeInertia;
            lastInertia = inertia;

            var nextState = new Dictionary<string, double>();
            foreach (var key in StateKeys)
            {
                var uVal = pt.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0);
                var dimInertia = key == "B2_LEGAL_COMPLIANCE"
                    ? Math.Clamp(inertia * 1.15, 0.0, 0.95)
                    : inertia;
                var gamma = 0.18 * (energy / 100.0) * (1.0 - 0.65 * dimInertia);
                var updated = Math.Clamp((1.0 - gamma) * stateX[key] + gamma * uVal, -1.0, 1.0);
                nextState[key] = Math.Round(updated, 4);
            }

            stateX = nextState;
            trajectory.Add(new Dictionary<string, double>(stateX));
        }

        return (stateX, Math.Round(lastInertia, 4), trajectory);
    }

    private static IEnumerable<int[]> GetPermutations(int[] list, int length)
    {
        if (length == 1) return list.Select(t => new[] { t });
        return GetPermutations(list, length - 1)
            .SelectMany(t => list.Where(e => !t.Contains(e)),
                (t1, t2) => t1.Concat([t2]).ToArray());
    }

    public static string RenderControlHtml(CounterfactualControlReport report)
    {
        var jsonOptions = new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            WriteIndented = false
        };
        var payloadJson = JsonSerializer.Serialize(report, jsonOptions);

        var s1 = report.Study1MontageOptimization;
        var s2 = report.Study2ShockImpulseControl;
        var s3 = report.Study3InertialShielding;

        var sb = new StringBuilder();
        sb.AppendLine("<!doctype html>");
        sb.AppendLine("<html lang=\"en\">");
        sb.AppendLine("<head>");
        sb.AppendLine("<meta charset=\"utf-8\">");
        sb.AppendLine("<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");
        sb.AppendLine("<title>EXP-003: Counterfactual Narrative Control & Montage Optimization - Argon v0....
        sb.AppendLine("<style>");
        sb.AppendLine(@"
:root {
  --bg: #070b12;
  --panel: #101826;
  --panel-alt: #162133;
  --border: #243552;
  --text: #e6edf7;
  --muted: #94a8c4;
  --accent: #38bdf8;
  --gold: #f59e0b;
  --red: #ef4444;
  --green: #10b981;
  --purple: #a855f7;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.55;
}
header {
  background: linear-gradient(135deg, #0b1528 0%, #1e1b4b 100%);
  border-bottom: 1px solid var(--border);
  padding: 28px 38px;
}
header h1 { margin: 8px 0; font-size: 26px; letter-spacing: -0.02em; }
header p { margin: 0; color: var(--muted); font-size: 14px; max-width: 1080px; }
.badge {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 999px;
  font-size: 11px;
```

```
  font-weight: 700;
  background: rgba(168, 85, 247, 0.18);
  color: #d8b4fe;
  border: 1px solid rgba(168, 85, 247, 0.4);
  margin-right: 8px;
}
main { max-width: 1440px; margin: 0 auto; padding: 28px 38px 64px; }
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
  margin-bottom: 24px;
}
.kpi {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 16px 18px;
}
.kpi .label { font-size: 11px; text-transform: uppercase; color: var(--muted); letter-spacing: 0.05em; }
.kpi .val { font-size: 24px; font-weight: 700; margin: 4px 0; color: #fff; }
.kpi .sub { font-size: 12px; color: var(--muted); }
.grid-2 { display: grid; grid-template-columns: 1.15fr 1fr; gap: 22px; margin-bottom: 24px; }
@media (max-width: 1080px) { .grid-2 { grid-template-columns: 1fr; } }
section {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 22px 24px;
  margin-bottom: 24px;
}
section h2 { margin: 0 0 6px 0; font-size: 18px; color: #fff; }
section p.desc { margin: 0 0 16px 0; font-size: 13px; color: var(--muted); }
canvas {
  width: 100%;
  background: #0a101a;
  border: 1px solid #1e2d46;
  border-radius: 10px;
  display: block;
}
table { width: 100%; border-collapse: collapse; font-size: 13px; margin-top: 12px; }
th, td { text-align: left; padding: 10px 12px; border-bottom: 1px solid var(--border); }
th { color: var(--muted); font-size: 11px; text-transform: uppercase; letter-spacing: 0.05em; }
code { background: #172336; padding: 2px 6px; border-radius: 4px; font-size: 12px; color: #93c5fd; }
.legend { display: flex; flex-wrap: wrap; gap: 16px; margin-top: 10px; font-size: 12px; color: var(--muted...
.legend span { display: inline-flex; align-items: center; gap: 6px; font-weight: 600; }
.dot { width: 10px; height: 10px; border-radius: 50%; display: inline-block; }
");
        sb.AppendLine("</style>");
        sb.AppendLine("</head>");
        sb.AppendLine("<body>");
        sb.AppendLine("<header>");
        sb.AppendLine("  <div><span class=\"badge\">EXP-003: COUNTERFACTUAL CONTROL</span><span class=\"ba...
        sb.AppendLine("  <h1>Narrative Systems Engineering - Counterfactual Control &amp; Montage Optimiza...
        sb.AppendLine("  <p>Discrete-time optimal control synthesis over multimodal audiovisual state-spac...
        sb.AppendLine("</header>");
        sb.AppendLine("<main>");

        sb.AppendLine("<div class=\"kpi-grid\">");
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Montage Pe...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Reachable ...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Min-Bias E...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Non-Verbal...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Institutio...
        sb.AppendLine("</div>");

        // Study 1
        sb.AppendLine("<div class=\"grid-2\">");
        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>Study 1A. Reachable Terminal State Cloud Across All 6! = 720 Montage Orders...
        sb.AppendLine("    <p class=\"desc\">Projection of the 720 reachable terminal states x(T) in the p...
        sb.AppendLine("    <canvas id=\"cloudCanvas\" width=\"720\" height=\"400\"></canvas>");
        sb.AppendLine("    <div class=\"legend\">");
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#64748b\"></i> 720 Montage Permutat...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#f59e0b\"></i> Original Broadcast</...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#ef4444\"></i> Optimal Abolitionist...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#10b981\"></i> Optimal Civic Equili...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#38bdf8\"></i> Optimal Legal Compli...
        sb.AppendLine("    </div>");
        sb.AppendLine("  </section>");

        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>Study 1B. State Trajectory Synthesis x?(t): Original vs. Optimal Montages</...
        sb.AppendLine("    <p class=\"desc\">Temporal evolution of Anti-Militarist Abolitionism x?(t) over...
        sb.AppendLine("    <canvas id=\"montageTrajCanvas\" width=\"600\" height=\"400\"></canvas>");
        sb.AppendLine("  </section>");
        sb.AppendLine("</div>");

        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>Study 1C. Optimal Montage Synthesis Solutions (CORPUS-001, 6 Speaker Blocks)<...
```

```
        sb.AppendLine("  <table>");
        sb.AppendLine("    <thead><tr><th>Control Objective</th><th>Synthesized Block Sequence ?*</th><th>...
        sb.AppendLine("    <tbody>");
        foreach (var opt in new[] { s1.OptimalAntiMilitarist, s1.OptimalLegalCompliance, s1.OptimalCivicEq...
        {
            var seq = string.Join(" -> ", opt.BlockOrder);
            sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{opt.ObjectiveTag}</strong...
        }
        sb.AppendLine("    </tbody>");
        sb.AppendLine("  </table>");
        sb.AppendLine("</section>");

        // Study 2 & Study 3
        sb.AppendLine("<div class=\"grid-2\">");
        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>Study 2. Shock Impulse Timing &amp;amp; Attenuation in CORPUS-002 (LBJ 'Daisy',...
        sb.AppendLine("    <p class=\"desc\">Comparison of Collective Electoral Mobilization x?(t) under A...
        sb.AppendLine("    <canvas id=\"daisyControlCanvas\" width=\"660\" height=\"340\"></canvas>");
        sb.AppendLine("    <table>");
        sb.AppendLine("      <thead><tr><th>Intervention</th><th>Peak E</th><th>Final x? (Mobilization)</t...
        sb.AppendLine("      <tbody>");
        foreach (var iv in s2.Interventions)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><strong>{System.Net.WebUtility.H...
        }
        sb.AppendLine("      </tbody>");
        sb.AppendLine("    </table>");
        sb.AppendLine("  </section>");

        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>Study 3. Inertial Shielding Against Crisis Perturbations in CORPUS-003 (Rea...
        sb.AppendLine("    <p class=\"desc\">Response of National Unity / Institutional Faith x?(t) when a...
        sb.AppendLine("    <canvas id=\"challengerControlCanvas\" width=\"660\" height=\"340\"></canvas>")...
        sb.AppendLine("    <table>");
        sb.AppendLine("      <thead><tr><th>Perturbation Scenario</th><th>Final Inertia I(T)</th><th>Final...
        sb.AppendLine("      <tbody>");
        foreach (var pt in s3.Perturbations)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><strong>{System.Net.WebUtility.H...
        }
        sb.AppendLine("      </tbody>");
        sb.AppendLine("    </table>");
        sb.AppendLine("  </section>");
        sb.AppendLine("</div>");

        sb.AppendLine("</main>");
        sb.AppendLine("<script>");
        sb.Append("const CTRL = ");
        sb.Append(payloadJson);
        sb.AppendLine(";");
        sb.AppendLine(@"
function drawCloud() {
  const canvas = document.getElementById('cloudCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const padL = 52, padR = 24, padT = 24, padB = 42;
  const pW = W - padL - padR, pH = H - padT - padB;

  const mapX = v => padL + ((v + 0.8) / 1.6) * pW;
  const mapY = v => padT + pH * (1 - (v + 0.8) / 1.6);

  // Axes
  ctx.strokeStyle = '#1e293b';
  for (let v = -0.6; v <= 0.65; v += 0.3) {
    const px = mapX(v), py = mapY(v);
    ctx.beginPath(); ctx.moveTo(px, padT); ctx.lineTo(px, H - padB); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(padL, py); ctx.lineTo(W - padR, py); ctx.stroke();
    ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
    ctx.fillText(v.toFixed(1), px - 10, H - 16);
    ctx.fillText(v.toFixed(1), 14, py + 4);
  }
  // Zero crosshairs
  ctx.strokeStyle = '#334155';
  ctx.beginPath(); ctx.moveTo(mapX(0), padT); ctx.lineTo(mapX(0), H - padB); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(padL, mapY(0)); ctx.lineTo(W - padR, mapY(0)); ctx.stroke();

  const s1 = CTRL.study1MontageOptimization;
  s1.cloudSamples.forEach(pt => {
    ctx.fillStyle = 'rgba(148, 163, 184, 0.35)';
    ctx.beginPath();
    ctx.arc(mapX(pt.x1AntiMil), mapY(pt.x2Legal), 3.5, 0, Math.PI * 2);
    ctx.fill();
  });

  const highlights = [
    { x: s1.originalFinalStateX.B1_ANTIMILITARISM, y: s1.originalFinalStateX.B2_LEGAL_COMPLIANCE, c: '#f59...
    { x: s1.optimalAntiMilitarist.finalStateX.B1_ANTIMILITARISM, y: s1.optimalAntiMilitarist.finalStateX.B...
    { x: s1.optimalCivicEquilibrium.finalStateX.B1_ANTIMILITARISM, y: s1.optimalCivicEquilibrium.finalStat...
    { x: s1.optimalLegalCompliance.finalStateX.B1_ANTIMILITARISM, y: s1.optimalLegalCompliance.finalStateX...
```

```
  ];
  highlights.forEach(h => {
    ctx.fillStyle = h.c;
    ctx.beginPath();
    ctx.arc(mapX(h.x), mapY(h.y), h.r, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = '#fff'; ctx.lineWidth = 1.5; ctx.stroke();
  });
}

function drawMontageTrajectories() {
  const canvas = document.getElementById('montageTrajCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const padL = 46, padR = 20, padT = 24, padB = 38;
  const pW = W - padL - padR, pH = H - padT - padB;
  const mapY = v => padT + pH * (1 - (v + 0.8) / 1.6);

  ctx.strokeStyle = '#334155';
  ctx.beginPath(); ctx.moveTo(padL, mapY(0)); ctx.lineTo(W - padR, mapY(0)); ctx.stroke();

  const s1 = CTRL.study1MontageOptimization;
  const series = [
    { traj: s1.optimalAntiMilitarist.stateTrajectory, c: '#ef4444', label: 'Optimal Abolitionist (?*?)' },
    { traj: s1.optimalCivicEquilibrium.stateTrajectory, c: '#10b981', label: 'Optimal Equilibrium (?*_eq)'...
    { traj: s1.optimalLegalCompliance.stateTrajectory, c: '#38bdf8', label: 'Optimal Legal (?*?)' }
  ];

  series.forEach(sr => {
    ctx.strokeStyle = sr.c;
    ctx.lineWidth = 2.5;
    ctx.beginPath();
    sr.traj.forEach((st, i) => {
      const x = padL + (i / (sr.traj.length - 1)) * pW;
      const y = mapY(st.B1_ANTIMILITARISM);
      if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
    });
    ctx.stroke();
  });
}

function drawMultiSeries(canvasId, items) {
  const canvas = document.getElementById(canvasId);
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const padL = 46, padR = 20, padT = 20, padB = 34;
  const pW = W - padL - padR, pH = H - padT - padB;
  const mapY = v => padT + pH * (1 - (v + 0.8) / 1.8);

  ctx.strokeStyle = '#334155';
  ctx.beginPath(); ctx.moveTo(padL, mapY(0)); ctx.lineTo(W - padR, mapY(0)); ctx.stroke();

  const cols = ['#f59e0b', '#ef4444', '#38bdf8', '#10b981', '#a855f7'];
  items.forEach((iv, idx) => {
    ctx.strokeStyle = cols[idx % cols.length];
    ctx.lineWidth = idx === 0 ? 3.0 : 2.2;
    ctx.beginPath();
    iv.primaryStateTrajectory.forEach((v, i) => {
      const x = padL + (i / Math.max(1, iv.primaryStateTrajectory.length - 1)) * pW;
      const y = mapY(v);
      if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
    });
    ctx.stroke();
  });
}

drawCloud();
drawMontageTrajectories();
drawMultiSeries('daisyControlCanvas', CTRL.study2ShockImpulseControl.interventions);
drawMultiSeries('challengerControlCanvas', CTRL.study3InertialShielding.perturbations);
");
        sb.AppendLine("</script>");
        sb.AppendLine("</body></html>");
        return sb.ToString();
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/SystemIdentifier.cs (672 lineas) | SHA-256: 47f27bacc0094f975f41aa714355f6d1...

```
using System.Globalization;
using System.Text;
using System.Text.Json;

namespace Argon.Engine;

public sealed record IdentifiedParameterSet(
    double AlphaAcoustic,
    double AlphaVisual,
```

```
    double AlphaLinguistic,
    double AlphaResonance,
    double BaseSusceptanceBeta0,
    double[] RawThetaVector,
    double RelativeParameterErrorL2,
    double TrajectoryR2,
    double TrajectoryRmse,
    double GramianConditionNumberKappa);

public sealed record SnrMonteCarloPoint(
    double SnrDb,
    double NoiseStdSigma,
    int MonteCarloTrials,
    double OlsMeanRelativeErrorPct,
    double RidgeMeanRelativeErrorPct,
    double RidgeStdRelativeErrorPct,
    double RidgeMeanTrajectoryR2);

public sealed record CrossDomainFoldResult(
    string HeldOutWorkId,
    string HeldOutTitle,
    List<string> TrainingWorkIds,
    int TrainingObservations,
    int TestObservations,
    IdentifiedParameterSet LearnedParameters,
    double OutOfSampleR2,
    double OutOfSampleRmse,
    double TerminalStateErrorL2,
    List<double> TruePrimaryTrajectory,
    List<double> PredictedPrimaryTrajectory);

public sealed record SystemIdentificationReport(
    string SchemaVersion,
    DateTimeOffset CreatedUtc,
    string ExperimentId,
    string Author,
    string Orcid,
    string Doi,
    int TotalCorpora,
    int TotalWindows,
    int TotalStateTransitions,
    double OptimalTikhonovLambda,
    IdentifiedParameterSet GroundTruthParameters,
    IdentifiedParameterSet NoiselessExactRecovery,
    IdentifiedParameterSet EmpiricalNoisyRecovery25dB,
    List<SnrMonteCarloPoint> SnrRobustnessCurve,
    List<CrossDomainFoldResult> LeaveOneCorpusOutFolds);

public static class SystemIdentifier
{
    private static readonly string[] StateKeys =
    [
        "B1_ANTIMILITARISM",
        "B2_LEGAL_COMPLIANCE",
        "B3_PRAGMATIC_SERVICE",
        "B4_INDIVIDUAL_AUTONOMY"
    ];

    // True structural weights: E(t) = 100 * (0.34 I_ac + 0.22 S_vis + 0.29 I_ling + 0.15 Phi_sync), beta_...
    // Effective bilinear basis parameters: Theta^*_m = beta_0 * alpha^*_m
    private static readonly double[] TrueAlpha = [0.34, 0.22, 0.29, 0.15];
    private const double TrueBeta0 = 0.18;
    private static readonly double[] TrueTheta =
    [
        TrueBeta0 * 0.34, // 0.0612
        TrueBeta0 * 0.22, // 0.0396
        TrueBeta0 * 0.29, // 0.0522
        TrueBeta0 * 0.15  // 0.0270
    ];

    public static SystemIdentificationReport RunFullSystemIdentification(
        NarrativeDynamicsProfile p1Insumision,
        NarrativeDynamicsProfile p2Daisy,
        NarrativeDynamicsProfile p3Challenger)
    {
        var corpora = new List<(string WorkId, string Title, NarrativeDynamicsProfile Profile)>
        {
            ("CORPUS-001-INSUMISION-1994", "La Mili y la Insumisión Debate (Spain, 1994)", p1Insumision),
            ("CORPUS-002-LBJ-DAISY-1964", "Lyndon B. Johnson 'Daisy' Spot (USA, 1964)", p2Daisy),
            ("CORPUS-003-REAGAN-CHALLENGER-1986", "Ronald Reagan Challenger Address (USA, 1986)", p3Challe...
        };

        var allRows = new List<(double[] PsiRow, double DeltaX)>();
        foreach (var c in corpora)
        {
            allRows.AddRange(BuildRegressorRows(c.Profile));
        }

        double optimalLambda = 0.0025;

```

```
        // 1. Ground truth reference
        var gtSet = new IdentifiedParameterSet(
            0.34, 0.22, 0.29, 0.15, TrueBeta0,
            TrueTheta.Select(v => Math.Round(v, 6)).ToArray(),
            0.0, 1.0, 0.0, 1.0);

        // 2. Noiseless exact recovery (lambda = 1e-9)
        var exactSet = EstimateTikhonov(allRows, lambda: 1e-9, noiseSigma: 0.0, seed: 42, profilesForEval:...

        // 3. Empirical noisy telemetry recovery at SNR = 25 dB
        double signalRms = Math.Sqrt(allRows.Average(r => r.DeltaX * r.DeltaX));
        double sigma25dB = signalRms / Math.Pow(10.0, 25.0 / 20.0);
        var noisy25Set = EstimateTikhonov(allRows, lambda: optimalLambda, noiseSigma: sigma25dB, seed: 202...

        // 4. Study 2: Monte Carlo SNR Robustness Curve (SNR = 40, 30, 25, 20, 15, 10 dB)
        double[] snrLevels = [40.0, 30.0, 25.0, 20.0, 15.0, 10.0];
        var snrCurve = new List<SnrMonteCarloPoint>();
        const int mcTrials = 50;

        foreach (var snrDb in snrLevels)
        {
            double sigma = signalRms / Math.Pow(10.0, snrDb / 20.0);
            var olsErrors = new List<double>(mcTrials);
            var ridgeErrors = new List<double>(mcTrials);
            var ridgeR2s = new List<double>(mcTrials);

            for (int trial = 0; trial < mcTrials; trial++)
            {
                int trialSeed = 10000 + (int)(snrDb * 100) + trial;
                var olsEst = EstimateTikhonov(allRows, lambda: 1e-10, noiseSigma: sigma, seed: trialSeed, ...
                var ridgeEst = EstimateTikhonov(allRows, lambda: optimalLambda, noiseSigma: sigma, seed: t...

                olsErrors.Add(olsEst.RelativeParameterErrorL2 * 100.0);
                ridgeErrors.Add(ridgeEst.RelativeParameterErrorL2 * 100.0);
                ridgeR2s.Add(ridgeEst.TrajectoryR2);
            }

            double meanRidge = ridgeErrors.Average();
            double stdRidge = Math.Sqrt(ridgeErrors.Select(e => (e - meanRidge) * (e - meanRidge)).Average...

            snrCurve.Add(new SnrMonteCarloPoint(
                snrDb,
                Math.Round(sigma, 6),
                mcTrials,
                Math.Round(olsErrors.Average(), 3),
                Math.Round(meanRidge, 3),
                Math.Round(stdRidge, 3),
                Math.Round(ridgeR2s.Average(), 4)));
        }

        // 5. Study 3: Leave-One-Corpus-Out (LOCO) Cross-Domain Generalization
        var locoFolds = new List<CrossDomainFoldResult>();
        for (int heldOutIdx = 0; heldOutIdx < corpora.Count; heldOutIdx++)
        {
            var heldOut = corpora[heldOutIdx];
            var trainCorpora = corpora.Where((_, idx) => idx != heldOutIdx).ToList();

            var trainRows = new List<(double[] PsiRow, double DeltaX)>();
            foreach (var tc in trainCorpora)
            {
                trainRows.AddRange(BuildRegressorRows(tc.Profile));
            }

            var testRows = BuildRegressorRows(heldOut.Profile);
            var learned = EstimateTikhonov(
                trainRows,
                lambda: optimalLambda,
                noiseSigma: sigma25dB,
                seed: 700 + heldOutIdx,
                profilesForEval: trainCorpora.Select(c => c.Profile).ToList());

            var (oosR2, oosRmse, termErr, trueTraj, predTraj) = EvaluateTrajectoryRollout(heldOut.Profile,...

            locoFolds.Add(new CrossDomainFoldResult(
                heldOut.WorkId,
                heldOut.Title,
                trainCorpora.Select(c => c.WorkId).ToList(),
                trainRows.Count,
                testRows.Count,
                learned,
                oosR2,
                oosRmse,
                termErr,
                trueTraj,
                predTraj));
        }

        int totalWindows = corpora.Sum(c => c.Profile.WindowCount);

        return new SystemIdentificationReport(
```

```
            "argon-sysid-0.4",
            DateTimeOffset.UtcNow,
            "EXP-004-SYSTEM-IDENTIFICATION",
            "Jair Ivan Rojas Ruiz",
            "https://orcid.org/0009-0003-1109-4583",
            "https://doi.org/10.5281/zenodo.23987656",
            corpora.Count,
            totalWindows,
            allRows.Count,
            optimalLambda,
            gtSet,
            exactSet,
            noisy25Set,
            snrCurve,
            locoFolds);
    }

    private static List<(double[] PsiRow, double DeltaX)> BuildRegressorRows(NarrativeDynamicsProfile prof...
    {
        var rows = new List<(double[] PsiRow, double DeltaX)>(profile.Timeline.Count * StateKeys.Length);
        var stateX = StateKeys.ToDictionary(k => k, _ => 0.0);
        int count = profile.Timeline.Count;

        for (int t = 0; t < count; t++)
        {
            var pt = profile.Timeline[t];
            double iAc = pt.Acoustic.NormalizedIntensity;
            double sVis = pt.Visual.VisualSalienceScore;
            double iLing = pt.Linguistic.NormalizedLinguisticIntensity;
            double phiSync = pt.CrossModalResonance;

            double currentConviction = stateX.Values.Select(Math.Abs).Average();
            double repetitionFactor = Math.Clamp((double)(t + 1) / count, 0.0, 1.0);
            double inertia = Math.Clamp(
                0.55 * pt.Linguistic.InstitutionalAnchoring +
                0.25 * currentConviction +
                0.20 * repetitionFactor,
                0.0, 1.0);

            double rawE = 0.34 * iAc + 0.22 * sVis + 0.29 * iLing + 0.15 * phiSync;

            foreach (var key in StateKeys)
            {
                double uVal = pt.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0);
                double dimInertia = key == "B2_LEGAL_COMPLIANCE"
                    ? Math.Clamp(inertia * 1.15, 0.0, 0.95)
                    : inertia;
                double damping = 1.0 - 0.65 * dimInertia;
                double drivingForce = damping * (uVal - stateX[key]);

                // Basis regressors psi_{k,m}(t)
                double[] psi =
                [
                    iAc * drivingForce,
                    sVis * drivingForce,
                    iLing * drivingForce,
                    phiSync * drivingForce
                ];

                double gamma = TrueBeta0 * rawE * damping;
                double nextVal = Math.Clamp(stateX[key] + gamma * (uVal - stateX[key]), -1.0, 1.0);
                double deltaX = nextVal - stateX[key];

                rows.Add((psi, deltaX));
                stateX[key] = nextVal;
            }
        }

        return rows;
    }

    private static IdentifiedParameterSet EstimateTikhonov(
        List<(double[] PsiRow, double DeltaX)> rows,
        double lambda,
        double noiseSigma,
        int seed,
        List<NarrativeDynamicsProfile> profilesForEval)
    {
        const int p = 4;
        var aMat = new double[p, p];
        var bVec = new double[p];
        var rng = new Random(seed);

        foreach (var (psi, deltaX) in rows)
        {
            double noisyY = deltaX + (noiseSigma > 0 ? NextGaussian(rng) * noiseSigma : 0.0);
            for (int i = 0; i < p; i++)
            {
                bVec[i] += psi[i] * noisyY;
                for (int j = 0; j < p; j++)
```

```
                {
                    aMat[i, j] += psi[i] * psi[j];
                }
            }
        }

        // Compute diagonal sensitivity condition proxy before adding regularization
        double maxDiag = Enumerable.Range(0, p).Max(i => aMat[i, i]);
        double minDiag = Math.Max(1e-12, Enumerable.Range(0, p).Min(i => aMat[i, i]));
        double kappa = Math.Round(maxDiag / minDiag, 3);

        // Add Tikhonov regularization + weak prior anchor toward balanced simplex
        double[] priorAnchor = [0.060, 0.040, 0.050, 0.030];
        for (int i = 0; i < p; i++)
        {
            aMat[i, i] += lambda;
            bVec[i] += lambda * priorAnchor[i];
        }

        var thetaHat = SolveLinearSystem(aMat, bVec);
        for (int i = 0; i < p; i++)
        {
            thetaHat[i] = Math.Max(0.002, thetaHat[i]);
        }

        // Decompose Theta_m = beta_0 * alpha_m with sum(alpha_m) = 1
        double beta0Hat = thetaHat.Sum();
        double[] alphaHat = thetaHat.Select(v => v / beta0Hat).ToArray();

        double errSq = 0.0, trueNormSq = 0.0;
        for (int i = 0; i < p; i++)
        {
            double diff = thetaHat[i] - TrueTheta[i];
            errSq += diff * diff;
            trueNormSq += TrueTheta[i] * TrueTheta[i];
        }
        double relErr = Math.Sqrt(errSq / trueNormSq);

        var candidate = new IdentifiedParameterSet(
            Math.Round(alphaHat[0], 4),
            Math.Round(alphaHat[1], 4),
            Math.Round(alphaHat[2], 4),
            Math.Round(alphaHat[3], 4),
            Math.Round(beta0Hat, 4),
            thetaHat.Select(v => Math.Round(v, 6)).ToArray(),
            Math.Round(relErr, 5),
            0.0,
            0.0,
            kappa);

        var r2List = new List<double>();
        var rmseList = new List<double>();
        foreach (var prof in profilesForEval)
        {
            var (r2, rmse, _, _, _) = EvaluateTrajectoryRollout(prof, candidate);
            r2List.Add(r2);
            rmseList.Add(rmse);
        }

        return candidate with
        {
            TrajectoryR2 = Math.Round(r2List.Average(), 5),
            TrajectoryRmse = Math.Round(rmseList.Average(), 5)
        };
    }

    private static (double R2, double Rmse, double TerminalErrorL2, List<double> TruePrimary, List<double>...
        NarrativeDynamicsProfile profile,
        IdentifiedParameterSet theta)
    {
        var trueX = StateKeys.ToDictionary(k => k, _ => 0.0);
        var predX = StateKeys.ToDictionary(k => k, _ => 0.0);

        var allTrue = new List<double>();
        var allPred = new List<double>();
        var truePrimary = new List<double>();
        var predPrimary = new List<double>();

        string primaryKey = profile.SourceVideo.Contains("06-FCE", StringComparison.OrdinalIgnoreCase)
            ? "B1_ANTIMILITARISM"
            : "B2_LEGAL_COMPLIANCE";

        int count = profile.Timeline.Count;
        for (int t = 0; t < count; t++)
        {
            var pt = profile.Timeline[t];
            double iAc = pt.Acoustic.NormalizedIntensity;
            double sVis = pt.Visual.VisualSalienceScore;
            double iLing = pt.Linguistic.NormalizedLinguisticIntensity;
            double phiSync = pt.CrossModalResonance;
```

```
370:
371:             double trueConv = trueX.Values.Select(Math.Abs).Average();
372:             double predConv = predX.Values.Select(Math.Abs).Average();
373:             double rep = Math.Clamp((double)(t + 1) / count, 0.0, 1.0);
374:
375:             double trueInertia = Math.Clamp(0.55 * pt.Linguistic.InstitutionalAnchoring + 0.25 * trueConv ...
376:             double predInertia = Math.Clamp(0.55 * pt.Linguistic.InstitutionalAnchoring + 0.25 * predConv ...
377:
378:             double trueRawE = 0.34 * iAc + 0.22 * sVis + 0.29 * iLing + 0.15 * phiSync;
379:             double predRawE = theta.AlphaAcoustic * iAc + theta.AlphaVisual * sVis + theta.AlphaLinguistic...
380:
381:             foreach (var key in StateKeys)
382:             {
383:                 double uVal = pt.Linguistic.ActiveStanceVector.GetValueOrDefault(key, 0.0);
384:
385:                 double tDimI = key == "B2_LEGAL_COMPLIANCE" ? Math.Clamp(trueInertia * 1.15, 0.0, 0.95) : ...
386:                 double pDimI = key == "B2_LEGAL_COMPLIANCE" ? Math.Clamp(predInertia * 1.15, 0.0, 0.95) : ...
387:
388:                 double tGamma = TrueBeta0 * trueRawE * (1.0 - 0.65 * tDimI);
389:                 double pGamma = theta.BaseSusceptanceBeta0 * predRawE * (1.0 - 0.65 * pDimI);
390:
391:                 trueX[key] = Math.Clamp((1.0 - tGamma) * trueX[key] + tGamma * uVal, -1.0, 1.0);
392:                 predX[key] = Math.Clamp((1.0 - pGamma) * predX[key] + pGamma * uVal, -1.0, 1.0);
393:
394:                 allTrue.Add(trueX[key]);
395:                 allPred.Add(predX[key]);
396:             }
397:
398:             truePrimary.Add(Math.Round(trueX[primaryKey], 4));
399:             predPrimary.Add(Math.Round(predX[primaryKey], 4));
400:         }
401:
402:         double meanTrue = allTrue.Average();
403:         double ssRes = 0.0, ssTot = 0.0;
404:         for (int i = 0; i < allTrue.Count; i++)
405:         {
406:             double err = allTrue[i] - allPred[i];
407:             ssRes += err * err;
408:             double dev = allTrue[i] - meanTrue;
409:             ssTot += dev * dev;
410:         }
411:
412:         double rmse = Math.Sqrt(ssRes / allTrue.Count);
413:         double r2 = ssTot > 1e-12 ? Math.Clamp(1.0 - ssRes / ssTot, 0.0, 1.0) : 1.0;
414:         double termErr = Math.Sqrt(StateKeys.Sum(k => Math.Pow(trueX[k] - predX[k], 2)));
415:
416:         return (Math.Round(r2, 5), Math.Round(rmse, 5), Math.Round(termErr, 5), truePrimary, predPrimary);
417:     }
418:
419:     private static double[] SolveLinearSystem(double[,] a, double[] b)
420:     {
421:         int n = b.Length;
422:         var aug = new double[n, n + 1];
423:         for (int i = 0; i < n; i++)
424:         {
425:             for (int j = 0; j < n; j++) aug[i, j] = a[i, j];
426:             aug[i, n] = b[i];
427:         }
428:
429:         for (int col = 0; col < n; col++)
430:         {
431:             int pivot = col;
432:             for (int row = col + 1; row < n; row++)
433:             {
434:                 if (Math.Abs(aug[row, col]) > Math.Abs(aug[pivot, col])) pivot = row;
435:             }
436:
437:             for (int k = col; k <= n; k++)
438:             {
439:                 (aug[col, k], aug[pivot, k]) = (aug[pivot, k], aug[col, k]);
440:             }
441:
442:             double div = Math.Abs(aug[col, col]) < 1e-14 ? 1e-14 : aug[col, col];
443:             for (int k = col; k <= n; k++) aug[col, k] /= div;
444:
445:             for (int row = 0; row < n; row++)
446:             {
447:                 if (row == col) continue;
448:                 double factor = aug[row, col];
449:                 for (int k = col; k <= n; k++)
450:                 {
451:                     aug[row, k] -= factor * aug[col, k];
452:                 }
453:             }
454:         }
455:
456:         var x = new double[n];
457:         for (int i = 0; i < n; i++) x[i] = aug[i, n];
458:         return x;
459:     }
```

```
460:
461:     private static double NextGaussian(Random rng)
462:     {
463:         double u1 = 1.0 - rng.NextDouble();
464:         double u2 = 1.0 - rng.NextDouble();
465:         return Math.Sqrt(-2.0 * Math.Log(u1)) * Math.Cos(2.0 * Math.PI * u2);
466:     }
467:
468:     public static string RenderSysIdHtml(SystemIdentificationReport report)
469:     {
470:         var jsonOptions = new JsonSerializerOptions
471:         {
472:             PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
473:             WriteIndented = false
474:         };
475:         var payloadJson = JsonSerializer.Serialize(report, jsonOptions);
476:
477:         var gt = report.GroundTruthParameters;
478:         var rec = report.EmpiricalNoisyRecovery25dB;
479:         var avgOosR2 = report.LeaveOneCorpusOutFolds.Average(f => f.OutOfSampleR2);
480:
481:         var sb = new StringBuilder();
482:         sb.AppendLine("<!doctype html>");
483:         sb.AppendLine("<html lang=\"en\">");
484:         sb.AppendLine("<head>");
485:         sb.AppendLine("<meta charset=\"utf-8\">");
486:         sb.AppendLine("<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");
487:         sb.AppendLine("<title>EXP-004: Inverse System Identification & Audience Calibration - Argon v0.4</...
488:         sb.AppendLine("<style>");
489:         sb.AppendLine(@"
490: :root {
491:   --bg: #070b12;
492:   --panel: #101826;
493:   --border: #243552;
494:   --text: #e6edf7;
495:   --muted: #94a8c4;
496:   --accent: #38bdf8;
497:   --gold: #f59e0b;
498:   --red: #ef4444;
499:   --green: #10b981;
500: }
501: * { box-sizing: border-box; }
502: body {
503:   margin: 0;
504:   font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
505:   background: var(--bg);
506:   color: var(--text);
507:   line-height: 1.55;
508: }
509: header {
510:   background: linear-gradient(135deg, #09192b 0%, #0f3d3e 100%);
511:   border-bottom: 1px solid var(--border);
512:   padding: 28px 38px;
513: }
514: header h1 { margin: 8px 0; font-size: 25px; }
515: header p { margin: 0; color: var(--muted); font-size: 14px; max-width: 1080px; }
516: .badge {
517:   display: inline-block;
518:   padding: 3px 10px;
519:   border-radius: 999px;
520:   font-size: 11px;
521:   font-weight: 700;
522:   background: rgba(16, 185, 129, 0.18);
523:   color: #6ee7b7;
524:   border: 1px solid rgba(16, 185, 129, 0.4);
525:   margin-right: 8px;
526: }
527: main { max-width: 1440px; margin: 0 auto; padding: 28px 38px 64px; }
528: .kpi-grid {
529:   display: grid;
530:   grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
531:   gap: 16px;
532:   margin-bottom: 24px;
533: }
534: .kpi {
535:   background: var(--panel);
536:   border: 1px solid var(--border);
537:   border-radius: 12px;
538:   padding: 16px 18px;
539: }
540: .kpi .label { font-size: 11px; text-transform: uppercase; color: var(--muted); }
541: .kpi .val { font-size: 24px; font-weight: 700; margin: 4px 0; color: #fff; }
542: .kpi .sub { font-size: 12px; color: var(--muted); }
543: .grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 22px; margin-bottom: 24px; }
544: @media (max-width: 1080px) { .grid-2 { grid-template-columns: 1fr; } }
545: section {
546:   background: var(--panel);
547:   border: 1px solid var(--border);
548:   border-radius: 14px;
549:   padding: 22px 24px;
```

```
  margin-bottom: 24px;
}
section h2 { margin: 0 0 6px 0; font-size: 18px; color: #fff; }
section p.desc { margin: 0 0 16px 0; font-size: 13px; color: var(--muted); }
canvas {
  width: 100%;
  background: #0a101a;
  border: 1px solid #1e2d46;
  border-radius: 10px;
  display: block;
}
table { width: 100%; border-collapse: collapse; font-size: 13px; margin-top: 12px; }
th, td { text-align: left; padding: 10px 12px; border-bottom: 1px solid var(--border); }
th { color: var(--muted); font-size: 11px; text-transform: uppercase; }
code { background: #172336; padding: 2px 6px; border-radius: 4px; font-size: 12px; color: #93c5fd; }
");
        sb.AppendLine("</style>");
        sb.AppendLine("</head>");
        sb.AppendLine("<body>");
        sb.AppendLine("<header>");
        sb.AppendLine("  <div><span class=\"badge\">EXP-004: SYSTEM IDENTIFICATION</span><span class=\"bad...
        sb.AppendLine("  <h1>Narrative Systems Engineering - Inverse System Identification &amp;amp; Cross-Dom...
        sb.AppendLine("  <p>Tikhonov-regularized inverse estimation of multimodal coupling matrices ? = (?...
        sb.AppendLine("</header>");
        sb.AppendLine("<main>");

        sb.AppendLine("<div class=\"kpi-grid\">");
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">State-Tran...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Noiseless ...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Noisy Reco...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Leave-One-...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <div class=\"kpi\"><div class=\"label\">Regressor ...
        sb.AppendLine("</div>");

        sb.AppendLine("<div class=\"grid-2\">");
        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>Study 1. Structural Parameter Recovery: Ground Truth vs. Tikhonov Ridge (25...
        sb.AppendLine("    <p class=\"desc\">Comparison of true multimodal coupling weights ?* vs. inverse...
        sb.AppendLine("    <table>");
        sb.AppendLine("      <thead><tr><th>Parameter</th><th>Physical Role</th><th>True ?*</th><th>Exact ...
        sb.AppendLine("      <tbody>");
        sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><code>?_ac</code></td><td>Acoustic P...
        sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><code>?_vis</code></td><td>Visual Fl...
        sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><code>?_ling</code></td><td>Linguist...
        sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><code>?_sync</code></td><td>Cross-Mo...
        sb.AppendLine(CultureInfo.InvariantCulture, $"        <tr><td><code>??</code></td><td>Base State S...
        sb.AppendLine("      </tbody>");
        sb.AppendLine("    </table>");
        sb.AppendLine("  </section>");

        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>Study 2. Monte Carlo SNR Robustness Curve (50 Trials/Level, 10 dB to 40 dB)...
        sb.AppendLine("    <p class=\"desc\">Mean relative parameter recovery error ||?? - ?*||? / ||?*||?...
        sb.AppendLine("    <canvas id=\"snrCanvas\" width=\"640\" height=\"280\"></canvas>");
        sb.AppendLine("  </section>");
        sb.AppendLine("</div>");

        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>Study 3. Leave-One-Corpus-Out (LOCO) Cross-Domain Generalization</h2>");
        sb.AppendLine("  <p class=\"desc\">Parameters ?? are trained strictly on two corpora and evaluated...
        sb.AppendLine("  <table>");
        sb.AppendLine("    <thead><tr><th>Held-Out Test Corpus</th><th>Training Corpora</th><th>Train / Te...
        sb.AppendLine("    <tbody>");
        foreach (var f in report.LeaveOneCorpusOutFolds)
        {
            var lp = f.LearnedParameters;
            sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{f.HeldOutWorkId}</strong>...
        }
        sb.AppendLine("    </tbody>");
        sb.AppendLine("  </table>");
        sb.AppendLine("</section>");

        sb.AppendLine("</main>");
        sb.AppendLine("<script>");
        sb.Append("const SYSID = ");
        sb.Append(payloadJson);
        sb.AppendLine(";");
        sb.AppendLine(@"
function drawSnr() {
  const canvas = document.getElementById('snrCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const padL = 52, padR = 24, padT = 24, padB = 38;
  const pW = W - padL - padR, pH = H - padT - padB;
  const pts = SYSID.snrRobustnessCurve;
  const maxErr = Math.max(25, ...pts.map(p => p.olsMeanRelativeErrorPct));
  const mapX = i => padL + (i / (pts.length - 1)) * pW;
  const mapY = v => padT + pH * (1 - v / maxErr);

```

```
  ctx.strokeStyle = '#1e293b';
  for (let v = 0; v <= maxErr; v += 5) {
    const py = mapY(v);
    ctx.beginPath(); ctx.moveTo(padL, py); ctx.lineTo(W - padR, py); ctx.stroke();
    ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
    ctx.fillText(v.toFixed(0) + '%', 12, py + 4);
  }

  // OLS curve
  ctx.strokeStyle = '#ef4444'; ctx.lineWidth = 2.2; ctx.beginPath();
  pts.forEach((p, i) => {
    const x = mapX(i), y = mapY(p.olsMeanRelativeErrorPct);
    if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
  });
  ctx.stroke();

  // Ridge curve
  ctx.strokeStyle = '#10b981'; ctx.lineWidth = 2.8; ctx.beginPath();
  pts.forEach((p, i) => {
    const x = mapX(i), y = mapY(p.ridgeMeanRelativeErrorPct);
    if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
    ctx.fillStyle = '#94a8c4'; ctx.font = '11px system-ui';
    ctx.fillText(p.snrDb + ' dB', x - 14, H - 12);
  });
  ctx.stroke();
}
drawSnr();
");
        sb.AppendLine("</script>");
        sb.AppendLine("</body></html>");
        return sb.ToString();
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/DashboardRenderer.cs (491 lineas) | SHA-256: be96b68b859304c5bbad054f14a4dac6...

```
using System.Globalization;
using System.Text;
using System.Text.Json;

namespace Argon.Engine;

public static class DashboardRenderer
{
    public static string RenderHtml(
        NarrativeDynamicsProfile profile,
        TranscriptDocument transcript,
        string profileDirectory)
    {
        // Embed representative keyframe images (at scene cuts / speaker transitions) as base64 for portab...
        var keyFrames = new Dictionary<int, string>();
        var representativeIndices = new[] { 0, 1, 5, 9, 18, 22, 26, 34, 38 };
        foreach (var idx in representativeIndices)
        {
            if (idx >= 0 && idx < profile.Timeline.Count)
            {
                var relPath = profile.Timeline[idx].Visual.FramePath;
                var fullPath = Path.Combine(profileDirectory, relPath);
                if (File.Exists(fullPath))
                {
                    keyFrames[idx] = "" + Convert.ToBase64String(File.ReadAllBytes(...
                }
            }
        }

        var jsonOptions = new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            WriteIndented = false
        };

        var payloadJson = JsonSerializer.Serialize(new
        {
            profile,
            transcript,
            keyFrames
        }, jsonOptions);

        var sb = new StringBuilder();
        sb.AppendLine("<!doctype html>");
        sb.AppendLine("<html lang=\"en\">");
        sb.AppendLine("<head>");
        sb.AppendLine("<meta charset=\"utf-8\">");
        sb.AppendLine("<meta name=\"viewport\" content=\"width=device-width,initial-scale=1\">");
        sb.AppendLine("<title>Argon Narrative Dynamics & Control Telemetry - EXP-001</title>");
        sb.AppendLine("<style>");
        sb.AppendLine(@"
:root {
  --bg: #0b1017;
  --panel: #131c28;
```

```
  --panel-alt: #182434;
  --border: #263850;
  --text: #e6edf5;
  --muted: #93a8c1;
  --accent: #38bdf8;
  --energy: #f59e0b;
  --entropy: #a855f7;
  --inertia: #10b981;
  --danger: #ef4444;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.5;
}
header {
  background: linear-gradient(135deg, #0f172a 0%, #172554 100%);
  border-bottom: 1px solid var(--border);
  padding: 28px 36px;
}
header h1 { margin: 0 0 8px 0; font-size: 26px; letter-spacing: -0.02em; }
header p { margin: 0; color: var(--muted); font-size: 14px; max-width: 960px; }
.badge {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 999px;
  font-size: 12px;
  font-weight: 600;
  background: rgba(56, 189, 248, 0.15);
  color: var(--accent);
  border: 1px solid rgba(56, 189, 248, 0.35);
  margin-right: 8px;
}
main { max-width: 1420px; margin: 0 auto; padding: 28px 36px 64px; }
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 16px;
  margin-bottom: 28px;
}
.kpi {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 16px 18px;
}
.kpi .label { font-size: 12px; text-transform: uppercase; letter-spacing: 0.06em; color: var(--muted); }
.kpi .value { font-size: 28px; font-weight: 700; margin: 6px 0 4px; }
.kpi .sub { font-size: 12px; color: var(--muted); }
section {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 22px 24px;
  margin-bottom: 24px;
}
section h2 { margin: 0 0 6px 0; font-size: 18px; color: #fff; }
section p.desc { margin: 0 0 16px 0; font-size: 13px; color: var(--muted); }
.two-col {
  display: grid;
  grid-template-columns: 1.35fr 1fr;
  gap: 24px;
}
@media (max-width: 1024px) { .two-col { grid-template-columns: 1fr; } }
canvas {
  width: 100%;
  background: #0d1520;
  border: 1px solid #1e2d42;
  border-radius: 10px;
  display: block;
}
.legend {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
  margin-top: 10px;
  font-size: 12px;
  color: var(--muted);
}
.legend span { display: inline-flex; align-items: center; gap: 6px; }
.dot { width: 10px; height: 10px; border-radius: 50%; display: inline-block; }
table {
  width: 100%;
  border-collapse: collapse;
  font-size: 13px;
}
th, td {
```

```
  text-align: left;
  padding: 10px 12px;
  border-bottom: 1px solid var(--border);
}
th { color: var(--muted); font-weight: 600; font-size: 12px; text-transform: uppercase; }
.pill {
  padding: 2px 8px;
  border-radius: 6px;
  font-weight: 600;
  font-size: 12px;
}
.frames-strip {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 14px;
}
.frame-card {
  background: var(--panel-alt);
  border: 1px solid var(--border);
  border-radius: 10px;
  overflow: hidden;
}
.frame-card img { width: 100%; height: 145px; object-fit: cover; display: block; }
.frame-card .meta { padding: 12px; font-size: 12px; }
.frame-card .meta strong { color: #fff; display: block; margin-bottom: 4px; }
.sim-controls {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(210px, 1fr));
  gap: 14px;
  background: var(--panel-alt);
  padding: 16px;
  border-radius: 10px;
  margin-bottom: 16px;
}
.sim-controls label { display: block; font-size: 12px; color: var(--muted); margin-bottom: 4px; }
.sim-controls input[type=range] { width: 100%; }
");
        sb.AppendLine("</style>");
        sb.AppendLine("</head>");
        sb.AppendLine("<body>");
        sb.AppendLine("<header>");
        sb.AppendLine("  <div><span class=\"badge\">ARGON LAB EXP-001</span><span class=\"badge\">AUTHOR: ...
        sb.AppendLine("  <h1>Narrative Systems Engineering - Multimodal Signal u(t) &amp;amp; Cognitive State ...
        sb.AppendLine($"  <p>Originated &amp;amp; Directed by <strong>Jair Ivan Rojas Ruiz</strong> (<a href=\...
        sb.AppendLine("</header>");
        sb.AppendLine("<main>");

        // KPI Row
        sb.AppendLine("<div class=\"kpi-grid\">");
        sb.AppendLine($"  <div class=\"kpi\"><div class=\"label\">Mean / Peak Narrative Energy</div><div c...
        sb.AppendLine($"  <div class=\"kpi\"><div class=\"label\">Narrative Stance Entropy H(T)</div><div ...
        sb.AppendLine($"  <div class=\"kpi\"><div class=\"label\">Mean Narrative Inertia</div><div class=\...
        sb.AppendLine($"  <div class=\"kpi\"><div class=\"label\">Mean Narrative Complexity</div><div clas...
        sb.AppendLine($"  <div class=\"kpi\"><div class=\"label\">Visual Scene Cuts (JSD &amp;ge; 0.30)</div><...
        sb.AppendLine($"  <div class=\"kpi\"><div class=\"label\">Control Condition &amp;kappa;(W_c)</div><div...
        sb.AppendLine("</div>");

        // Section 1: Multimodal Signal Telemetry
        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>1. Multimodal Signal Telemetry u(t) &amp;amp; Coupled Metrics (t = 0s to 194.8s)<...
        sb.AppendLine("  <p class=\"desc\">Top chart: Narrative Energy E(t), Complexity C(t), and Acoustic...
        sb.AppendLine("  <canvas id=\"signalCanvas\" width=\"1320\" height=\"320\"></canvas>");
        sb.AppendLine("  <div class=\"legend\">");
        sb.AppendLine("    <span><i class=\"dot\" style=\"background:#f59e0b\"></i> Narrative Energy E(t) ...
        sb.AppendLine("    <span><i class=\"dot\" style=\"background:#38bdf8\"></i> Narrative Complexity C...
        sb.AppendLine("    <span><i class=\"dot\" style=\"background:#ec4899\"></i> Visual Cut Divergence ...
        sb.AppendLine("    <span><i class=\"dot\" style=\"background:#a855f7\"></i> Normalized Stance Entr...
        sb.AppendLine("    <span><i class=\"dot\" style=\"background:#10b981\"></i> Narrative Inertia I(t)...
        sb.AppendLine("  </div>");
        sb.AppendLine("</section>");

        // Section 2: Control State Space & Interactive Simulator
        sb.AppendLine("<div class=\"two-col\">");
        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>2. State-Space Control Trajectory: x(t+1) = (I - &amp;Gamma;(t))x(t) + &amp;Gamma;(...
        sb.AppendLine("    <p class=\"desc\">Evolution of the 4 collective belief states across the docume...
        sb.AppendLine("    <div class=\"sim-controls\">");
        sb.AppendLine("      <div><label>Abolitionist Gain (SPK_01 + Poster): <strong id=\"lblBoost1\">1.0...
        sb.AppendLine("      <div><label>Legal/Civic Gain (SPK_02..05): <strong id=\"lblBoost2\">1.0x</str...
        sb.AppendLine("      <div><label>Institutional Inertia Damping: <strong id=\"lblInertia\">0.65</st...
        sb.AppendLine("    </div>");
        sb.AppendLine("    <canvas id=\"stateCanvas\" width=\"760\" height=\"300\"></canvas>");
        sb.AppendLine("    <div class=\"legend\">");
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#ef4444\"></i> x1: Anti-Militarist ...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#3b82f6\"></i> x2: Civic &amp;amp; Lega...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#10b981\"></i> x3: Pragmatic Social...
        sb.AppendLine("      <span><i class=\"dot\" style=\"background:#eab308\"></i> x4: Individual Consc...
        sb.AppendLine("    </div>");
        sb.AppendLine("  </section>");

```

```
        sb.AppendLine("  <section>");
        sb.AppendLine("    <h2>3. Narrative DNA - Signed Speaker-Belief Bipartite Graph</h2>");
        sb.AppendLine("    <p class=\"desc\">Green edges denote reinforcement (+); red dashed edges denote...
        sb.AppendLine("    <canvas id=\"dnaCanvas\" width=\"540\" height=\"395\"></canvas>");
        sb.AppendLine("  </section>");
        sb.AppendLine("</div>");

        // Section 4: Speaker Comparison Table
        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>4. Speaker-Level Multimodal &amp;amp; Cognitive Vector Breakdown (Phases 1-6)</h2...
        sb.AppendLine("  <p class=\"desc\">Direct comparison across the 6 segments/speakers combining acou...
        sb.AppendLine("  <table>");
        sb.AppendLine("    <thead><tr><th>Speaker / Turn</th><th>Span</th><th>Acoustic dBFS</th><th>Speech...
        sb.AppendLine("    <tbody>");
        foreach (var s in profile.SpeakerSummaries)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{System.Net.WebUtility.Htm...
        }
        sb.AppendLine("    </tbody>");
        sb.AppendLine("  </table>");
        sb.AppendLine("</section>");

        // Section 5: Phase 8 & 9 Controllability & Counterfactuals
        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>5. Phase 8 &amp;amp; 9 - Controllability Gramian &amp;amp; Preregistered Counterfactu...
        sb.AppendLine("  <p class=\"desc\">Comparison of final collective cognitive states under Scenario ...
        sb.AppendLine("  <table>");
        sb.AppendLine("    <thead><tr><th>Scenario</th><th>Description</th><th>Final x1 (Anti-Mil)</th><th...
        sb.AppendLine("    <tbody>");
        foreach (var sc in profile.ControlAnalysis.CounterfactualScenarios)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{System.Net.WebUtility.Htm...
        }
        sb.AppendLine("    </tbody>");
        sb.AppendLine("  </table>");
        sb.AppendLine("</section>");

        // Section 6: Visual Evidence Keyframes
        sb.AppendLine("<section>");
        sb.AppendLine("  <h2>6. Time-Aligned Visual &amp;amp; Symbolic Evidence Anchors</h2>");
        sb.AppendLine("  <p class=\"desc\">Representative frames at detected shot transitions and speaker ...
        sb.AppendLine("  <div class=\"frames-strip\" id=\"framesStrip\"></div>");
        sb.AppendLine("</section>");

        sb.AppendLine("</main>");
        sb.AppendLine("<script>");
        sb.Append("const DATA = ");
        sb.Append(payloadJson);
        sb.AppendLine(";");
        sb.AppendLine(@"
const pts = DATA.profile.timeline;

function drawSignalCanvas() {
  const canvas = document.getElementById('signalCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);

  const padL = 48, padR = 20, padT = 20, padB = 34;
  const plotW = W - padL - padR;
  const plotH = H - padT - padB;

  // Grid lines
  ctx.strokeStyle = '#1e293b';
  ctx.lineWidth = 1;
  for (let v = 0; v <= 100; v += 25) {
    const y = padT + plotH * (1 - v / 100);
    ctx.beginPath(); ctx.moveTo(padL, y); ctx.lineTo(W - padR, y); ctx.stroke();
    ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
    ctx.fillText(v.toString(), 14, y + 4);
  }

  // Speaker vertical boundaries
  let prevSpk = pts[0].speakerId;
  pts.forEach((p, i) => {
    const x = padL + (i / (pts.length - 1)) * plotW;
    if (p.speakerId !== prevSpk || i === 0) {
      ctx.strokeStyle = 'rgba(148, 163, 184, 0.28)';
      ctx.setLineDash([4, 4]);
      ctx.beginPath(); ctx.moveTo(x, padT); ctx.lineTo(x, H - padB); ctx.stroke();
      ctx.setLineDash([]);
      ctx.fillStyle = '#94a3b8'; ctx.font = 'bold 11px system-ui';
      ctx.fillText(p.speakerId, x + 4, padT + 12);
      prevSpk = p.speakerId;
    }
    if (i % 4 === 0) {
      ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
      ctx.fillText(p.midpointSeconds.toFixed(0) + 's', x - 10, H - 10);
    }
  });
```

```
325:
326:   function plotSeries(fn, color, width = 2.2) {
327:     ctx.strokeStyle = color;
328:     ctx.lineWidth = width;
329:     ctx.beginPath();
330:     pts.forEach((p, i) => {
331:       const x = padL + (i / (pts.length - 1)) * plotW;
332:       const val = Math.max(0, Math.min(100, fn(p)));
333:       const y = padT + plotH * (1 - val / 100);
334:       if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
335:     });
336:     ctx.stroke();
337:   }
338:
339:   plotSeries(p => p.visual.jensenShannonDivergenceFromPrev * 100, '#ec4899', 1.8);
340:   plotSeries(p => p.normalizedNarrativeEntropy * 100, '#a855f7', 2.0);
341:   plotSeries(p => p.narrativeInertia * 100, '#10b981', 2.0);
342:   plotSeries(p => p.narrativeComplexity, '#38bdf8', 2.0);
343:   plotSeries(p => p.narrativeEnergy, '#f59e0b', 2.8);
344: }
345:
346: function simulateAndDrawState() {
347:   const boost1 = parseFloat(document.getElementById('rngBoost1').value);
348:   const boost2 = parseFloat(document.getElementById('rngBoost2').value);
349:   const inertiaDamp = parseFloat(document.getElementById('rngInertia').value);
350:
351:   document.getElementById('lblBoost1').textContent = boost1.toFixed(1) + 'x';
352:   document.getElementById('lblBoost2').textContent = boost2.toFixed(1) + 'x';
353:   document.getElementById('lblInertia').textContent = inertiaDamp.toFixed(2);
354:
355:   const keys = ['B1_ANTIMILITARISM', 'B2_LEGAL_COMPLIANCE', 'B3_PRAGMATIC_SERVICE', 'B4_INDIVIDUAL_AUTONOM...
356:   const colors = {
357:     B1_ANTIMILITARISM: '#ef4444',
358:     B2_LEGAL_COMPLIANCE: '#3b82f6',
359:     B3_PRAGMATIC_SERVICE: '#10b981',
360:     B4_INDIVIDUAL_AUTONOMY: '#eab308'
361:   };
362:
363:   const state = { B1_ANTIMILITARISM: 0, B2_LEGAL_COMPLIANCE: 0, B3_PRAGMATIC_SERVICE: 0, B4_INDIVIDUAL_AUT...
364:   const history = [];
365:
366:   pts.forEach(p => {
367:     const spkBoost = (p.speakerId === 'SPK_01' || p.speakerId === 'ENV_POSTER') ? boost1 : boost2;
368:     keys.forEach(k => {
369:       const uVal = p.linguistic.activeStanceVector[k] || 0;
370:       const dimIn = k === 'B2_LEGAL_COMPLIANCE' ? Math.min(0.95, p.narrativeInertia * 1.15) : p.narrativeI...
371:       const gamma = Math.min(0.55, 0.18 * spkBoost * (p.narrativeEnergy / 100.0) * (1.0 - inertiaDamp * di...
372:       state[k] = Math.max(-1, Math.min(1, (1 - gamma) * state[k] + gamma * uVal));
373:     });
374:     history.push({ ...state });
375:   });
376:
377:   const canvas = document.getElementById('stateCanvas');
378:   const ctx = canvas.getContext('2d');
379:   const W = canvas.width, H = canvas.height;
380:   ctx.clearRect(0, 0, W, H);
381:
382:   const padL = 44, padR = 16, padT = 18, padB = 28;
383:   const plotW = W - padL - padR, plotH = H - padT - padB;
384:
385:   // Zero axis
386:   const zeroY = padT + plotH * 0.5;
387:   ctx.strokeStyle = '#334155'; ctx.lineWidth = 1.2;
388:   ctx.beginPath(); ctx.moveTo(padL, zeroY); ctx.lineTo(W - padR, zeroY); ctx.stroke();
389:
390:   [-1, -0.5, 0, 0.5, 1].forEach(v => {
391:     const y = padT + plotH * (1 - (v + 1) / 2);
392:     ctx.fillStyle = '#64748b'; ctx.font = '11px system-ui';
393:     ctx.fillText(v.toFixed(1), 10, y + 4);
394:   });
395:
396:   keys.forEach(k => {
397:     ctx.strokeStyle = colors[k];
398:     ctx.lineWidth = 2.5;
399:     ctx.beginPath();
400:     history.forEach((h, i) => {
401:       const x = padL + (i / (history.length - 1)) * plotW;
402:       const y = padT + plotH * (1 - (h[k] + 1) / 2);
403:       if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);
404:     });
405:     ctx.stroke();
406:   });
407: }
408:
409: function drawDnaGraph() {
410:   const canvas = document.getElementById('dnaCanvas');
411:   const ctx = canvas.getContext('2d');
412:   const W = canvas.width, H = canvas.height;
413:   ctx.clearRect(0, 0, W, H);
414:
```

```
  const spks = DATA.profile.speakerSummaries;
  const beliefs = DATA.transcript.beliefDimensions;

  const leftX = 125, rightX = W - 145;
  const spkPos = {};
  spks.forEach((s, i) => {
    spkPos[s.speakerId] = { x: leftX, y: 45 + i * ((H - 80) / Math.max(1, spks.length - 1)), label: s.spea...
  });

  const belPos = {};
  beliefs.forEach((b, i) => {
    belPos[b.id] = { x: rightX, y: 65 + i * ((H - 120) / Math.max(1, beliefs.length - 1)), label: b.id.rep...
  });

  // Edges
  DATA.profile.dnaEdges.forEach(e => {
    if (!spkPos[e.source] || !belPos[e.target]) return;
    const p1 = spkPos[e.source], p2 = belPos[e.target];
    ctx.save();
    ctx.strokeStyle = e.weight >= 0 ? 'rgba(16, 185, 129, 0.75)' : 'rgba(239, 68, 68, 0.75)';
    ctx.lineWidth = Math.max(1.2, Math.abs(e.weight) * 4.2);
    if (e.weight < 0) ctx.setLineDash([5, 4]);
    ctx.beginPath(); ctx.moveTo(p1.x + 14, p1.y); ctx.lineTo(p2.x - 14, p2.y); ctx.stroke();
    ctx.restore();
  });

  // Speaker nodes
  Object.values(spkPos).forEach(p => {
    ctx.fillStyle = '#f59e0b';
    ctx.beginPath(); ctx.arc(p.x, p.y, 11, 0, Math.PI * 2); ctx.fill();
    ctx.fillStyle = '#e2e8f0'; ctx.font = 'bold 11px system-ui';
    ctx.textAlign = 'right';
    ctx.fillText(p.label + ' (E=' + p.energy.toFixed(0) + ')', p.x - 16, p.y + 4);
  });

  // Belief nodes
  Object.values(belPos).forEach(p => {
    ctx.fillStyle = '#38bdf8';
    ctx.beginPath(); ctx.arc(p.x, p.y, 12, 0, Math.PI * 2); ctx.fill();
    ctx.fillStyle = '#e2e8f0'; ctx.font = 'bold 11px system-ui';
    ctx.textAlign = 'left';
    ctx.fillText(p.label, p.x + 18, p.y + 4);
  });
}

function renderKeyframes() {
  const container = document.getElementById('framesStrip');
  Object.entries(DATA.keyFrames).forEach(([idxStr, b64]) => {
    const idx = parseInt(idxStr, 10);
    const pt = pts[idx];
    const card = document.createElement('div');
    card.className = 'frame-card';
    card.innerHTML = `<img src=""${b64}"" alt=""Frame ${idx}""><div class=""meta""><strong>Window #${idx} ...
    container.appendChild(card);
  });
}

document.getElementById('rngBoost1').addEventListener('input', simulateAndDrawState);
document.getElementById('rngBoost2').addEventListener('input', simulateAndDrawState);
document.getElementById('rngInertia').addEventListener('input', simulateAndDrawState);

drawSignalCanvas();
simulateAndDrawState();
drawDnaGraph();
renderKeyframes();
");
        sb.AppendLine("</script>");
        sb.AppendLine("</body></html>");
        return sb.ToString();
    }

    private static string FormatStance(double v)
    {
        var color = v > 0.15 ? "#34d399" : v < -0.15 ? "#f87171" : "#94a3b8";
        return FormattableString.Invariant($"<span style=\"color:{color};font-weight:600\">{v:+0.00;-0.00;...
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/PaperManuscriptRenderer.cs (732 lineas) | SHA-256: 5112e9048be76d3c0c10e42e3113e023...

```
using System.Globalization;
using System.Text;

namespace Argon.Engine;

public static class PaperManuscriptRenderer
{
    public static string RenderPrintReadyHtml(
        CrossDomainAtlasReport atlas,
        CounterfactualControlReport control)
```

```
    {
        var s1 = control.Study1MontageOptimization;
        var s2 = control.Study2ShockImpulseControl;
        var s3 = control.Study3InertialShielding;

        var sb = new StringBuilder();
        sb.AppendLine("<!doctype html>");
        sb.AppendLine("<html lang=\"en\">");
        sb.AppendLine("<head>");
        sb.AppendLine("<meta charset=\"utf-8\">");
        sb.AppendLine("<title>Multimodal Narrative Systems Engineering - Jair Ivan Rojas Ruiz (2026)</titl...
        sb.AppendLine("<style>");
        sb.AppendLine(@"
@page {
  size: letter;
  margin: 16mm 15mm 18mm 15mm;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  padding: 0;
  font-family: 'Georgia', 'Times New Roman', Times, serif;
  font-size: 9.6pt;
  line-height: 1.44;
  color: #111827;
  background: #ffffff;
}
.page-wrap {
  max-width: 190mm;
  margin: 0 auto;
}
.title-block {
  text-align: center;
  border-bottom: 1.5px solid #1e293b;
  padding-bottom: 12px;
  margin-bottom: 14px;
}
.title-block h1 {
  font-size: 16.5pt;
  line-height: 1.22;
  margin: 0 0 8px 0;
  color: #0f172a;
}
.author-name {
  font-size: 11.5pt;
  font-weight: bold;
  margin-bottom: 3px;
}
.author-meta {
  font-size: 8.8pt;
  color: #334155;
  font-family: 'Consolas', 'Courier New', monospace;
}
.abstract-box {
  background: #f8fafc;
  border: 1px solid #cbd5e1;
  border-left: 3.5px solid #0f172a;
  padding: 10px 14px;
  margin-bottom: 14px;
  text-align: justify;
  font-size: 9.1pt;
}
.abstract-box strong.abs-title {
  text-transform: uppercase;
  letter-spacing: 0.05em;
  font-size: 8.5pt;
  color: #0f172a;
}
.two-col {
  column-count: 2;
  column-gap: 7mm;
  text-align: justify;
}
h2 {
  font-size: 11pt;
  text-transform: uppercase;
  letter-spacing: 0.03em;
  color: #0f172a;
  border-bottom: 0.75px solid #94a3b8;
  padding-bottom: 2px;
  margin: 12px 0 6px 0;
  break-after: avoid;
}
h3 {
  font-size: 9.8pt;
  font-weight: bold;
  color: #1e293b;
  margin: 9px 0 4px 0;
  break-after: avoid;
}
```

```
p { margin: 0 0 7px 0; }
.eq {
  background: #f8fafc;
  border: 0.5px solid #e2e8f0;
  border-radius: 3px;
  padding: 5px 8px;
  margin: 6px 0;
  font-family: 'Cambria Math', 'Times New Roman', serif;
  font-size: 9.2pt;
  text-align: center;
  break-inside: avoid;
}
.eq .eq-num {
  float: right;
  color: #64748b;
  font-size: 8.5pt;
}
.full-span {
  column-span: all;
  margin: 12px 0;
  break-inside: avoid;
}
table {
  width: 100%;
  border-collapse: collapse;
  font-size: 8.2pt;
  margin: 4px 0 6px 0;
  font-family: system-ui, -apple-system, 'Segoe UI', sans-serif;
}
th, td {
  padding: 4.5px 6px;
  border-bottom: 0.6px solid #cbd5e1;
  text-align: left;
}
thead th {
  border-top: 1.2px solid #0f172a;
  border-bottom: 1.2px solid #0f172a;
  font-weight: 700;
  color: #0f172a;
  background: #f1f5f9;
}
tbody tr:last-child td {
  border-bottom: 1.2px solid #0f172a;
}
.caption {
  font-size: 8.3pt;
  color: #334155;
  margin-bottom: 4px;
  font-weight: bold;
}
.fig-box {
  border: 0.75px solid #cbd5e1;
  background: #ffffff;
  padding: 8px;
  border-radius: 4px;
  text-align: center;
}
ul, ol { margin: 0 0 7px 0; padding-left: 16px; }
li { margin-bottom: 3px; }
code {
  font-family: 'Consolas', monospace;
  font-size: 8.3pt;
  background: #f1f5f9;
  padding: 1px 3px;
  border-radius: 2px;
}
.ref-list {
  font-size: 8.3pt;
  line-height: 1.35;
}
");
        sb.AppendLine("</style>");
        sb.AppendLine("</head>");
        sb.AppendLine("<body>");
        sb.AppendLine("<div class=\"page-wrap\">");

        sb.AppendLine("<div class=\"title-block\">");
        sb.AppendLine("  <h1>Multimodal Narrative Systems Engineering: Signal-Grounded Metrics and State-S...
        sb.AppendLine("  <div class=\"author-name\">Jair Ivan Rojas Ruiz (<em>Jair Rojas</em>)</div>");
        sb.AppendLine("  <div class=\"author-meta\">Founder &amp;amp; Principal Investigator, Narrative System...
        sb.AppendLine("  <div class=\"author-meta\">Repository: https://github.com/jaiborojas-cloud/narrat...
        sb.AppendLine("</div>");

        sb.AppendLine("<div class=\"abstract-box\">");
        sb.AppendLine("  <strong class=\"abs-title\">Abstract -</strong> Narratives across film, televisio...
        sb.AppendLine("  <strong>Keywords:</strong> Narrative Systems Engineering, Computational Narrative...
        sb.AppendLine("</div>");

        sb.AppendLine("<div class=\"two-col\">");

```

```
191:         sb.AppendLine("<h2>1. Introduction</h2>");
192:         sb.AppendLine("<p>Human societies are structured, stabilized, and transformed by the transmission ...
193:         sb.AppendLine("<p>Existing computational approaches divide into two disconnected paradigms: (1) <e...
194:         sb.AppendLine("<p>To establish <strong>Narrative Systems Engineering (NSE)</strong> as a falsifiab...
195:         sb.AppendLine("<p><strong>Terminology Disambiguation &amp; Chronological Priority:</strong> While ...
196:
197:         sb.AppendLine("<h2>2. Multimodal Signal Vector u(t)</h2>");
198:         sb.AppendLine("<p>Let an audiovisual work of duration <em>T</em> seconds be partitioned into <em>N...
199:         sb.AppendLine("<h3>2.1 Acoustic Prosody Sub-Vector u<sub>ac</sub>(t)</h3>");
200:         sb.AppendLine("<p>From the downmixed mono 16 kHz PCM16 waveform containing <em>M</em> normalized s...
201:         sb.AppendLine("<div class=\"eq\">RMS(t) = [ (1/M) &Sigma; s<sub>i</sub><sup>2</sup> ]<sup>1/2</sup>...
202:         sb.AppendLine("<p>Sub-framing window <em>t</em> into 100 ms intervals yields the Speech Activity R...
203:         sb.AppendLine("<div class=\"eq\">I<sub>ac</sub>(t) = clip(0.55 L&#770;<sub>dB</sub>(t) + 0.30 VAD(...
204:
205:         sb.AppendLine("<h3>2.2 Visual Dynamics Sub-Vector u<sub>vis</sub>(t)</h3>");
206:         sb.AppendLine("<p>For each frame resized to a 64&times;64 RGB24 raster (4096 pixels), we compute I...
207:         sb.AppendLine("<div class=\"eq\">D<sub>JS</sub>(P<sub>t</sub> || P<sub>t&minus;1</sub>) = &frac12;...
208:         sb.AppendLine("<p>where <em>M = &frac12;(P<sub>t</sub> + P<sub>t&minus;1</sub>)</em>. A hard scene...
209:
210:         sb.AppendLine("<h2>3. Six Core NSE Metrics &amp; Control</h2>");
211:         sb.AppendLine("<h3>3.1 Narrative Energy E(t) &amp; Cross-Modal Resonance</h3>");
212:         sb.AppendLine("<p>To prevent zero-collapse while rewarding vocal-lexical synchronization, Narrativ...
213:         sb.AppendLine("<div class=\"eq\">E(t) = 100 &middot; clip(0.34 I<sub>ac</sub>(t) + 0.22 &psi;<sub>...
214:
215:         sb.AppendLine("<h3>3.2 Shannon Narrative Stance Entropy H<sub>narr</sub>(t)</h3>");
216:         sb.AppendLine("<p>Across <em>K = 4</em> belief dimensions (8 signed poles <em>p<sub>k,&plusmn;</su...
217:         sb.AppendLine("<div class=\"eq\">H<sub>narr</sub>(t) = &minus;&Sigma;<sub>k,s</sub> p<sub>k,s</sub>...
218:
219:         sb.AppendLine("<h3>3.3 Narrative Inertia I(t) &amp; LTV State-Space Control</h3>");
220:         sb.AppendLine("<p>Narrative Inertia <em>I(t) = clip(0.55 A<sub>inst</sub>(t) + 0.25 ||x(t)||<sub>1...
221:         sb.AppendLine("<div class=\"eq\">x<sub>k</sub>(t+1) = (1 &minus; &gamma;<sub>k</sub>(t)) x<sub>k</...
222:         sb.AppendLine("<p>The diagonal Controllability Gramian over horizon <em>[0, N]</em> is <em>W<sub>c...
223:
224:         // Full-width Table 1: Cross-Domain Atlas (EXP-001 & EXP-002)
225:         sb.AppendLine("<div class=\"full-span\">");
226:         sb.AppendLine("  <div class=\"caption\">Table 1: Cross-Domain Empirical Atlas (EXP-001 &amp; EXP-0...
227:         sb.AppendLine("  <table>");
228:         sb.AppendLine("    <thead><tr><th>Metric / Dimension</th><th>CORPUS-001: Insumisión Debate (1994, ...
229:         sb.AppendLine("    <tbody>");
230:         sb.AppendLine("      <tr><td><strong>Communicative Regime</strong></td><td>Polyphonic Civic Debate...
231:         sb.AppendLine("      <tr><td><strong>Duration &amp; Windows</strong></td><td>194.84 s (39 &times; ...
232:         sb.AppendLine("      <tr><td><strong>Mean Narrative Energy E&#772;</strong></td><td><strong>63.58<...
233:         sb.AppendLine("      <tr><td><strong>Peak Narrative Energy E<sub>max</sub></strong></td><td>76.54 ...
234:         sb.AppendLine("      <tr><td><strong>Final Stance Entropy H(T)</strong></td><td><strong>2.7574 bit...
235:         sb.AppendLine("      <tr><td><strong>Mean / Peak Inertia I(t)</strong></td><td>0.5019 (peak 0.728)...
236:         sb.AppendLine("      <tr><td><strong>Scene Cuts (D<sub>JS</sub> &ge; 0.30)</strong></td><td>4 cuts...
237:         sb.AppendLine("      <tr><td><strong>Final Polarization P(T)</strong></td><td><strong>0.5515</stro...
238:         sb.AppendLine("      <tr><td><strong>Control Condition &kappa;(W<sub>c</sub>)</strong></td><td>2.7...
239:         sb.AppendLine("    </tbody>");
240:         sb.AppendLine("  </table>");
241:         sb.AppendLine("</div>");
242:
243:         sb.AppendLine("<h2>4. Empirical Results: EXP-001 &amp; EXP-002</h2>");
244:         sb.AppendLine("<p>In <code>EXP-001</code> (<code>CORPUS-001</code>, Spanish university conscriptio...
245:         sb.AppendLine("<p>In <code>EXP-002</code> (Table 1 &amp; Figure 1), projecting all three works int...
246:
247:         // Full-width Figure 1 (Vector SVG of Cross-Domain Phase Space & Montage Optimization)
248:         sb.AppendLine("<div class=\"full-span\">");
249:         sb.AppendLine("  <div class=\"fig-box\">");
250:         sb.AppendLine(BuildVectorFigureSvg(atlas, control));
251:         sb.AppendLine("  </div>");
252:         sb.AppendLine("  <div class=\"caption\" style=\"margin-top:5px\">Figure 1: (Left) Normalized Cross...
253:         sb.AppendLine("</div>");
254:
255:         // Full-width Table 2: EXP-003 Counterfactual Control
256:         sb.AppendLine("<div class=\"full-span\">");
257:         sb.AppendLine("  <div class=\"caption\">Table 2: EXP-003 Optimal Montage Synthesis Solutions Acros...
258:         sb.AppendLine("  <table>");
259:         sb.AppendLine("    <thead><tr><th>Synthesis Objective</th><th>Optimal Editorial Block Sequence &pi...
260:         sb.AppendLine("    <tbody>");
261:         sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>Original 1994 Broadcast</stron...
262:         foreach (var opt in new[] { s1.OptimalAntiMilitarist, s1.OptimalLegalCompliance, s1.OptimalCivicEq...
263:         {
264:             var seq = string.Join(" &rarr; ", opt.BlockOrder);
265:             sb.AppendLine(CultureInfo.InvariantCulture, $"      <tr><td><strong>{opt.ObjectiveTag}</strong...
266:         }
267:         sb.AppendLine("    </tbody>");
268:         sb.AppendLine("  </table>");
269:         sb.AppendLine("</div>");
270:
271:         sb.AppendLine("<h2>5. Counterfactual Control (EXP-003) &amp; System Identification (EXP-004)</h2>"...
272:         sb.AppendLine("<p>To demonstrate constructive systems synthesis and inverse parameter identifiabil...
273:         sb.AppendLine("<ol>");
274:         sb.AppendLine("  <li><strong>Exhaustive 6! = 720 Montage Reachability (CORPUS-001):</strong> Holdi...
275:         sb.AppendLine("  <li><strong>Non-Verbal Shock Impulse Leverage (CORPUS-002):</strong> Attenuating ...
276:         sb.AppendLine("  <li><strong>Institutional Inertial Shielding (CORPUS-003):</strong> Removing pres...
277:         sb.AppendLine("  <li><strong>Inverse System Identification &amp; LOCO Generalization (EXP-004):</s...
278:         sb.AppendLine("</ol>");
279:
280:         sb.AppendLine("<h2>6. Conclusion</h2>");
```

```
        sb.AppendLine("<p>By unifying physical signal operators (PCM prosody, Jensen-Shannon visual diverg...

        sb.AppendLine("<h2>References</h2>");
        sb.AppendLine("<ol class=\"ref-list\">");
        sb.AppendLine("  <li>Rojas Ruiz, J. I. (2026). <em>Narrative Systems Engineering (NSE) and Computa...
        sb.AppendLine("  <li>Shannon, C. E. (1948). A Mathematical Theory of Communication. <em>Bell Syste...
        sb.AppendLine("  <li>Lin, J. (1991). Divergence measures based on the Shannon entropy. <em>IEEE Tr...
        sb.AppendLine("  <li>Friedkin, N. E., &amp;amp; Johnsen, E. C. (1990). Social influence and opinions. ...
        sb.AppendLine("  <li>Vicol, P., Tapaswi, M., Castrejon, L., &amp;amp; Fidler, S. (2018). MovieGraphs: ...
        sb.AppendLine("  <li>Green, M. C., &amp;amp; Brock, T. C. (2000). The role of transportation in the pe...
        sb.AppendLine("  <li>Jones, M. D., &amp;amp; McBeth, M. K. (2010). A Narrative Policy Framework: Clear...
        sb.AppendLine("  <li>Shiller, R. J. (2017). Narrative Economics. <em>American Economic Review</em>...
        sb.AppendLine("</ol>");

        sb.AppendLine("</div>"); // two-col
        sb.AppendLine("</div>"); // page-wrap
        sb.AppendLine("</body></html>");
        return sb.ToString();
    }

    private static string BuildVectorFigureSvg(CrossDomainAtlasReport atlas, CounterfactualControlReport c...
    {
        var sb = new StringBuilder();
        sb.AppendLine("<svg viewBox=\"0 0 760 220\" width=\"100%\" height=\"210\" xmlns=\"http://www.w3.or...

        // Left Panel: Normalized Energy Trajectories E(t/T) [x: 35..355, y: 20..185]
        sb.AppendLine("  <rect x=\"35\" y=\"20\" width=\"320\" height=\"165\" fill=\"#f8fafc\" stroke=\"#c...
        sb.AppendLine("  <text x=\"195\" y=\"13\" text-anchor=\"middle\" font-weight=\"bold\" font-size=\"...
        for (int v = 0; v <= 100; v += 25)
        {
            double py = 20 + 165 * (1.0 - v / 100.0);
            sb.AppendLine(CultureInfo.InvariantCulture, $"  <line x1=\"35\" y1=\"{py:F1}\" x2=\"355\" y2=\...
            sb.AppendLine(CultureInfo.InvariantCulture, $"  <text x=\"30\" y=\"{py + 3:F1}\" text-anchor=\...
        }
        sb.AppendLine("  <text x=\"195\" y=\"202\" text-anchor=\"middle\" fill=\"#334155\" font-size=\"8.5...

        string[] colors = ["#d97706", "#dc2626", "#0284c7"];
        for (int wi = 0; wi < atlas.Works.Count; wi++)
        {
            var w = atlas.Works[wi];
            var pts = new StringBuilder();
            foreach (var pt in w.Orbit)
            {
                double px = 35 + pt.NormalizedTime * 320.0;
                double py = 20 + 165 * (1.0 - pt.Energy / 100.0);
                pts.Append(CultureInfo.InvariantCulture, $"{px:F1},{py:F1} ");
            }
            sb.AppendLine(CultureInfo.InvariantCulture, $"  <polyline fill=\"none\" stroke=\"{colors[wi]}\...
        }
        sb.AppendLine("  <text x=\"42\" y=\"34\" fill=\"#d97706\" font-weight=\"bold\" font-size=\"8\">- C...
        sb.AppendLine("  <text x=\"145\" y=\"34\" fill=\"#dc2626\" font-weight=\"bold\" font-size=\"8\">- ...
        sb.AppendLine("  <text x=\"42\" y=\"46\" fill=\"#0284c7\" font-weight=\"bold\" font-size=\"8\">- C...

        // Right Panel: 720 Permutations Cloud [x: 415..735, y: 20..185]
        sb.AppendLine("  <rect x=\"415\" y=\"20\" width=\"320\" height=\"165\" fill=\"#f8fafc\" stroke=\"#...
        sb.AppendLine("  <text x=\"575\" y=\"13\" text-anchor=\"middle\" font-weight=\"bold\" font-size=\"...

        double MapCloudX(double x1) => 415 + ((x1 + 0.4) / 1.05) * 320.0;
        double MapCloudY(double x2) => 20 + 165 * (1.0 - (x2 + 0.3) / 1.05);

        sb.AppendLine(CultureInfo.InvariantCulture, $"  <line x1=\"{MapCloudX(0):F1}\" y1=\"20\" x2=\"{Map...
        sb.AppendLine(CultureInfo.InvariantCulture, $"  <line x1=\"415\" y1=\"{MapCloudY(0):F1}\" x2=\"735...

        var s1 = control.Study1MontageOptimization;
        foreach (var c in s1.CloudSamples)
        {
            sb.AppendLine(CultureInfo.InvariantCulture, $"  <circle cx=\"{MapCloudX(c.X1AntiMil):F1}\" cy=...
        }

        // Highlights
        void DrawHighlight(double x1, double x2, string col, string label, double dx, double dy)
        {
            double cx = MapCloudX(x1), cy = MapCloudY(x2);
            sb.AppendLine(CultureInfo.InvariantCulture, $"  <circle cx=\"{cx:F1}\" cy=\"{cy:F1}\" r=\"4.5\...
            sb.AppendLine(CultureInfo.InvariantCulture, $"  <text x=\"{cx + dx:F1}\" y=\"{cy + dy:F1}\" fi...
        }

        DrawHighlight(s1.OriginalFinalStateX["B1_ANTIMILITARISM"], s1.OriginalFinalStateX["B2_LEGAL_COMPLI...
        DrawHighlight(s1.OptimalAntiMilitarist.FinalStateX["B1_ANTIMILITARISM"], s1.OptimalAntiMilitarist....
        DrawHighlight(s1.OptimalCivicEquilibrium.FinalStateX["B1_ANTIMILITARISM"], s1.OptimalCivicEquilibr...
        DrawHighlight(s1.OptimalLegalCompliance.FinalStateX["B1_ANTIMILITARISM"], s1.OptimalLegalComplianc...

        sb.AppendLine("  <text x=\"575\" y=\"202\" text-anchor=\"middle\" fill=\"#334155\" font-size=\"8.5...
        sb.AppendLine("</svg>");
        return sb.ToString();
    }

    public static byte[] RenderNativePdfBytes(
        CrossDomainAtlasReport atlas,
        CounterfactualControlReport control)
```

```
    {
        var s1 = control.Study1MontageOptimization;
        var s2 = control.Study2ShockImpulseControl;
        var s3 = control.Study3InertialShielding;

        var pages = new List<StringBuilder>();

        // ---------------- PAGE 1: TITLE, ABSTRACT, SECTIONS 1-3 ----------------
        var p1 = new StringBuilder();
        double y = 742;

        // Header banner line
        DrawLine(p1, 48, y + 14, 564, y + 14, 1.2, 0.08, 0.15, 0.28);
        DrawText(p1, "F2", 8.0, 48, y + 3, "NARRATIVE SYSTEMS ENGINEERING (NSE) - PREPRINT PAPER-001 v1.1 ...
        DrawText(p1, "F2", 8.0, 462, y + 3, "ORCID: 0009-0003-1109-4583");
        y -= 20;

        // Title
        DrawText(p1, "F2", 14.5, 48, y, "Multimodal Narrative Systems Engineering: Signal-Grounded Metrics...
        y -= 17;
        DrawText(p1, "F2", 14.5, 48, y, "and State-Space Control Dynamics for Audiovisual Media");
        y -= 18;

        // Author & Affiliation
        DrawText(p1, "F2", 10.5, 48, y, "Jair Ivan Rojas Ruiz (Jair Rojas) - Founder & Principal Investiga...
        y -= 13;
        DrawText(p1, "F4", 7.6, 48, y, "ORCID: https://orcid.org/0009-0003-1109-4583 | DOI: https://doi.or...
        y -= 10;
        DrawLine(p1, 48, y, 564, y, 0.8, 0.2, 0.25, 0.35);
        y -= 16;

        // Abstract Box
        DrawRect(p1, 48, y - 96, 516, 104, 0.97, 0.98, 1.0, 0.75, 0.80, 0.88);
        DrawText(p1, "F2", 9.0, 56, y - 6, "ABSTRACT");
        string[] absLines =
        [
            "Narratives across film, television, political discourse, and digital media are typically anal...
            "frameworks or isolated text-only NLP. Conversely, prompting LLMs to score 'narrative impact' ...
            "uncalibrated units, stochastic hallucination, and detachment from the physical audiovisual si...
            "Engineering (NSE) and Computational Narrative Dynamics (CND), originated by Jair Ivan Rojas R...
            "multimodal inputs u(t) = (u_ac(t), u_vis(t), u_ling(t)) driving a bounded cognitive-social st...
            "six core quantities: (1) Narrative DNA (signed tripartite graph), (2) Shannon Narrative Entro...
            "via cross-modal resonance, (4) Complexity C(t), (5) Narrative Inertia I(t), and (6) Controlla...
            "(513.6 s, 77 windows: EXP-001, EXP-002, EXP-003), NSE cleanly separates civic debate, fear-ap...
            "crisis consolation (Reagan 'Challenger' 1986), while solving all 6! = 720 montage permutation...
        ];
        double ay = y - 18;
        foreach (var line in absLines)
        {
            DrawText(p1, "F1", 8.6, 56, ay, line);
            ay -= 9.6;
        }
        y -= 114;

        // Section 1
        DrawSectionHeader(p1, ref y, "1. INTRODUCTION, AXIOMS & CHRONOLOGICAL PRIORITY DISAMBIGUATION");
        WriteParagraph(p1, ref y,
        [
            "Bridging the gap between raw media signals (PCM16 acoustic waveforms, RGB24 pixel rasters, an...
            "and social state dynamics requires falsifiable mathematical operators. Naive multiplicative f...
            "Repetition x Identification fail dimensional analysis and collapse to zero whenever any singl...
            "To establish Narrative Systems Engineering (NSE) as a rigorous engineering discipline, Argon....
            "  (1) Signal Grounding: Direct derivation from physical PCM16 audio samples, 64x64 RGB24 rast...
            "  (2) Stated Units & Bounds: Entropy and visual divergence in Shannon bits, audio in dBFS, an...
            "  (3) Layer Separation: Strict separation between observable media signal u(t), normative sta...
            "  (4) Executable Reproducibility: Deterministic verification via automated invariant test sui...
            "Terminology Disambiguation & Chronological Priority: While homonymous, purely qualitative or ...
            "Systems Engineering' exist on the web (including informal game-design notes or qualitative sc...
            "Nov 2, 2026 NSE-001 post), those texts lack mathematical formulations, physical audio/video s...
            "entropy, state-space control models, and executable software. In contrast, the foundational w...
            "September 8, 2026 and registered in Zenodo under DOI: 10.5281/zenodo.23987656, ORCID: 0009-00...
            "quantitative, signal-grounded discipline of Multimodal Narrative Systems Engineering & Comput...
        ]);

        // Section 2
        DrawSectionHeader(p1, ref y, "2. MULTIMODAL SIGNAL FORMULATION u(t) = (u_ac(t), u_vis(t), u_ling(t...
        WriteParagraph(p1, ref y,
        [
            "Let an audiovisual work of duration T seconds be partitioned into N discrete windows t in {0,...
            "  * Acoustic Sub-Vector u_ac(t): From mono 16 kHz PCM16 samples s_i in [-1, 1], we compute RM...
            "    20 log10(max(RMS(t), 1e-5)), 100ms Speech Activity Ratio VAD(t) in [0, 1], and intra-wind...
            "  * Visual Sub-Vector u_vis(t): From 64x64 RGB24 rasters, we extract ITU-R BT.709 luminance Y...
            "    entropy H_vis(t), and the 512-bin color distribution P_t(b). Camera cuts are detected via...
        ]);
        DrawEquationBox(p1, ref y, "D_JS(P_t || P_{t-1}) = 0.5 D_KL(P_t || M) + 0.5 D_KL(P_{t-1} || M) in ...

        // Section 3
        DrawSectionHeader(p1, ref y, "3. FORMAL DEFINITION OF THE SIX CORE NSE METRICS & STATE CONTROL");
        DrawEquationBox(p1, ref y, "Narrative Energy: E(t) = 100 * clip(0.34 I_ac + 0.22 psi_vis + 0.29 I_...
        DrawEquationBox(p1, ref y, "Shannon Stance Entropy: H_narr(t) = - Sum_{k,s} p_{k,s}(t) log2 p_{k,s...
```

```
        DrawEquationBox(p1, ref y, "Narrative Inertia: I(t) = clip(0.55 A_inst(t) + 0.25 (||x(t)||_1 / K) ...
        DrawEquationBox(p1, ref y, "LTV State Update: x_k(t+1) = (1 - gamma_k(t)) x_k(t) + gamma_k(t) z_k(...
        DrawEquationBox(p1, ref y, "Controllability Gramian: W_{c,kk} = Sum_{t=0..N-1} [Phi_k(N,t+1) b_k(t...

        DrawFooter(p1, 1, 2);
        pages.Add(p1);

        // ---------------- PAGE 2: EMPIRICAL TABLES, VECTOR PLOTS, EXP-001/002/003 & REFERENCES ---------...
        var p2 = new StringBuilder();
        double y2 = 742;
        DrawLine(p2, 48, y2 + 14, 564, y2 + 14, 1.0, 0.08, 0.15, 0.28);
        DrawText(p2, "F2", 8.5, 48, y2 + 3, "JAIR IVAN ROJAS RUIZ - MULTIMODAL NARRATIVE SYSTEMS ENGINEERI...
        y2 -= 16;

        DrawSectionHeader(p2, ref y2, "4. CROSS-DOMAIN EMPIRICAL ATLAS (EXP-001 & EXP-002: 3 WORKS, 513.6s...
        string[] t1Headers = ["Metric / Dimension", "CORPUS-001: Debate (ES)", "CORPUS-002: LBJ Daisy (EN)...
        double[] t1Cols = [48, 200, 322, 444];
        string[][] t1Rows =
        [
            ["Communicative Regime", "Polyphonic Civic Debate", "Acute Fear-Appeal Spot", "Institutional C...
            ["Duration & Windows", "194.84 s (39 x 5s)", "59.83 s (12 x 5s)", "258.93 s (26 x 10s)"],
            ["Mean / Peak Energy E", "63.58  (Peak: 76.54)", "50.51  (Peak: 83.29)", "56.45  (Peak: 61.98)...
            ["Final Stance Entropy H(T)", "2.7574 bits (91.9%)", "2.2292 bits (74.3%)", "2.0240 bits (67.5...
            ["Mean / Peak Inertia I(t)", "0.5019 (Peak: 0.728)", "0.4420 (Peak: 0.799)", "0.7266 (Peak: 0....
            ["Scene Cuts (D_JS >= 0.30)", "4 cuts (max 0.490 bits)", "3 cuts (max 0.577 bits)", "1 cut (ma...
            ["Final Polarization P(T)", "0.5515 (High Contest)", "0.4200 (Contrastive)", "0.0617 (Unified ...
            ["Control Condition kappa(W_c)", "2.738", "1.783", "1.457"]
        ];
        DrawTable(p2, ref y2, "Table 1: Cross-Domain Multimodal Benchmark Telemetry Across Three Narrative...

        // Vector Plots Panel on Page 2
        DrawVectorPlotsPdf(p2, ref y2, atlas, control);

        DrawSectionHeader(p2, ref y2, "5. COUNTERFACTUAL CONTROL (EXP-003) & INVERSE SYSTEM IDENTIFICATION...
        string[] t2Headers = ["Control Synthesis Objective", "Optimal Block Sequence pi*", "x1 AntiMil", "...
        double[] t2Cols = [48, 188, 368, 416, 464, 512];
        string[][] t2Rows =
        [
            ["Original 1994 Broadcast", "P -> S1 -> S2 -> S3 -> S4 -> S5", "-0.1445", "+0.5524", "+0.6446"...
            ["pi*_anti (Max Abolitionism)", "S3 -> S2 -> S5 -> S4 -> P -> S1", "+0.5130", "-0.1717", "-0.0...
            ["pi*_legal (Max Legal Comp.)", "P -> S1 -> S4 -> S5 -> S2 -> S3", "-0.2601", "+0.5830", "+0.6...
            ["pi*_eq (Min-Bias Equilibrium)", "S4 -> P -> S5 -> S2 -> S1 -> S3", "+0.2262", "+0.0606", "+0...
            ["pi*_aut (Max Autonomy)", "S3 -> S2 -> S5 -> S1 -> P -> S4", "+0.3958", "-0.0083", "+0.0669",...
        ];
        DrawTable(p2, ref y2, "Table 2: Exhaustive 6! = 720 Montage Optimization Solutions on CORPUS-001 (...

        WriteParagraph(p2, ref y2,
        [
            $"Key EXP-003 & EXP-004 Engineering Findings: (1) Across all 720 permutations of CORPUS-001, e...
            $"([-0.2603, +0.5130]), while the Civic Equilibrium Montage pi*_eq reduces terminal normative ...
            $"damping the non-verbal nuclear blast reduces mobilization x2(T) by {s2.NonVerbalShockLeverag...
            "inertia stabilization. (4) In EXP-004 (SystemIdentifier.cs), Tikhonov Ridge identification ac...
            "0.00% noiseless error, Trajectory R^2 = 0.9999 at 25 dB SNR, and Leave-One-Corpus-Out (LOCO) ...
        ]);

        DrawSectionHeader(p2, ref y2, "6. REFERENCES");
        WriteParagraph(p2, ref y2,
        [
            "[1] Rojas Ruiz, J. I. (2026). Narrative Systems Engineering (NSE) and Computational Narrative...
            "[2] Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Jour...
            "[3] Lin, J. (1991). Divergence Measures Based on the Shannon Entropy. IEEE Trans. Information...
            "[4] Friedkin, N. E., & Johnsen, E. C. (1990). Social Influence and Opinions. J. Mathematical ...
        ]);

        DrawFooter(p2, 2, 2);
        pages.Add(p2);

        return AssemblePdfDocument(pages);
    }

    private static void DrawSectionHeader(StringBuilder sb, ref double y, string title)
    {
        y -= 4;
        DrawText(sb, "F2", 9.6, 48, y, title);
        y -= 4;
        DrawLine(sb, 48, y, 564, y, 0.6, 0.3, 0.4, 0.55);
        y -= 11;
    }

    private static void WriteParagraph(StringBuilder sb, ref double y, string[] lines)
    {
        foreach (var line in lines)
        {
            DrawText(sb, "F1", 8.6, 48, y, line);
            y -= 10.2;
        }
        y -= 3;
    }

    private static void DrawEquationBox(StringBuilder sb, ref double y, string expr, string num)
```

```
    {
        DrawRect(sb, 48, y - 4, 516, 15, 0.96, 0.97, 0.99, 0.82, 0.86, 0.92);
        DrawText(sb, "F4", 7.4, 54, y, expr);
        DrawText(sb, "F2", 8.0, 544, y, num);
        y -= 18;
    }

    private static void DrawTable(StringBuilder sb, ref double y, string caption, string[] headers, double...
    {
        DrawText(sb, "F2", 8.3, 48, y, caption);
        y -= 6;
        DrawRect(sb, 48, y - 11, 516, 13, 0.92, 0.95, 0.98, 0.2, 0.3, 0.45);
        for (int c = 0; c < headers.Length; c++)
        {
            DrawText(sb, "F2", 8.0, cols[c] + 3, y - 8, headers[c]);
        }
        y -= 11;
        foreach (var row in rows)
        {
            y -= 11.2;
            DrawLine(sb, 48, y - 2, 564, y - 2, 0.3, 0.80, 0.84, 0.90);
            for (int c = 0; c < row.Length; c++)
            {
                DrawText(sb, c == 0 ? "F2" : "F1", 7.9, cols[c] + 3, y + 1, row[c]);
            }
        }
        DrawLine(sb, 48, y - 2, 564, y - 2, 0.9, 0.1, 0.15, 0.25);
        y -= 14;
    }

    private static void DrawVectorPlotsPdf(StringBuilder sb, ref double y, CrossDomainAtlasReport atlas, C...
    {
        double boxH = 128;
        double topY = y;
        double botY = y - boxH;

        // Left Plot Box: Normalized Energy E(t/T) [x: 48..292]
        DrawRect(sb, 48, botY, 244, boxH, 0.98, 0.99, 1.0, 0.72, 0.78, 0.86);
        DrawText(sb, "F2", 8.0, 56, topY - 11, "Fig 1A. Cross-Domain Energy Trajectories E(t/T)");

        double p1X = 68, p1W = 214, p1Y = botY + 18, p1H = boxH - 36;
        DrawLine(sb, p1X, p1Y, p1X + p1W, p1Y, 0.5, 0.5, 0.55, 0.65);
        DrawLine(sb, p1X, p1Y, p1X, p1Y + p1H, 0.5, 0.5, 0.55, 0.65);

        (double R, double G, double B)[] cols = [(0.85, 0.47, 0.02), (0.86, 0.15, 0.15), (0.01, 0.52, 0.78...
        for (int wi = 0; wi < atlas.Works.Count; wi++)
        {
            var orbit = atlas.Works[wi].Orbit;
            var (r, g, b) = cols[wi % cols.Length];
            sb.AppendLine(CultureInfo.InvariantCulture, $"{r:F2} {g:F2} {b:F2} RG 1.5 w");
            for (int i = 0; i < orbit.Count; i++)
            {
                double px = p1X + orbit[i].NormalizedTime * p1W;
                double py = p1Y + (orbit[i].Energy / 100.0) * p1H;
                sb.AppendLine(CultureInfo.InvariantCulture, $"{px:F1} {py:F1} {(i == 0 ? "m" : "l")}");
            }
            sb.AppendLine("S");
        }
        DrawText(sb, "F1", 7.2, 72, botY + 5, "Orange: CORPUS-001 | Red: CORPUS-002 (Peak 83.3) | Blue: CO...

        // Right Plot Box: 720 Montage Cloud [x: 312..564]
        DrawRect(sb, 312, botY, 252, boxH, 0.98, 0.99, 1.0, 0.72, 0.78, 0.86);
        DrawText(sb, "F2", 8.0, 320, topY - 11, "Fig 1B. 6! = 720 Montage Reachability Hull (EXP-003)");

        double p2X = 330, p2W = 222, p2Y = botY + 18, p2H = boxH - 36;
        double MapX(double v) => p2X + ((v + 0.35) / 0.95) * p2W;
        double MapY(double v) => p2Y + ((v + 0.25) / 0.95) * p2H;

        DrawLine(sb, MapX(0), p2Y, MapX(0), p2Y + p2H, 0.4, 0.65, 0.70, 0.78);
        DrawLine(sb, p2X, MapY(0), p2X + p2W, MapY(0), 0.4, 0.65, 0.70, 0.78);

        var s1 = control.Study1MontageOptimization;
        foreach (var pt in s1.CloudSamples)
        {
            DrawCircle(sb, MapX(pt.X1AntiMil), MapY(pt.X2Legal), 1.4, 0.55, 0.62, 0.72);
        }
        DrawCircle(sb, MapX(s1.OriginalFinalStateX["B1_ANTIMILITARISM"]), MapY(s1.OriginalFinalStateX["B2_...
        DrawCircle(sb, MapX(s1.OptimalAntiMilitarist.FinalStateX["B1_ANTIMILITARISM"]), MapY(s1.OptimalAnt...
        DrawCircle(sb, MapX(s1.OptimalCivicEquilibrium.FinalStateX["B1_ANTIMILITARISM"]), MapY(s1.OptimalC...

        DrawText(sb, "F1", 7.2, 326, botY + 5, "Gray: 720 Montages | Green: Optimal Equilibrium (0.327) | ...

        y = botY - 14;
    }

    private static void DrawFooter(StringBuilder sb, int pageNum, int totalPages)
    {
        DrawLine(sb, 48, 38, 564, 38, 0.5, 0.6, 0.65, 0.75);
        DrawText(sb, "F1", 8.0, 48, 26, "Rojas Ruiz, J. I. (2026) - Narrative Systems Engineering (NSE) - ...
        DrawText(sb, "F2", 8.0, 515, 26, $"Page {pageNum} of {totalPages}");
```

```
    }

    private static void DrawText(StringBuilder sb, string fontRef, double size, double x, double y, string...
    {
        var escaped = text
            .Replace("\\", "\\\\")
            .Replace("(", "\\(")
            .Replace(")", "\\)");
        sb.AppendLine(CultureInfo.InvariantCulture, $"0.08 0.11 0.18 rg BT /{fontRef} {size:F1} Tf {x:F1} ...
    }

    private static void DrawLine(StringBuilder sb, double x1, double y1, double x2, double y2, double widt...
    {
        sb.AppendLine(CultureInfo.InvariantCulture, $"{r:F2} {g:F2} {b:F2} RG {width:F2} w {x1:F1} {y1:F1}...
    }

    private static void DrawRect(StringBuilder sb, double x, double y, double w, double h, double fr, doub...
    {
        sb.AppendLine(CultureInfo.InvariantCulture, $"{fr:F2} {fg:F2} {fb:F2} rg {sr:F2} {sg:F2} {sbCol:F2...
    }

    private static void DrawCircle(StringBuilder sb, double cx, double cy, double r, double red, double gr...
    {
        double k = 0.55228475 * r;
        sb.AppendLine(CultureInfo.InvariantCulture, $"{red:F2} {green:F2} {blue:F2} rg");
        sb.AppendLine(CultureInfo.InvariantCulture, $"{cx + r:F2} {cy:F2} m");
        sb.AppendLine(CultureInfo.InvariantCulture, $"{cx + r:F2} {cy + k:F2} {cx + k:F2} {cy + r:F2} {cx:...
        sb.AppendLine(CultureInfo.InvariantCulture, $"{cx - k:F2} {cy + r:F2} {cx - r:F2} {cy + k:F2} {cx ...
        sb.AppendLine(CultureInfo.InvariantCulture, $"{cx - r:F2} {cy - k:F2} {cx - k:F2} {cy - r:F2} {cx:...
        sb.AppendLine(CultureInfo.InvariantCulture, $"{cx + k:F2} {cy - r:F2} {cx + r:F2} {cy - k:F2} {cx ...
    }

    private static byte[] AssemblePdfDocument(List<StringBuilder> pageStreams)
    {
        using var ms = new MemoryStream();
        var offsets = new List<long>();

        void WriteAscii(string s)
        {
            var bytes = Encoding.GetEncoding("ISO-8859-1").GetBytes(s);
            ms.Write(bytes, 0, bytes.Length);
        }

        WriteAscii("%PDF-1.4\n");

        // Object 1: Catalog
        offsets.Add(ms.Position);
        WriteAscii("1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n");

        // Object 2: Pages
        int firstPageObjId = 7;
        var kids = string.Join(" ", Enumerable.Range(0, pageStreams.Count).Select(i => $"{firstPageObjId +...
        offsets.Add(ms.Position);
        WriteAscii($"2 0 obj\n<< /Type /Pages /Kids [{kids}] /Count {pageStreams.Count} >>\nendobj\n");

        // Objects 3..6: Standard Type1 Fonts (Times-Roman, Times-Bold, Times-Italic, Courier)
        string[] fontNames = ["Times-Roman", "Times-Bold", "Times-Italic", "Courier"];
        for (int f = 0; f < fontNames.Length; f++)
        {
            offsets.Add(ms.Position);
            WriteAscii($"{3 + f} 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont /{fontNames[f]} /Encoding...
        }

        // Page and Content objects
        for (int i = 0; i < pageStreams.Count; i++)
        {
            int pageObjId = firstPageObjId + i * 2;
            int contentObjId = pageObjId + 1;
            var contentStr = pageStreams[i].ToString();
            var contentBytes = Encoding.GetEncoding("ISO-8859-1").GetBytes(contentStr);

            offsets.Add(ms.Position);
            WriteAscii($"{pageObjId} 0 obj\n<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Resource...

            offsets.Add(ms.Position);
            WriteAscii($"{contentObjId} 0 obj\n<< /Length {contentBytes.Length} >>\nstream\n");
            ms.Write(contentBytes, 0, contentBytes.Length);
            WriteAscii("\nendstream\nendobj\n");
        }

        long xrefPos = ms.Position;
        WriteAscii($"xref\n0 {offsets.Count + 1}\n");
        WriteAscii("0000000000 65535 f \n");
        foreach (var off in offsets)
        {
            WriteAscii(off.ToString("D10", CultureInfo.InvariantCulture) + " 00000 n \n");
        }
        WriteAscii($"trailer\n<< /Size {offsets.Count + 1} /Root 1 0 R >>\nstartxref\n{xrefPos}\n%%EOF\n")...

        return ms.ToArray();
```

```
    }
}
```

ARCHIVO FUENTE: argon/src/Argon.Engine/IndautorPackageGenerator.cs (402 lineas) | SHA-256: f38d5a01f6c30a3e1be376dce3ef7567...

```
using System.Globalization;
using System.Security.Cryptography;
using System.Text;

namespace Argon.Engine;

public static class IndautorPackageGenerator
{
    public static void GenerateIndautorPdfs(string repoRoot)
    {
        var outDir = Path.Combine(repoRoot, "legal", "indautor");
        Directory.CreateDirectory(outDir);

        var sourceFiles = new[]
        {
            "argon/src/Argon.Engine/Program.cs",
            "argon/src/Argon.Engine/AcousticAnalyzer.cs",
            "argon/src/Argon.Engine/VisualDynamicsAnalyzer.cs",
            "argon/src/Argon.Engine/LinguisticAnalyzer.cs",
            "argon/src/Argon.Engine/NarrativeMetricsEngine.cs",
            "argon/src/Argon.Engine/AtlasComparator.cs",
            "argon/src/Argon.Engine/CounterfactualController.cs",
            "argon/src/Argon.Engine/SystemIdentifier.cs",
            "argon/src/Argon.Engine/DashboardRenderer.cs",
            "argon/src/Argon.Engine/PaperManuscriptRenderer.cs",
            "argon/src/Argon.Engine/IndautorPackageGenerator.cs",
            "src/Nse.Pipeline/Program.cs",
            "src/Nse.Pipeline/VideoProfiler.cs",
            "src/Nse.Pipeline/ReviewWorkflow.cs"
        };

        var fileEntries = new List<(string RelPath, string Sha256, string[] Lines)>();
        foreach (var rel in sourceFiles)
        {
            var full = Path.Combine(repoRoot, rel.Replace('/', Path.DirectorySeparatorChar));
            if (File.Exists(full))
            {
                var bytes = File.ReadAllBytes(full);
                var sha = Convert.ToHexString(SHA256.HashData(bytes)).ToLowerInvariant();
                var lines = File.ReadAllLines(full);
                fileEntries.Add((rel, sha, lines));
            }
        }

        var memoriaPath = Path.Combine(outDir, "MEMORIA_DESCRIPTIVA_INDAUTOR_ES.pdf");
        File.WriteAllBytes(memoriaPath, BuildMemoriaDescriptivaPdf(fileEntries));

        var codePath = Path.Combine(outDir, "CODIGO_FUENTE_ARGON_NSE_INDAUTOR.pdf");
        File.WriteAllBytes(codePath, BuildSourceCodeListingPdf(fileEntries));

        var obraPath = Path.Combine(outDir, "OBRA_CIENTIFICA_NSE_CND_INDAUTOR.pdf");
        File.WriteAllBytes(obraPath, BuildObraCientificaCompendioPdf(fileEntries));
    }

    private static byte[] BuildMemoriaDescriptivaPdf(List<(string RelPath, string Sha256, string[] Lines)>...
    {
        var pages = new List<StringBuilder>();

        // Page 1: Ficha Registral, Declaración de Autoría, Arquitectura y Módulos
        var p1 = new StringBuilder();
        double y = 742;

        DrawLine(p1, 48, y + 14, 564, y + 14, 1.4, 0.08, 0.15, 0.28);
        DrawText(p1, "F2", 8.0, 48, y + 3, "REGISTRO PUBLICO DEL DERECHO DE AUTOR (INDAUTOR - MEXICO) - ME...
        DrawText(p1, "F2", 8.0, 462, y + 3, "ORCID: 0009-0003-1109-4583");
        y -= 20;

        DrawText(p1, "F2", 13.0, 48, y, "MEMORIA DESCRIPTIVA Y ESPECIFICACION FUNCIONAL DEL SISTEMA:");
        y -= 15;
        DrawText(p1, "F2", 11.5, 48, y, "ARGON.ENGINE & NSE.PIPELINE - INGENIERIA DE SISTEMAS NARRATIVOS (...
        y -= 16;

        // Metadata Box
        DrawRect(p1, 48, y - 82, 516, 90, 0.96, 0.98, 1.0, 0.70, 0.78, 0.88);
        string[] metaLines =
        [
            "AUTOR Y TITULAR UNICO (100% DERECHOS MORALES Y PATRIMONIALES): Jair Ivan Rojas Ruiz (Jair Roj...
            "ORCID iD: https://orcid.org/0009-0003-1109-4583   |   DOI Oficial (Zenodo / CERN): https://do...
            "RAMA DE REGISTRO (LFDA ART. 13): Programa de Computo (INDAUTOR-01-002) y Obra Cientifica/Tecn...
            "FECHA DE CREACION Y ARCHIVO CRIPTOGRAFICO PRIMIGENIO: 8 de septiembre de 2026 (Caracter de la...
            "LENGUAJE Y PLATAFORMA: C# 13 sobre .NET 10.0 (Ejecucion determinista multiplataforma Windows ...
            "REPOSITORIO OFICIAL: https://github.com/jaiborojas-cloud/narrative-systems-engineering (Relea...
        ];
        double my = y - 10;
        foreach (var ml in metaLines)
```

```
        {
            DrawText(p1, "F2", 8.1, 56, my, ml);
            my -= 12.5;
        }
        y -= 98;

        DrawSection(p1, ref y, "1. DECLARACION DE ORIGINALIDAD, PRIORIDAD CRONOLOGICA Y DESAMBIGUACION");
        WritePara(p1, ref y,
        [
            "La presente obra es una creacion intelectual 100% original y primigenia de Jair Ivan Rojas Ru...
            "en espanol que originan su arquitectura (Doc 1: Atlas de la Narrativa Humana [HNA], Doc 2: Mo...
            "Doc 3: Ingenieria de Sistemas Narrativos [NSE], 63 paginas en total) fueron archivados con hu...
            "septiembre de 2026. El sistema establece formalmente la disciplina cuantitativa y computacion...
            "Multimodal (NSE) y Dinamica Narrativa Computacional (CND), fundamentada en procesamiento fisi...
            "de Shannon, control en espacio de estados e identificacion inversa de sistemas, diferenciando...
        ]);

        DrawSection(p1, ref y, "2. ARQUITECTURA DE CUATRO PILARES Y FORMULACION MATEMATICA DEL MOTOR");
        WritePara(p1, ref y,
        [
            "El sistema Argon.Engine v0.4 y Nse.Pipeline implementa los 4 pilares completos de la Ingenier...
            "  * Pilar 1 - Extraccion Multimodal (AcousticAnalyzer.cs, VisualDynamicsAnalyzer.cs, Linguist...
            "    (RMS, dBFS, VAD), matrices RGB24 de 64x64 (luminancia BT.709, Divergencia de Jensen-Shann...
            "  * Pilar 2 - Simulacion en Espacio de Estados y Atlas de Fase (NarrativeMetricsEngine.cs, At...
            "    Integra x(t+1) = (I - Gamma(t))x(t) + Gamma(t)z(t) y proyecta obras reales en el espacio ...
            "  * Pilar 3 - Control Contrafactual y Optimizacion de Montaje (CounterfactualController.cs, E...
            "    permutaciones de bloques editoriales para minimizar el sesgo normativo terminal ||x*(T)||...
            "  * Pilar 4 - Identificacion Inversa de Sistemas y Calibracion Empirica (SystemIdentifier.cs,...
            "    acoplamiento Theta = (alpha_ac, alpha_vis, alpha_ling, alpha_sync, beta_0) mediante Regre...
            "    sobre N = 308 transiciones con validacion cruzada Leave-One-Corpus-Out (R^2 fuera de mues...
        ]);

        DrawSection(p1, ref y, "3. INVENTARIO CRIPTOGRAFICO DE LOS 14 MODULOS FUENTE C# (SHA-256)");
        DrawRect(p1, 48, y - 11, 516, 13, 0.92, 0.95, 0.98, 0.2, 0.3, 0.45);
        DrawText(p1, "F2", 7.8, 52, y - 8, "Modulo / Archivo Fuente C#");
        DrawText(p1, "F2", 7.8, 225, y - 8, "Lineas");
        DrawText(p1, "F2", 7.8, 268, y - 8, "Huella Criptografica SHA-256 Verificable");
        y -= 11;

        foreach (var f in files)
        {
            y -= 10.6;
            DrawLine(p1, 48, y - 2, 564, y - 2, 0.3, 0.80, 0.84, 0.90);
            DrawText(p1, "F2", 7.3, 52, y + 1, f.RelPath);
            DrawText(p1, "F1", 7.3, 228, y + 1, f.Lines.Length.ToString(CultureInfo.InvariantCulture));
            DrawText(p1, "F4", 6.6, 268, y + 1, f.Sha256);
        }
        DrawLine(p1, 48, y - 2, 564, y - 2, 0.9, 0.1, 0.15, 0.25);

        DrawFooter(p1, 1, 1);
        pages.Add(p1);

        return AssemblePdf(pages);
    }

    private static byte[] BuildObraCientificaCompendioPdf(List<(string RelPath, string Sha256, string[] Li...
    {
        var pages = new List<StringBuilder>();

        // Page 1: Compendio Científico y Matemático (INDAUTOR-01-001 Obra Literaria / Científica)
        var p1 = new StringBuilder();
        double y = 742;

        DrawLine(p1, 48, y + 14, 564, y + 14, 1.4, 0.08, 0.15, 0.28);
        DrawText(p1, "F2", 8.0, 48, y + 3, "REGISTRO DE OBRA LITERARIA / CIENTIFICA (INDAUTOR-01-001) - CO...
        DrawText(p1, "F2", 8.0, 462, y + 3, "ORCID: 0009-0003-1109-4583");
        y -= 20;

        DrawText(p1, "F2", 12.8, 48, y, "INGENIERIA DE SISTEMAS NARRATIVOS MULTIMODALES Y DINAMICA NARRATI...
        y -= 15;
        DrawText(p1, "F2", 11.8, 48, y, "COMPUTACIONAL: ESPECIFICACION MATEMATICA Y CORPUS EXPERIMENTAL (v...
        y -= 16;

        DrawRect(p1, 48, y - 70, 516, 78, 0.96, 0.98, 1.0, 0.70, 0.78, 0.88);
        string[] meta =
        [
            "AUTOR UNICO Y TITULAR ORIGINARIO (100%): Jair Ivan Rojas Ruiz (Jair Rojas) - ORCID: https://o...
            "MODALIDAD REGISTRAL: Obra Literaria / Cientifica y Tecnica (INDAUTOR-01-001) vinculada a Prog...
            "FECHA DE PRIMERA DIVULGACION Y SELLO CRIPTOGRAFICO: 8 de septiembre de 2026 (Mexico) | DOI: 1...
            "ALCANCE INTEGRAL DE LA OBRA: Formulacion Axiomatica + Extraccion de Senales + Simulacion + Co...
            "VOLUMENES PRIMIGENIOS INCLUIDOS: Doc 1 (Atlas HNA, 19 pp.), Doc 2 (Motor NDE, 22 pp.), Doc 3 ...
        ];
        double my = y - 10;
        foreach (var m in meta)
        {
            DrawText(p1, "F2", 8.0, 56, my, m);
            my -= 12.5;
        }
        y -= 86;

```

```
        DrawSection(p1, ref y, "1. OBJETO CIENTIFICO Y AXIOMAS DE LA DISCIPLINA (NSE / CND)");
        WritePara(p1, ref y,
        [
            "La Ingenieria de Sistemas Narrativos (Narrative Systems Engineering - NSE) y su nucleo matema...
            "Computacional (Computational Narrative Dynamics - CND), creadas por Jair Ivan Rojas Ruiz el 8...
            "la primera formulacion fisico-matematica que modela obras audiovisuales como sistemas dinamic...
            "vectores multimodales u(t) = (u_ac(t), u_vis(t), u_ling(t)). La disciplina se rige por cuatro...
            "  Axioma I (Anclaje Fisico en la Senal): Toda metrica se deriva directamente de muestras acus...
            "    pixeles RGB24 de 64x64 y segmentos lexicos alineados en el tiempo, prohibiendo puntajes s...
            "  Axioma II (Unidades Fisicas e Informacionales Acotadas): La entropia cromatica, la divergen...
            "    dispersion de posturas se expresan en bits de Shannon; la energia acustica en dBFS; y el ...
            "  Axioma III (Separacion de Capas): Distincion formal entre la senal fisica observable u(t), ...
            "    normativa z(t) y la respuesta dinamica del estado cognitivo-social x(t).",
            "  Axioma IV (Reproducibilidad Computacional Exacta): Verificacion determinista de invariantes...
        ]);

        DrawSection(p1, ref y, "2. FORMULACION MATEMATICA DE LAS OCHO ECUACIONES CANONICAS NSE/CND");
        WritePara(p1, ref y,
        [
            "  (Ec. 1) Intensidad Acustica: RMS(t) = sqrt((1/M) Sum s_i^2),  L_dB(t) = 20 log10(max(RMS(t)...
            "  (Ec. 2) Flujo Visual de Jensen-Shannon: D_JS(P_t || P_{t-1}) = 0.5 D_KL(P_t || M) + 0.5 D_K...
            "  (Ec. 3) Resonancia Intermodal: Phi_sync(t) = sqrt(I_ac(t) * I_ling(t)) en [0, 1].",
            "  (Ec. 4) Energia Narrativa: E(t; alpha) = 100 * clip(alpha_ac I_ac + alpha_vis psi_vis + alp...
            "  (Ec. 5) Entropia Narrativa de Postura de Shannon: H_narr(t) = - Sum_{k,s} p_{k,s}(t) log2 p...
            "  (Ec. 6) Inercia Narrativa Institucional: I(t) = clip(0.55 A_inst(t) + 0.25 (||x(t)||_1 / K)...
            "  (Ec. 7) Ecuacion de Estado LTV y Control Optimo de Montaje: x_k(t+1) = (1 - gamma_k(t))x_k(...
            "          controlador sintetiza la permutacion optima pi* en S_n que minimiza ||x(T; pi)||_2 ...
            "  (Ec. 8) Identificacion Inversa de Sistemas (Tikhonov Ridge): Theta_hat(lambda) = (Psi^T Psi...
            "          recuperando empiricamente los pesos multimodales (alpha_ac, alpha_vis, alpha_ling, ...
        ]);

        DrawSection(p1, ref y, "3. SINTESIS DE LOS CUATRO EXPERIMENTOS DE VALIDACION EMPIRICA (EXP-001 A E...
        WritePara(p1, ref y,
        [
            "  * EXP-001 (CORPUS-001, Debate 'La Mili y la Insumision', Espana, 1994, 194.84 s, 39 ventana...
            "    de los cortes de camara reales sin falsos positivos; H_narr(T) alcanza 2.7574 bits (91.9%...
            "  * EXP-002 (Atlas Trans-Dominio, 3 obras bilingues, 513.6 s, 77 ventanas): Separa geometrica...
            "    el spot nuclear 'Daisy' de LBJ (CORPUS-002, 1964, pico E = 83.29) y el discurso 'Challeng...
            "    inercia I = 0.8921, polarizacion minima P = 0.0617), con distancia euclidiana en espacio ...
            "  * EXP-003 (Control Contrafactual y Optimizacion 6! = 720): Evalua las 720 permutaciones de ...
            "    Montaje de Equilibrio Civico pi*_eq que reduce el sesgo normativo terminal en -62.0% (0.8...
            "    cuadro, y demuestra que el 29.84% de la movilizacion del spot 'Daisy' proviene exclusivam...
            "  * EXP-004 (Identificacion Inversa de Sistemas y Generalizacion Trans-Dominio): Recupera exa...
            "    (error 0.000%), logra R^2 = 0.9999 bajo ruido gaussiano de 25 dB SNR (50 ensayos Monte Ca...
            "    alcanza R^2 fuera de muestra de 0.9999 en validacion cruzada Leave-One-Corpus-Out (LOCO) ...
        ]);

        DrawFooter(p1, 1, 1);
        pages.Add(p1);
        return AssemblePdf(pages);
    }

    private static byte[] BuildSourceCodeListingPdf(List<(string RelPath, string Sha256, string[] Lines)> ...
    {
        var pages = new List<StringBuilder>();
        var current = new StringBuilder();
        double y = StartNewCodePage(current);

        foreach (var f in files)
        {
            if (y < 110)
            {
                pages.Add(current);
                current = new StringBuilder();
                y = StartNewCodePage(current);
            }

            // File banner
            DrawRect(current, 42, y - 12, 528, 16, 0.91, 0.95, 0.99, 0.25, 0.38, 0.58);
            DrawText(current, "F2", 8.0, 48, y - 8, $"ARCHIVO FUENTE: {f.RelPath}   ({f.Lines.Length} line...
            y -= 20;

            for (int i = 0; i < f.Lines.Length; i++)
            {
                if (y < 52)
                {
                    pages.Add(current);
                    current = new StringBuilder();
                    y = StartNewCodePage(current);
                }

                var rawLine = SanitizeLatin1(f.Lines[i]);
                if (rawLine.Length > 106) rawLine = rawLine[..106] + "...";
                var formatted = $"{i + 1,4}: {rawLine}";
                DrawText(current, "F4", 6.6, 46, y, formatted);
                y -= 7.6;
            }

            y -= 8;
        }
```

```
266: 
267:         pages.Add(current);
268: 
269:         for (int p = 0; p < pages.Count; p++)
270:         {
271:             DrawFooter(pages[p], p + 1, pages.Count);
272:         }
273: 
274:         return AssemblePdf(pages);
275:     }
276: 
277:     private static double StartNewCodePage(StringBuilder sb)
278:     {
279:         double y = 748;
280:         DrawLine(sb, 42, y + 12, 570, y + 12, 1.0, 0.08, 0.15, 0.28);
281:         DrawText(sb, "F2", 7.8, 42, y + 2, "EJEMPLAR DE CODIGO FUENTE (INDAUTOR) - AUTOR Y TITULAR UNICO (...
282:         return y - 12;
283:     }
284: 
285:     private static string SanitizeLatin1(string input)
286:     {
287:         var sb = new StringBuilder(input.Length);
288:         foreach (var ch in input)
289:         {
290:             if (ch == '\t') sb.Append("    ");
291:             else if (ch >= 32 && ch <= 126) sb.Append(ch);
292:             else if (ch is '->' or '->') sb.Append("->");
293:             else if (ch is '-' or '-') sb.Append('-');
294:             else if (ch == '+/-') sb.Append("+/-");
295:             else if (ch is '"' or '"') sb.Append('"');
296:             else if (ch is ''' or ''') sb.Append('\'');
297:             else if (ch >= 160 && ch <= 255) sb.Append(ch);
298:             else sb.Append('?');
299:         }
300:         return sb.ToString();
301:     }
302: 
303:     private static void DrawSection(StringBuilder sb, ref double y, string title)
304:     {
305:         DrawText(sb, "F2", 9.2, 48, y, title);
306:         y -= 4;
307:         DrawLine(sb, 48, y, 564, y, 0.6, 0.25, 0.35, 0.50);
308:         y -= 11;
309:     }
310: 
311:     private static void WritePara(StringBuilder sb, ref double y, string[] lines)
312:     {
313:         foreach (var line in lines)
314:         {
315:             DrawText(sb, "F1", 8.4, 48, y, line);
316:             y -= 10.2;
317:         }
318:         y -= 4;
319:     }
320: 
321:     private static void DrawFooter(StringBuilder sb, int pageNum, int totalPages)
322:     {
323:         DrawLine(sb, 42, 36, 570, 36, 0.5, 0.6, 0.65, 0.75);
324:         DrawText(sb, "F1", 7.6, 42, 25, "Copyright (c) 2026 Jair Ivan Rojas Ruiz (ORCID: 0009-0003-1109-45...
325:         DrawText(sb, "F2", 7.8, 505, 25, $"Pagina {pageNum} de {totalPages}");
326:     }
327: 
328:     private static void DrawText(StringBuilder sb, string fontRef, double size, double x, double y, string...
329:     {
330:         var escaped = SanitizeLatin1(text)
331:             .Replace("\\", "\\\\")
332:             .Replace("(", "\\(")
333:             .Replace(")", "\\)");
334:         sb.AppendLine(CultureInfo.InvariantCulture, $"0.08 0.11 0.18 rg BT /{fontRef} {size:F1} Tf {x:F1} ...
335:     }
336: 
337:     private static void DrawLine(StringBuilder sb, double x1, double y1, double x2, double y2, double widt...
338:     {
339:         sb.AppendLine(CultureInfo.InvariantCulture, $"{r:F2} {g:F2} {b:F2} RG {width:F2} w {x1:F1} {y1:F1}...
340:     }
341: 
342:     private static void DrawRect(StringBuilder sb, double x, double y, double w, double h, double fr, doub...
343:     {
344:         sb.AppendLine(CultureInfo.InvariantCulture, $"{fr:F2} {fg:F2} {fb:F2} rg {sr:F2} {sg:F2} {sbCol:F2...
345:     }
346: 
347:     private static byte[] AssemblePdf(List<StringBuilder> pageStreams)
348:     {
349:         using var ms = new MemoryStream();
350:         var offsets = new List<long>();
351:         var latin1 = Encoding.GetEncoding("ISO-8859-1");
352: 
353:         void WriteAscii(string s)
354:         {
355:             var bytes = latin1.GetBytes(s);
```

```
            ms.Write(bytes, 0, bytes.Length);
        }

        WriteAscii("%PDF-1.4\n");

        offsets.Add(ms.Position);
        WriteAscii("1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n");

        int firstPageObjId = 7;
        var kids = string.Join(" ", Enumerable.Range(0, pageStreams.Count).Select(i => $"{firstPageObjId +...
        offsets.Add(ms.Position);
        WriteAscii($"2 0 obj\n<< /Type /Pages /Kids [{kids}] /Count {pageStreams.Count} >>\nendobj\n");

        string[] fontNames = ["Times-Roman", "Times-Bold", "Times-Italic", "Courier"];
        for (int f = 0; f < fontNames.Length; f++)
        {
            offsets.Add(ms.Position);
            WriteAscii($"{3 + f} 0 obj\n<< /Type /Font /Subtype /Type1 /BaseFont /{fontNames[f]} /Encoding...
        }

        for (int i = 0; i < pageStreams.Count; i++)
        {
            int pageObjId = firstPageObjId + i * 2;
            int contentObjId = pageObjId + 1;
            var contentBytes = latin1.GetBytes(pageStreams[i].ToString());

            offsets.Add(ms.Position);
            WriteAscii($"{pageObjId} 0 obj\n<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Resource...

            offsets.Add(ms.Position);
            WriteAscii($"{contentObjId} 0 obj\n<< /Length {contentBytes.Length} >>\nstream\n");
            ms.Write(contentBytes, 0, contentBytes.Length);
            WriteAscii("\nendstream\nendobj\n");
        }

        long xrefPos = ms.Position;
        WriteAscii($"xref\n0 {offsets.Count + 1}\n");
        WriteAscii("0000000000 65535 f \n");
        foreach (var off in offsets)
        {
            WriteAscii(off.ToString("D10", CultureInfo.InvariantCulture) + " 00000 n \n");
        }
        WriteAscii($"trailer\n<< /Size {offsets.Count + 1} /Root 1 0 R >>\nstartxref\n{xrefPos}\n%%EOF\n")...

        return ms.ToArray();
    }
}
```

ARCHIVO FUENTE: src/Nse.Pipeline/Program.cs (176 lineas) | SHA-256: 164abd3151d28465a093246c2e32ade1...

```
using System.Diagnostics;
using System.Globalization;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;

return await Pipeline.Run(args);

static class Pipeline
{
    static readonly JsonSerializerOptions JsonOptions = new() { WriteIndented = true, PropertyNamingPolicy...

    public static async Task<int> Run(string[] args)
    {
        if (args.Length == 0 || args.Contains("--help"))
        {
            Console.WriteLine("NSE local audiovisual profile\nUsage: dotnet run --project src/Nse.Pipeline...
            return args.Length == 0 ? 2 : 0;
        }
        try
        {
            if (args[0] is "review" or "import-review") return await ReviewWorkflow.Run(args);
            var input = Path.GetFullPath(args[0]);
            if (!File.Exists(input)) throw new ArgumentException($"Video does not exist: {input}");
            var options = new Dictionary<string, string>();
            for (var i = 1; i < args.Length; i += 2)
            {
                if (i + 1 == args.Length || !new[] { "--output", "--interval", "--max-frames", "--tools" }...
                    throw new ArgumentException($"Unknown option or missing value: {args[i]}");
                if (!options.TryAdd(args[i], args[i + 1])) throw new ArgumentException($"Duplicate option:...
            }
            var interval = double.Parse(options.GetValueOrDefault("--interval", "5"), CultureInfo.Invarian...
            var maxFrames = int.Parse(options.GetValueOrDefault("--max-frames", "120"), CultureInfo.Invari...
            if (!double.IsFinite(interval) || interval < 0.1) throw new ArgumentException("Interval must b...
            if (maxFrames is < 1 or > 10000) throw new ArgumentException("max-frames must be between 1 and...
            var output = Path.GetFullPath(options.GetValueOrDefault("--output", Path.Combine("data", "proc...
            if (Directory.Exists(output) || File.Exists(output)) throw new ArgumentException("Output must ...
            var ffmpeg = FindTool("ffmpeg", options.GetValueOrDefault("--tools"));
            var ffprobe = FindTool("ffprobe", options.GetValueOrDefault("--tools"));
            var ffmpegVersion = (await Execute(ffmpeg, "-version")).Split('\n')[0].Trim();
```

```
            var ffprobeVersion = (await Execute(ffprobe, "-version")).Split('\n')[0].Trim();
            var probeText = await Execute(ffprobe, "-v", "error", "-show_format", "-show_streams", "-of", ...
            using var probe = JsonDocument.Parse(probeText);
            var streams = probe.RootElement.GetProperty("streams").EnumerateArray().ToArray();
            var video = streams.FirstOrDefault(s => s.GetProperty("codec_type").GetString() == "video" &&
                (!s.TryGetProperty("disposition", out var d) || !d.TryGetProperty("attached_pic", out var ...
            if (video.ValueKind == JsonValueKind.Undefined) throw new ArgumentException("Input has no vide...
            var duration = double.Parse(probe.RootElement.GetProperty("format").GetProperty("duration").Ge...
            if (!double.IsFinite(duration) || duration <= 0) throw new ArgumentException("A finite, positi...
            var audio = streams.FirstOrDefault(s => s.GetProperty("codec_type").GetString() == "audio");
            var hasAudio = audio.ValueKind != JsonValueKind.Undefined;
            var videoMap = $"0:{video.GetProperty("index").GetInt32()}";
            using var source = File.OpenRead(input);
            var hash = Convert.ToHexString(await SHA256.HashDataAsync(source)).ToLowerInvariant();
            Directory.CreateDirectory(Path.Combine(output, "frames"));
            await File.WriteAllTextAsync(Path.Combine(output, "probe.json"), probeText);
            await File.WriteAllTextAsync(Path.Combine(output, "status.json"), "{\"status\":\"running\"}");
            Console.WriteLine($"Video: {duration:F2}s; audio track: {hasAudio}; output: {output}");
            if (hasAudio)
            {
                await Execute(ffmpeg, "-nostdin", "-v", "error", "-n", "-i", input, "-map", $"0:{audio.Get...
            }
            // Increase the interval for long inputs to cover the entire work within the frame budget.
            var effectiveInterval = Math.Max(interval, duration / maxFrames);
            var frameCount = Math.Min(maxFrames, (int)Math.Min(maxFrames, Math.Ceiling(duration / effectiv...
            var samples = new List<FrameSample>();
            for (var i = 0; i < frameCount; i++)
            {
                // Midpoints avoid sampling beyond the last decodable frame at the media boundary.
                var start = i * effectiveInterval;
                var end = Math.Min(duration, (i + 1) * effectiveInterval);
                var timestamp = (start + end) / 2;
                var stem = $"frames/frame-{i:D4}";
                var seek = timestamp.ToString("F6", CultureInfo.InvariantCulture);
                await Execute(ffmpeg, "-nostdin", "-v", "error", "-n", "-ss", seek, "-i", input, "-map", v...
                // Small RGB raster is kept as reproducible evidence for the color histogram.
                await Execute(ffmpeg, "-nostdin", "-v", "error", "-n", "-i", Path.Combine(output, stem + "...
                var rgb = await File.ReadAllBytesAsync(Path.Combine(output, stem + ".rgb"));
                if (rgb.Length != 64 * 64 * 3) throw new InvalidDataException("Unexpected RGB raster lengt...
                var bins = new Dictionary<int, int>();
                for (var j = 0; j < rgb.Length; j += 3)
                {
                    var bin = (rgb[j] / 32 << 6) | (rgb[j + 1] / 32 << 3) | rgb[j + 2] / 32;
                    bins[bin] = bins.GetValueOrDefault(bin) + 1;
                }
                var colors = bins.OrderByDescending(b => b.Value).ThenBy(b => b.Key).Take(5)
                    .Select(b => new ColorBin($"#{((b.Key >> 6) * 32 + 16):X2}{(((b.Key >> 3) & 7) * 32 + ...
                samples.Add(new(timestamp, start, end, stem + ".jpg", stem + ".rgb", colors));
                Console.WriteLine($"Frame {i + 1}/{frameCount}: {timestamp:F2}s");
            }
            var profile = new
            {
                schemaVersion = "0.1", createdUtc = DateTimeOffset.UtcNow,
                source = new { fileName = Path.GetFileName(input), sha256 = hash, bytes = source.Length, d...
                methods = new { ffmpegVersion, ffprobeVersion, requestedIntervalSeconds = interval, effect...
                    sampling = "One midpoint frame per interval; intervals are sampling windows, not detec...
                    color = "FFmpeg decoded JPEG resized to 64x64 with area interpolation, RGB24; 8 equal-...
                    audio = "First audio stream, downmixed mono, 16 kHz PCM16 WAV for future transcription...
                audio = new { status = hasAudio ? "extracted_untranscribed" : "no_audio_stream", path = ha...
                transcription = new { status = hasAudio ? "not_run" : "not_applicable", reason = hasAudio ...
                frames = samples,
                annotations = Array.Empty<object>(), interpretations = Array.Empty<object>(),
                limitations = new[] { "No speech, object, scene, emotion, or semantic analysis has been pe...
            };
            await File.WriteAllTextAsync(Path.Combine(output, "profile.json"), JsonSerializer.Serialize(pr...
            var csv = new StringBuilder("requested_timestamp_seconds,window_start_seconds,window_end_secon...
            foreach (var f in samples) csv.AppendLine(FormattableString.Invariant($"{f.TimestampSeconds:F6...
            await File.WriteAllTextAsync(Path.Combine(output, "timeline.csv"), csv.ToString());
            await File.WriteAllTextAsync(Path.Combine(output, "report.html"), RenderReport(Path.GetFileNam...
            await File.WriteAllTextAsync(Path.Combine(output, "status.json"), "{\"status\":\"complete\"}")...
            Console.WriteLine($"Complete: {Path.Combine(output, "report.html")}");
            return 0;
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Error: {ex.Message}\nIf a run directory was created, it is incomplet...
            return 1;
        }
    }

    static string FindTool(string name, string? explicitDirectory)
    {
        var executable = OperatingSystem.IsWindows() ? name + ".exe" : name;
        if (explicitDirectory != null)
        {
            var path = Path.GetFullPath(Path.Combine(explicitDirectory, executable));
            if (!File.Exists(path)) throw new FileNotFoundException($"Missing tool: {path}");
            return path;
        }
        var localTools = Path.Combine(Environment.CurrentDirectory, ".tools");
```

```
        if (Directory.Exists(localTools))
        {
            var match = Directory.EnumerateFiles(localTools, executable, SearchOption.AllDirectories).Orde...
            if (match != null) return match;
        }
        foreach (var directory in (Environment.GetEnvironmentVariable("PATH") ?? "").Split(Path.PathSepara...
        {
            if (string.IsNullOrWhiteSpace(directory)) continue;
            var path = Path.Combine(directory.Trim('"'), executable);
            if (File.Exists(path)) return path;
        }
        throw new FileNotFoundException($"{name} was not found. Install FFmpeg (including ffprobe), unpack...
    }

    static async Task<string> Execute(string executable, params string[] arguments)
    {
        var info = new ProcessStartInfo(executable) { RedirectStandardOutput = true, RedirectStandardError...
        foreach (var argument in arguments) info.ArgumentList.Add(argument);
        using var process = Process.Start(info) ?? throw new InvalidOperationException($"Unable to start {...
        var stdout = process.StandardOutput.ReadToEndAsync();
        var stderr = process.StandardError.ReadToEndAsync();
        using var timeout = new CancellationTokenSource(TimeSpan.FromMinutes(15));
        try { await process.WaitForExitAsync(timeout.Token); }
        catch (OperationCanceledException) { process.Kill(entireProcessTree: true); await process.WaitForE...
        var result = await stdout;
        var error = await stderr;
        if (process.ExitCode != 0) throw new InvalidOperationException($"{Path.GetFileName(executable)} ex...
        return result;
    }

    static string RenderReport(string name, double duration, bool audio, List<FrameSample> frames)
    {
        var html = new StringBuilder("<!doctype html><html lang=\"en\"><meta charset=\"utf-8\"><meta name=...
        html.Append($"<p>{System.Net.WebUtility.HtmlEncode(name)} · {duration.ToString("F2", CultureInfo.I...
        foreach (var f in frames)
        {
            html.Append(CultureInfo.InvariantCulture, $"<article><img src=\"{f.Image}\" alt=\"Frame reques...
            foreach (var c in f.Colors) html.Append(CultureInfo.InvariantCulture, $"<span style=\"backgrou...
            html.Append("</div><small>RGB bin centers; equal-width swatches. Hover for pixel fractions.</s...
        }
        return html.Append("</div></html>").ToString();
    }
}

record ColorBin(string Hex, double PixelFraction);
record FrameSample(double TimestampSeconds, double WindowStartSeconds, double WindowEndSeconds, string Ima...
```

ARCHIVO FUENTE: src/Nse.Pipeline/ReviewWorkflow.cs (143 lineas) | SHA-256: 2aa54eac26fa990259ec157e9c245ff8...

```
using System.Security.Cryptography;
using System.Text.Json;
using System.Text.Json.Nodes;
using System.Text.Json.Serialization;

static class ReviewWorkflow
{
    internal static readonly string[] Categories = ["objects", "clothing", "setting", "actions"];
    internal static readonly JsonSerializerOptions Json = new()
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase, WriteIndented = true,
        UnmappedMemberHandling = JsonUnmappedMemberHandling.Disallow
    };

    public static async Task<int> Run(string[] args)
    {
        var importing = args[0] == "import-review";
        var minimum = importing ? 3 : 2;
        if (args.Length != minimum && (args.Length != minimum + 2 || args[minimum] != "--output"))
            throw new ArgumentException("Usage: review <profile.json> [--output <new-directory>] or import...
        var profilePath = Path.GetFullPath(args[1]);
        var profileBytes = await File.ReadAllBytesAsync(profilePath);
        var profile = JsonNode.Parse(profileBytes)?.AsObject() ?? throw new InvalidDataException("Invalid ...
        var profileHash = Convert.ToHexString(SHA256.HashData(profileBytes)).ToLowerInvariant();
        var sourceHash = profile["source"]?["sha256"]?.GetValue<string>() ?? throw new InvalidDataExceptio...
        var frames = profile["frames"]?.Deserialize<List<FrameSample>>(Json) ?? throw new InvalidDataExcep...
        if (frames.Count == 0 || frames.Select(f => f.Image).Distinct().Count() != frames.Count || frames....
            throw new InvalidDataException("Profile requires uniquely referenced frames with finite timest...
        AnnotationDocument document;
        if (importing)
        {
            document = JsonSerializer.Deserialize<AnnotationDocument>(await File.ReadAllTextAsync(args[2])...
                ?? throw new InvalidDataException("Invalid annotation document.");
            Validate(document, sourceHash, profileHash, frames);
        }
        else
        {
            document = new("0.1", "visual-0.1", sourceHash, profileHash, new("", "human"),
                frames.Select(f => new FrameReview(f.Image, f.TimestampSeconds,
                    Categories.Select(c => new CategoryReview(c, "not_reviewed", [])).ToList())).ToList())...
        }
```

```
        var output = Path.GetFullPath(args.Length == minimum + 2 ? args[minimum + 1] : Path.Combine("data"...
        if (Directory.Exists(output) || File.Exists(output)) throw new ArgumentException("Output must be a...
        // Embed reviewed evidence so review/report pages remain usable when opened directly from disk.
        var images = new Dictionary<string, string>();
        var root = Path.GetDirectoryName(profilePath)!;
        foreach (var frame in frames)
        {
            var imagePath = ResolveImage(root, frame.Image);
            images[frame.Image] = "" + Convert.ToBase64String(await File.ReadAllByt...
        }
        Directory.CreateDirectory(output);
        await File.WriteAllTextAsync(Path.Combine(output, "status.json"), "{\"status\":\"running\"}");
        await File.WriteAllTextAsync(Path.Combine(output, "annotations.json"), JsonSerializer.Serialize(do...
        await File.WriteAllTextAsync(Path.Combine(output, "review.html"), ReviewPage.Render(document, imag...
        if (importing)
        {
            // Keep the extracted measurements and their paths intact; record the source profile location.
            profile["annotationReview"] = JsonSerializer.SerializeToNode(new
            {
                document.SchemaVersion, document.CodebookVersion, document.SourceSha256, document.ProfileS...
                document.Reviewer, importedUtc = DateTimeOffset.UtcNow,
                measurementProfilePath = profilePath,
                evidenceBaseDirectory = root,
                status = document.Frames.All(f => f.Categories.All(c => c.Status != "not_reviewed")) ? "co...
            }, Json);
            profile["annotations"] = JsonSerializer.SerializeToNode(document.Frames, Json);
            if (profile["limitations"] is JsonArray limitations)
                for (var i = 0; i < limitations.Count; i++)
                    if (limitations[i]?.GetValue<string>() == "No speech, object, scene, emotion, or seman...
                        limitations[i] = "The measurement stage performed no speech, object, scene, emotio...
            await File.WriteAllTextAsync(Path.Combine(output, "profile.reviewed.json"), profile.ToJsonStri...
            await File.WriteAllTextAsync(Path.Combine(output, "report.html"), ReviewPage.Render(document, ...
        }
        await File.WriteAllTextAsync(Path.Combine(output, "status.json"), "{\"status\":\"complete\"}");
        Console.WriteLine($"{(importing ? "Imported review" : "Review workspace ready")}: {output}");
        return 0;
    }

    static string ResolveImage(string root, string relative)
    {
        if (string.IsNullOrWhiteSpace(relative) || Path.IsPathRooted(relative)) throw new InvalidDataExcep...
        var full = Path.GetFullPath(Path.Combine(root, relative));
        var prefix = Path.GetFullPath(root) + Path.DirectorySeparatorChar;
        if (!full.StartsWith(prefix, OperatingSystem.IsWindows() ? StringComparison.OrdinalIgnoreCase : St...
            throw new InvalidDataException("Frame image escapes the profile directory.");
        // Reject links so imported profiles cannot pull unrelated files into the embedded review page.
        for (var current = full; current != Path.GetFullPath(root); current = Path.GetDirectoryName(curren...
            if ((File.GetAttributes(current) & FileAttributes.ReparsePoint) != 0) throw new InvalidDataExc...
        return full;
    }

    static void Require(bool condition, string message)
    {
        if (!condition) throw new InvalidDataException(message);
    }

    static bool Text(string? value, int max) => !string.IsNullOrWhiteSpace(value) && value.Length <= max;

    static void Validate(AnnotationDocument doc, string sourceHash, string profileHash, List<FrameSample> ...
    {
        Require(doc.SchemaVersion == "0.1" && doc.CodebookVersion == "visual-0.1", "Unsupported annotation...
        Require(doc.SourceSha256 == sourceHash && doc.ProfileSha256 == profileHash, "Annotations belong to...
        Require(doc.Reviewer != null && Text(doc.Reviewer.Name, 200) && new[] { "human", "ai_assisted" }.C...
        Require(doc.Frames != null && doc.Frames.Count == frames.Count, "Every sampled frame must appear e...
        var seen = new HashSet<string>();
        foreach (var frame in doc.Frames!)
        {
            Require(frame != null && frame.FrameImage != null && seen.Add(frame.FrameImage), "Duplicate or...
            var expected = frames.SingleOrDefault(f => f.Image == frame!.FrameImage);
            Require(expected != null && frame!.TimestampSeconds == expected.TimestampSeconds, "Unknown fra...
            Require(frame!.Categories != null && frame.Categories.Count == Categories.Length, "All four ca...
            var categories = new HashSet<string>();
            foreach (var category in frame.Categories!)
            {
                Require(category != null && Categories.Contains(category.Category) && categories.Add(categ...
                Require(new[] { "not_reviewed", "none_visible", "observed", "uncertain" }.Contains(categor...
                Require(category.Observations != null && category.Observations.Count <= 100, "An observati...
                Require(category.Status is "observed" or "uncertain" ? category.Observations!.Count > 0 : ...
                    "Observed/uncertain categories require evidence; not_reviewed/none_visible categories ...
                foreach (var observation in category.Observations!)
                {
                    Require(observation != null && Text(observation.Label, 200) && Text(observation.Eviden...
                    Require(new[] { "clear", "uncertain" }.Contains(observation!.Certainty), "Certainty mu...
                }
                if (category.Status == "uncertain") Require(category.Observations!.Any(o => o.Certainty ==...
                if (category.Status == "observed") Require(category.Observations!.All(o => o.Certainty == ...
            }
        }
    }
}
```

```
132:
133: sealed record AnnotationDocument(
134:     [property: JsonRequired] string SchemaVersion,
135:     [property: JsonRequired] string CodebookVersion,
136:     [property: JsonRequired] string SourceSha256,
137:     [property: JsonRequired] string ProfileSha256,
138:     [property: JsonRequired] Reviewer Reviewer,
139:     [property: JsonRequired] List<FrameReview> Frames);
140: sealed record Reviewer([property: JsonRequired] string Name, [property: JsonRequired] string Kind);
141: sealed record FrameReview([property: JsonRequired] string FrameImage, [property: JsonRequired] double Time...
142: sealed record CategoryReview([property: JsonRequired] string Category, [property: JsonRequired] string Sta...
143: sealed record Observation([property: JsonRequired] string Label, [property: JsonRequired] string Evidence,...
```

Copyright (c) 2026 Jair Ivan Rojas Ruiz (ORCID: 0009-0003-1109-4583 | DOI: 10.5281/zenodo.23987656) - Registro INDAUTOR